Process & Decision Documentation

This process and documentation document is to show my process for creating the side quest. My goal was to create a calming scene for the player to scroll through and feel a sense of relaxation. The key decision I made was to make the vibe feel like a nighttime in a field stargazing, this really resonated with me as someone who grew up in the country and had many nights like this looking up at the stars, the moon, and sometimes cattle.

Role-Based Process Evidence

- Received a really strong first draft from chatgpt, felt inspired to take it from there and make adjustments
- Committed to the night atmosphere and added elements of my personal life
- Created a score element so the player can stay more focused while scrolling through the scene

Entry Header

Name: Cameron Emsley-Drummond
Role(s): Lead
Primary responsibility for this work: For completing it

Goal of Work Session

My goal for the work session is to create an interactive story that the player can experience, where they choose how it goes.
Tools, Resources, or Inputs Used

- p5.js
- p5.js web editor
- GenAI (ChatGpt)
- Self Playtesting
- Lecture materials

GenAI Documentation

Everyone must complete this section. If not GenAI was used, write, “No GenAI used for this task.” When GenAI is not used, process evidence should still demonstrate iteration, revision, or development over time.
Because GenAI can closely mimic human-created work, instructors or TAs may occasionally request additional process evidence to confirm non-use. This may include original working files (e.g., an illustrator file), intermediate drafts, or a brief check-in with a TA to walk through your process.

These requests are not an assumption of misconduct. They are part of ensuring academic integrity in an environment where distinguishing between human-created and AI-generated work is increasingly difficult.

If GenAI was used (keep each response as brief as possible):

Date Used: over the course of adjusting the game
Tool Disclosure: ChatGPT 5.2 (Thinking)
Purpose of Use: Code generation, comments, and debugging
Summary of Interaction: Wrote the code and comments, including the camera movements, the control scheme,
Human Decision Point(s): I added more visual elements to the sketch, like cows, grass, the moon, houses and silos to commit to the field aesthetic
Integrity & Verification Note: Verified through playtesting through web editor
Scope of GenAI Use: Main parts genAI did not contribute to, is what I used to build off what it gave me. It gave me a really strong rough draft, that I proceeded to transform into the idea of a peaceful night in the field
Limitations or Misfires: A big limitation I had was getting the moon right. It continually misunderstood me, I wanted the moon stay along the skyline and move as the camera did but it messed up a few times and required me to adjust things and make stronger descriptions of what I want

Summary of Process (Human + Tool)

- Added a point system using shiny gold coins
- Added farmstyle atmosphere including cows, silos, farmhouses, grass
- Added a moon to highlight the night atmosphere

Decision Points & Trade-offs

Describe one or two key decisions you made:
A key decision I made was creating the setting and atmosphere for the world. The original prompt I gave ChatGPT called for a meditative and relaxing experience, and included the required camera instructions. It gave a very strong first attempt, but from there I saw potential. I wanted to create a visceral setting, the night sky and the stars made me think of a nice farm field. From there I added visual flair to sell the setting, like cattle, silos, grass, and more.

Verification & Judgement

Explain how you evaluated whether your decision or change was appropriate:

- I just kept playtesting it in a web based p5js editor to see if things worked or not or if they did not look the way I intended

Limitations, Dead Ends, or Open Questions

I had a few issues with chatGPT, this was the first time I have used the thinking version of the software, I wanted to see what it could do. It was poor at understanding exactly what I wanted the moon to look like. It also had a bug at the beginning I had to fix with it, it did not know what the problem was, so I had to solve it by way of process of elimination. Otherwise it was very strong when handling the camera elements

Appendix

Please include a full transcript of your conversation with the GenAI. GenAI transcripts are used for transparency and verification only and are not assessed for writing quality or completeness.

CHATGPT SHARE LINK: https://chatgpt.com/share/699e68de-deb8-8006-9de9-2f40d840dac6

You said:

in p5js create a reflective or meditative camera experience that scrolls through a world larger than the screen. Use pacing and motion to evoke emotion

ChatGPT said:

```
/*
Meditative Camera Scroll — p5.js (single-file)
Drop into https://editor.p5js.org

Goal: a reflective, slow-moving camera that scrolls through a world larger
than the screen.
Pacing + motion are tuned to feel calm (ease, drift, “breathing” light,
quiet parallax).

Controls:
- Move focus: Arrow keys / WASD (gentle nudge)
- Pause drift: hold SPACE
- Reset: R
- Toggle minimal UI text: H
```

```
*/

let cam, world;
let showHUD = true;

function setup() {
 createCanvas(900, 520);
 pixelDensity(1);

 world = new World();
 cam = new CameraRig();

 textFont("Georgia");
}

function draw() {
 background(6, 8, 12);

 // Update world + camera pacing
 world.update();
 cam.update(world);

 // Draw scene with camera transform
 push();
 cam.apply();

 world.drawFar();   // distant parallax
 world.drawMid();   // main terrain
 world.drawNear();  // near silhouettes
 world.drawFloaters(); // drifting particles in world-space

 pop();

 // Screen-space mood overlays
 world.drawBreathOverlay();
 world.drawVignette();
 world.drawGrain();
```

```
 if (showHUD) drawHUD();
}

// -------------------- Input --------------------
function keyPressed() {
 if (key === "h" || key === "H") showHUD = !showHUD;
 if (key === "r" || key === "R") {
   world = new World();
   cam = new CameraRig();
 }
}

// -------------------- HUD --------------------
function drawHUD() {
 const pad = 16;
 noStroke();
 fill(0, 90);
 rect(pad, pad, 365, 72, 14);

 fill(230, 220);
 textSize(14);
 textAlign(LEFT, TOP);
 text(
   "Meditative camera drift\nWASD / Arrows: nudge focus  •  Hold SPACE:
pause drift\nH: hide text  •  R: reset",
   pad + 12,
   pad + 10
 );
}

// ==================== Camera ====================
class CameraRig {
 constructor() {
   this.pos = createVector(0, 0);
   this.vel = createVector(0, 0);

   // The “focus” point the camera tries to keep near center
   this.focus = createVector(0, 0);
```

```
  // Gentle drift target that wanders across the world
  this.drift = createVector(0, 0);

  // Timing seeds
  this.t = random(1000);
}

update(worldRef) {
  const dt = 1 / max(1, frameRate());

  // --- User nudge (soft, not game-y) ---
  const nudge = createVector(0, 0);
  const push = 0.9;

  if (keyIsDown(LEFT_ARROW) || keyIsDown(65)) nudge.x -= push;  // A
  if (keyIsDown(RIGHT_ARROW) || keyIsDown(68)) nudge.x += push; // D
  if (keyIsDown(UP_ARROW) || keyIsDown(87)) nudge.y -= push;    // W
  if (keyIsDown(DOWN_ARROW) || keyIsDown(83)) nudge.y += push;  // S

  // Drift pause
  const paused = keyIsDown(32); // SPACE

  // --- Drift logic: slow wandering through a large space ---
  this.t += paused ? 0 : 0.0035;
  const wanderX = (noise(this.t) - 0.5) * 2;
  const wanderY = (noise(this.t + 50.0) - 0.5) * 2;

  // Drift target gently slides forward; mostly horizontal travel
  if (!paused) {
    this.drift.x += 0.55 + wanderX * 0.22;
    this.drift.y = sin(this.t * 1.2) * 55 + wanderY * 35;
  }

  // Keep the drift within a big "soft" world span
  const softClamp = worldRef.softBounds;
  this.drift.x = constrain(this.drift.x, softClamp.minX, softClamp.maxX);
  this.drift.y = constrain(this.drift.y, softClamp.minY, softClamp.maxY);
```

```
    // Focus is drift + user nudge (nudge decays)
    this.focus.add(nudge.mult(2.2));
    this.focus.lerp(this.drift, 0.035);

    // Camera follows focus with easing (emotional pacing)
    // A slow spring: comfortable, not snappy.
    const desired = p5.Vector.sub(this.focus, createVector(width * 0.5,
height * 0.52));
    const accel = p5.Vector.sub(desired, this.pos).mult(0.028);
    this.vel.add(accel);
    this.vel.mult(0.86); // damping
    this.pos.add(this.vel);

    // Subtle "handheld" float (very tiny)
    const floatX = (noise(this.t + 200) - 0.5) * 1.8;
    const floatY = (noise(this.t + 300) - 0.5) * 1.8;
    this.pos.x += floatX;
    this.pos.y += floatY;
  }

  apply() {
    // translate world by negative camera position
    translate(-this.pos.x, -this.pos.y);
  }
}

// ==================== World ====================
class World {
  constructor() {
    this.seed = floor(random(1e9));
    noiseSeed(this.seed);
    randomSeed(this.seed);

    // Soft bounds for drifting (world is "larger than screen")
    this.softBounds = {
      minX: -900,
      maxX: 6800,
```

```
    minY: -650,
    maxY: 650
  };

  // "Breathing" overlay timing
  this.breathT = random(1000);

  // Particles (world-space)
  this.floaters = [];
  for (let i = 0; i < 220; i++) {
    this.floaters.push(this.makeFloater());
  }

  // Stars / dust in far layer
  this.stars = [];
  for (let i = 0; i < 420; i++) {
    this.stars.push({
      x: random(this.softBounds.minX, this.softBounds.maxX),
      y: random(this.softBounds.minY, this.softBounds.maxY),
      s: random(0.5, 2.2),
      a: random(70, 170)
    });
  }
}

makeFloater() {
  return {
    x: random(this.softBounds.minX, this.softBounds.maxX),
    y: random(-260, 260),
    r: random(1.2, 3.8),
    phase: random(TWO_PI),
    drift: random(0.2, 0.55),
    glow: random(40, 120)
  };
}

update() {
  this.breathT += 0.01;
```

```
  // Floaters slowly drift upward, re-spawn softly
  for (let p of this.floaters) {
    const w = noise(p.x * 0.0012, p.y * 0.0012, frameCount * 0.002) -
0.5;
    p.x += w * 0.5;
    p.y += sin(frameCount * 0.004 + p.phase) * 0.22 - p.drift;

    // If too high, respawn lower in the world
    if (p.y < -520) {
      p.y = random(120, 420);
      p.x = random(this.softBounds.minX, this.softBounds.maxX);
    }
  }
}

// ---------- Layers ----------
drawFar() {
  // Far parallax: stars + soft aurora band
  push();

  // Parallax factor depends on camera; but we're already in world-space.
  // We'll just draw wide enough and let it scroll.
  noStroke();

  // Stars
  for (let st of this.stars) {
    fill(210, 220, 245, st.a);
    circle(st.x, st.y, st.s);
  }

  // Aurora ribbon (very subtle)
  for (let x = this.softBounds.minX; x < this.softBounds.maxX; x += 22) {
    const n = noise(x * 0.0007, this.seed * 0.00001);
    const y = -250 + n * 120 + sin(x * 0.002 + this.breathT * 0.35) * 18;
    const a = 22 + n * 28;
    fill(110, 160, 190, a);
    ellipse(x, y, 120, 70);
```

```
    }

    pop();
  }

  drawMid() {
    // Mid: rolling terrain + reflective water line
    push();

    const horizonY = 80;
    const groundBase = 260;

    // Sky gradient slab (world-space)
    noStroke();
    for (let y = -400; y < groundBase; y += 2) {
      const t = map(y, -400, groundBase, 0, 1);
      fill(10 + t * 18, 14 + t * 18, 22 + t * 24, 255);
      rect(this.softBounds.minX, y, this.softBounds.maxX -
this.softBounds.minX, 2);
    }

    // Distant hills silhouette
    fill(12, 16, 22, 220);
    beginShape();
    vertex(this.softBounds.minX, groundBase);
    for (let x = this.softBounds.minX; x <= this.softBounds.maxX; x += 18)
{
      const n = noise(x * 0.0009, 10.0);
      const y = horizonY + n * 140;
      vertex(x, y);
    }
    vertex(this.softBounds.maxX, groundBase);
    endShape(CLOSE);

    // Main ground
    fill(8, 10, 14, 255);
    beginShape();
    vertex(this.softBounds.minX, height + 900);
```

```
  for (let x = this.softBounds.minX; x <= this.softBounds.maxX; x += 16)
{
    const n1 = noise(x * 0.0013, 100.0);
    const n2 = noise(x * 0.0035, 200.0);
    const y = 240 + n1 * 210 + n2 * 45;
    vertex(x, y);
  }
  vertex(this.softBounds.maxX, height + 900);
  endShape(CLOSE);

  // Water line (reflective band)
  const waterY = 300;
  for (let x = this.softBounds.minX; x < this.softBounds.maxX; x += 10) {
    const ripple = sin(x * 0.02 + this.breathT * 1.2) * 2 + (noise(x *
0.01, this.breathT * 0.2) - 0.5) * 3;
    const a = 14 + (noise(x * 0.003, 400) * 22);
    stroke(180, 210, 230, a);
    line(x, waterY + ripple, x + 8, waterY + ripple + 0.4);
  }

  // Occasional "memory posts" (tiny landmarks)
  noStroke();
  for (let i = 0; i < 26; i++) {
    const px = i * 280 + 90 + sin(i * 2.1) * 35;
    const py = this.sampleGroundY(px) - 8;
    const tall = 18 + noise(i * 10.1) * 55;

    fill(18, 22, 30, 220);
    rect(px - 2, py - tall, 4, tall, 2);

    // soft glow dot
    fill(200, 220, 240, 40);
    circle(px, py - tall - 6, 18);
  }

  pop();
}
```

```
drawNear() {
  // Near parallax silhouettes: sparse trees / reeds
  push();

  for (let i = 0; i < 42; i++) {
    const x = i * 160 + 40 + sin(i * 0.9) * 20;
    const gy = this.sampleGroundY(x);
    const sway = sin(frameCount * 0.01 + i) * 2.5;

    // trunk
    stroke(12, 14, 18, 210);
    strokeWeight(3);
    line(x, gy - 8, x + sway, gy - 70);

    // canopy
    noStroke();
    fill(10, 12, 16, 240);
    ellipse(x + sway, gy - 82, 36, 24);
    ellipse(x + sway + 16, gy - 76, 22, 16);
    ellipse(x + sway - 14, gy - 74, 20, 14);
  }

  pop();
}

drawFloaters() {
  push();
  noStroke();
  for (let p of this.floaters) {
    const tw = sin(frameCount * 0.02 + p.phase) * 0.4 + 0.6;
    fill(190, 210, 230, p.glow * tw);
    circle(p.x, p.y, p.r * (0.8 + tw));
  }
  pop();
}

// ---------- Screen-space overlays ----------
drawBreathOverlay() {
```

```
  // A slow inhale/exhale light band: makes the scene feel alive without
"doing" anything
  const breath = (sin(this.breathT * 0.35) * 0.5 + 0.5); // 0..1
  const alpha = 22 + breath * 26;

  noStroke();
  fill(90, 140, 170, alpha);
  rect(0, 0, width, height);
}

drawVignette() {
  // Soft vignette using concentric rectangles (fast + stable)
  push();
  noFill();
  for (let i = 0; i < 18; i++) {
    const t = i / 17;
    const a = 18 * t;
    stroke(0, 0, 0, a);
    const m = t * 70;
    rect(m, m, width - 2 * m, height - 2 * m, 28);
  }
  pop();
}

drawGrain() {
  // Gentle film grain
  loadPixels();
  const d = pixelDensity();
  const w = width * d;
  const h = height * d;

  // Keep it light: only touch a subset of pixels
  for (let y = 0; y < h; y += 2) {
    for (let x = 0; x < w; x += 2) {
      const idx = 4 * (y * w + x);
      const g = (random() - 0.5) * 10;
      pixels[idx + 0] = constrain(pixels[idx + 0] + g, 0, 255);
      pixels[idx + 1] = constrain(pixels[idx + 1] + g, 0, 255);
```

```
      pixels[idx + 2] = constrain(pixels[idx + 2] + g, 0, 255);
    }
  }
  updatePixels();
}

// ---------- Helpers ----------
sampleGroundY(x) {
  // Match the ground function used in drawMid
  const n1 = noise(x * 0.0013, 100.0);
  const n2 = noise(x * 0.0035, 200.0);
  return 240 + n1 * 210 + n2 * 45;
}
}
```

Thought for 7s

```
let grainLayer;
```

```
grainLayer = createGraphics(width, height);

makeStaticGrain(grainLayer, 18); // 18 = intensity (try 10-30)
```

```
function makeStaticGrain(pg, intensity = 18) {
 pg.pixelDensity(1);
 pg.loadPixels();

 for (let y = 0; y < pg.height; y++) {
   for (let x = 0; x < pg.width; x++) {
     const idx = 4 * (y * pg.width + x);

     // stable “grain” value per pixel using noise (not random per frame)
     const n = noise(x * 0.08, y * 0.08);
     const g = (n - 0.5) * 2 * intensity; // -intensity .. +intensity

     // store grain as light/dark in RGB, keep alpha low-ish
     pg.pixels[idx + 0] = 128 + g;
     pg.pixels[idx + 1] = 128 + g;
     pg.pixels[idx + 2] = 128 + g;
     pg.pixels[idx + 3] = 35; // alpha baked in
   }
 }

 pg.updatePixels();
}
```

```
drawGrain() {
 // draw the precomputed grain (stable, no flicker)
 push();
```

```
 blendMode(OVERLAY); // or try SOFT_LIGHT / MULTIPLY
 image(grainLayer, 0, 0);
 blendMode(BLEND);
 pop();
}
```

```
/*
Meditative Camera Scroll — p5.js (single-file)
Drop into https://editor.p5js.org

Changes:
- Removed flickering grain by generating a STATIC grain layer once in
setup(),
 then drawing it each frame (no per-frame random pixel writes).

Controls:
- Move focus: Arrow keys / WASD (gentle nudge)
- Pause drift: hold SPACE
- Reset: R
- Toggle minimal UI text: H
*/

let cam, world;
let showHUD = true;
```

```
// Static, precomputed grain to prevent flicker
let grainLayer;

function setup() {
 createCanvas(900, 520);
 pixelDensity(1);

 world = new World();
 cam = new CameraRig();

 textFont("Georgia");

 // Build static grain ONCE (stable background)
 grainLayer = createGraphics(width, height);
 makeStaticGrain(grainLayer, 18); // intensity: try 10-30
}

function draw() {
 background(6, 8, 12);

 // Update world + camera pacing
 world.update();
 cam.update(world);

 // Draw scene with camera transform
 push();
 cam.apply();

 world.drawFar();      // distant parallax
 world.drawMid();      // main terrain
 world.drawNear();     // near silhouettes
 world.drawFloaters(); // drifting particles in world-space

 pop();

 // Screen-space mood overlays
 world.drawBreathOverlay();
```

```
 world.drawVignette();
 world.drawGrain(); // now stable

 if (showHUD) drawHUD();
}

// -------------------- Input --------------------
function keyPressed() {
 if (key === "h" || key === "H") showHUD = !showHUD;
 if (key === "r" || key === "R") {
   world = new World();
   cam = new CameraRig();

   // rebuild grain so reset still works consistently
   grainLayer = createGraphics(width, height);
   makeStaticGrain(grainLayer, 18);
 }
}

// -------------------- HUD --------------------
function drawHUD() {
 const pad = 16;
 noStroke();
 fill(0, 90);
 rect(pad, pad, 365, 72, 14);

 fill(230, 220);
 textSize(14);
 textAlign(LEFT, TOP);
 text(
   "Meditative camera drift\nWASD / Arrows: nudge focus  •  Hold SPACE:
pause drift\nH: hide text  •  R: reset",
   pad + 12,
   pad + 10
 );
}

// -------------------- Static Grain Builder --------------------
```

```
function makeStaticGrain(pg, intensity = 18) {
 pg.pixelDensity(1);
 pg.loadPixels();

 // Use noise so the pattern is stable (not random each frame)
 for (let y = 0; y < pg.height; y++) {
   for (let x = 0; x < pg.width; x++) {
     const idx = 4 * (y * pg.width + x);

     const n = noise(x * 0.08, y * 0.08);
     const g = (n - 0.5) * 2 * intensity; // -intensity .. +intensity

     pg.pixels[idx + 0] = 128 + g;
     pg.pixels[idx + 1] = 128 + g;
     pg.pixels[idx + 2] = 128 + g;
     pg.pixels[idx + 3] = 35; // alpha baked in (subtle)
   }
 }

 pg.updatePixels();
}

// ==================== Camera ====================
class CameraRig {
 constructor() {
   this.pos = createVector(0, 0);
   this.vel = createVector(0, 0);

   // The "focus" point the camera tries to keep near center
   this.focus = createVector(0, 0);

   // Gentle drift target that wanders across the world
   this.drift = createVector(0, 0);

   // Timing seeds
   this.t = random(1000);
 }
```

```
update(worldRef) {
  // --- User nudge (soft, not game-y) ---
  const nudge = createVector(0, 0);
  const push = 0.9;

  if (keyIsDown(LEFT_ARROW) || keyIsDown(65)) nudge.x -= push;  // A
  if (keyIsDown(RIGHT_ARROW) || keyIsDown(68)) nudge.x += push; // D
  if (keyIsDown(UP_ARROW) || keyIsDown(87)) nudge.y -= push;    // W
  if (keyIsDown(DOWN_ARROW) || keyIsDown(83)) nudge.y += push;  // S

  // Drift pause
  const paused = keyIsDown(32); // SPACE

  // --- Drift logic: slow wandering through a large space ---
  this.t += paused ? 0 : 0.0035;
  const wanderX = (noise(this.t) - 0.5) * 2;
  const wanderY = (noise(this.t + 50.0) - 0.5) * 2;

  // Drift target gently slides forward; mostly horizontal travel
  if (!paused) {
    this.drift.x += 0.55 + wanderX * 0.22;
    this.drift.y = sin(this.t * 1.2) * 55 + wanderY * 35;
  }

  // Keep the drift within a big "soft" world span
  const softClamp = worldRef.softBounds;
  this.drift.x = constrain(this.drift.x, softClamp.minX, softClamp.maxX);
  this.drift.y = constrain(this.drift.y, softClamp.minY, softClamp.maxY);

  // Focus is drift + user nudge (nudge decays)
  this.focus.add(nudge.mult(2.2));
  this.focus.lerp(this.drift, 0.035);

  // Camera follows focus with easing (emotional pacing)
  const desired = p5.Vector.sub(
    this.focus,
    createVector(width * 0.5, height * 0.52)
  );
```

```
    const accel = p5.Vector.sub(desired, this.pos).mult(0.028);
    this.vel.add(accel);
    this.vel.mult(0.86); // damping
    this.pos.add(this.vel);

    // Subtle "handheld" float (very tiny)
    const floatX = (noise(this.t + 200) - 0.5) * 1.8;
    const floatY = (noise(this.t + 300) - 0.5) * 1.8;
    this.pos.x += floatX;
    this.pos.y += floatY;
  }

  apply() {
    // translate world by negative camera position
    translate(-this.pos.x, -this.pos.y);
  }
}

// ==================== World ====================
class World {
  constructor() {
    this.seed = floor(random(1e9));
    noiseSeed(this.seed);
    randomSeed(this.seed);

    // Soft bounds for drifting (world is "larger than screen")
    this.softBounds = {
      minX: -900,
      maxX: 6800,
      minY: -650,
      maxY: 650
    };

    // "Breathing" overlay timing
    this.breathT = random(1000);

    // Particles (world-space)
    this.floaters = [];
```

```
  for (let i = 0; i < 220; i++) {
    this.floaters.push(this.makeFloater());
  }

  // Stars / dust in far layer
  this.stars = [];
  for (let i = 0; i < 420; i++) {
    this.stars.push({
      x: random(this.softBounds.minX, this.softBounds.maxX),
      y: random(this.softBounds.minY, this.softBounds.maxY),
      s: random(0.5, 2.2),
      a: random(70, 170)
    });
  }
}

makeFloater() {
  return {
    x: random(this.softBounds.minX, this.softBounds.maxX),
    y: random(-260, 260),
    r: random(1.2, 3.8),
    phase: random(TWO_PI),
    drift: random(0.2, 0.55),
    glow: random(40, 120)
  };
}

update() {
  this.breathT += 0.01;

  // Floaters slowly drift upward, re-spawn softly
  for (let p of this.floaters) {
    const w = noise(p.x * 0.0012, p.y * 0.0012, frameCount * 0.002) -
0.5;
    p.x += w * 0.5;
    p.y += sin(frameCount * 0.004 + p.phase) * 0.22 - p.drift;

    // If too high, respawn lower in the world
```

```
    if (p.y < -520) {
      p.y = random(120, 420);
      p.x = random(this.softBounds.minX, this.softBounds.maxX);
    }
  }
}

// ---------- Layers ----------
drawFar() {
  // Far parallax: stars + soft aurora band
  push();
  noStroke();

  // Stars
  for (let st of this.stars) {
    fill(210, 220, 245, st.a);
    circle(st.x, st.y, st.s);
  }

  // Aurora ribbon (very subtle)
  for (let x = this.softBounds.minX; x < this.softBounds.maxX; x += 22) {
    const n = noise(x * 0.0007, this.seed * 0.00001);
    const y = -250 + n * 120 + sin(x * 0.002 + this.breathT * 0.35) * 18;
    const a = 22 + n * 28;
    fill(110, 160, 190, a);
    ellipse(x, y, 120, 70);
  }

  pop();
}

drawMid() {
  // Mid: rolling terrain + reflective water line
  push();

  const horizonY = 80;
  const groundBase = 260;
```

```
  // Sky gradient slab (world-space)
  noStroke();
  for (let y = -400; y < groundBase; y += 2) {
    const t = map(y, -400, groundBase, 0, 1);
    fill(10 + t * 18, 14 + t * 18, 22 + t * 24, 255);
    rect(this.softBounds.minX, y, this.softBounds.maxX -
this.softBounds.minX, 2);
  }

  // Distant hills silhouette
  fill(12, 16, 22, 220);
  beginShape();
  vertex(this.softBounds.minX, groundBase);
  for (let x = this.softBounds.minX; x <= this.softBounds.maxX; x += 18)
{
    const n = noise(x * 0.0009, 10.0);
    const y = horizonY + n * 140;
    vertex(x, y);
  }
  vertex(this.softBounds.maxX, groundBase);
  endShape(CLOSE);

  // Main ground
  fill(8, 10, 14, 255);
  beginShape();
  vertex(this.softBounds.minX, height + 900);
  for (let x = this.softBounds.minX; x <= this.softBounds.maxX; x += 16)
{
    const n1 = noise(x * 0.0013, 100.0);
    const n2 = noise(x * 0.0035, 200.0);
    const y = 240 + n1 * 210 + n2 * 45;
    vertex(x, y);
  }
  vertex(this.softBounds.maxX, height + 900);
  endShape(CLOSE);

  // Water line (reflective band)
  const waterY = 300;
```

```
  for (let x = this.softBounds.minX; x < this.softBounds.maxX; x += 10) {
    const ripple =
      sin(x * 0.02 + this.breathT * 1.2) * 2 +
      (noise(x * 0.01, this.breathT * 0.2) - 0.5) * 3;
    const a = 14 + noise(x * 0.003, 400) * 22;
    stroke(180, 210, 230, a);
    line(x, waterY + ripple, x + 8, waterY + ripple + 0.4);
  }

  // Occasional "memory posts" (tiny landmarks)
  noStroke();
  for (let i = 0; i < 26; i++) {
    const px = i * 280 + 90 + sin(i * 2.1) * 35;
    const py = this.sampleGroundY(px) - 8;
    const tall = 18 + noise(i * 10.1) * 55;

    fill(18, 22, 30, 220);
    rect(px - 2, py - tall, 4, tall, 2);

    // soft glow dot
    fill(200, 220, 240, 40);
    circle(px, py - tall - 6, 18);
  }

  pop();
}

drawNear() {
  // Near silhouettes: sparse trees / reeds
  push();

  for (let i = 0; i < 42; i++) {
    const x = i * 160 + 40 + sin(i * 0.9) * 20;
    const gy = this.sampleGroundY(x);
    const sway = sin(frameCount * 0.01 + i) * 2.5;

    // trunk
    stroke(12, 14, 18, 210);
```

```
    strokeWeight(3);
    line(x, gy - 8, x + sway, gy - 70);

    // canopy
    noStroke();
    fill(10, 12, 16, 240);
    ellipse(x + sway, gy - 82, 36, 24);
    ellipse(x + sway + 16, gy - 76, 22, 16);
    ellipse(x + sway - 14, gy - 74, 20, 14);
  }

  pop();
}

drawFloaters() {
  push();
  noStroke();
  for (let p of this.floaters) {
    const tw = sin(frameCount * 0.02 + p.phase) * 0.4 + 0.6;
    fill(190, 210, 230, p.glow * tw);
    circle(p.x, p.y, p.r * (0.8 + tw));
  }
  pop();
}

// ---------- Screen-space overlays ----------
drawBreathOverlay() {
  // Slow inhale/exhale light band
  const breath = sin(this.breathT * 0.35) * 0.5 + 0.5; // 0..1
  const alpha = 22 + breath * 26;

  noStroke();
  fill(90, 140, 170, alpha);
  rect(0, 0, width, height);
}

drawVignette() {
  // Soft vignette using concentric rectangles
```

```
  push();
  noFill();
  for (let i = 0; i < 18; i++) {
    const t = i / 17;
    const a = 18 * t;
    stroke(0, 0, 0, a);
    const m = t * 70;
    rect(m, m, width - 2 * m, height - 2 * m, 28);
  }
  pop();
}

drawGrain() {
  // Draw the precomputed grain (stable, no flicker)
  push();
  blendMode(OVERLAY); // try SOFT_LIGHT / MULTIPLY if you want
  image(grainLayer, 0, 0);
  blendMode(BLEND);
  pop();
}

// ---------- Helpers ----------
sampleGroundY(x) {
  const n1 = noise(x * 0.0013, 100.0);
  const n2 = noise(x * 0.0035, 200.0);
  return 240 + n1 * 210 + n2 * 45;
}
}
```

```
// world.drawGrain();
```

```
drawGrain() {
 push();
 blendMode(BLEND);       // no OVERLAY (overlay exaggerates banding)
 tint(255, 18);          // lower opacity a lot (0-255). Try 10-25
 image(grainLayer, 0, 0);
 noTint();
 pop();
}
```

```
drawBreathOverlay() {
 noStroke();
```

```
 fill(90, 140, 170, 26); // constant alpha (no sin wave)

 rect(0, 0, width, height);

}
```

```
// Replace your current drawGrain() with this (Option B)
drawGrain() {
 push();
 blendMode(BLEND);   // no OVERLAY (overlay exaggerates banding)
 tint(255, 18);      // subtle opacity; try 10-25 if needed
 image(grainLayer, 0, 0);
 noTint();
 pop();
}
```

```
let SHOW_FAR = true;
let SHOW_MID = true;
let SHOW_NEAR = true;
let SHOW_FLOATERS = true;
let SHOW_BREATH = true;
let SHOW_VIGNETTE = true;
let SHOW_GRAIN = true;
```

```
function keyPressed() {
 if (key === "h" || key === "H") showHUD = !showHUD;
 if (key === "r" || key === "R") {
   world = new World();
   cam = new CameraRig();
   grainLayer = createGraphics(width, height);
   makeStaticGrain(grainLayer, 18);
 }

 if (key === "1") SHOW_FAR = !SHOW_FAR;
 if (key === "2") SHOW_MID = !SHOW_MID;
 if (key === "3") SHOW_NEAR = !SHOW_NEAR;
 if (key === "4") SHOW_FLOATERS = !SHOW_FLOATERS;
 if (key === "5") SHOW_BREATH = !SHOW_BREATH;
 if (key === "6") SHOW_VIGNETTE = !SHOW_VIGNETTE;
 if (key === "7") SHOW_GRAIN = !SHOW_GRAIN;
}
```

```
push();
cam.apply();

if (SHOW_FAR) world.drawFar();
if (SHOW_MID) world.drawMid();
if (SHOW_NEAR) world.drawNear();
if (SHOW_FLOATERS) world.drawFloaters();

pop();

if (SHOW_BREATH) world.drawBreathOverlay();
if (SHOW_VIGNETTE) world.drawVignette();
if (SHOW_GRAIN) world.drawGrain();
```

```
// Aurora ribbon (very subtle)
for (let x = this.softBounds.minX; x < this.softBounds.maxX; x += 22) {
 ...
 fill(110, 160, 190, a);
 ellipse(x, y, 120, 70);
}
```

```
const a = 8 + n * 12;      // was 22 + n * 28
ellipse(x, y, 90, 50);     // smaller
```

```
drawBreathOverlay() {
 noStroke();
 fill(90, 140, 170, 18); // constant alpha
 rect(0, 0, width, height);
}
```

```
stroke(180, 210, 230, a);
line(...)
```

```
const a = 6 + noise(x * 0.003, 400) * 10; // lower than before
```

```
/*
Meditative Camera Scroll — p5.js (single-file)
Drop into https://editor.p5js.org

Debug additions:
```

```
- Layer toggles (1-7) to isolate artifacts like the white “stream”
- Option B grain draw (BLEND + low tint) to reduce scanline/banding look

Controls:
- Move focus: Arrow keys / WASD (gentle nudge)
- Pause drift: hold SPACE
- Reset: R
- Toggle UI: H
- Debug toggles:
 1 Far  2 Mid  3 Near  4 Floaters  5 Breath  6 Vignette  7 Grain
*/

let cam, world;
let showHUD = true;

// Static, precomputed grain
let grainLayer;

// Debug layer toggles
let SHOW_FAR = true;
let SHOW_MID = true;
let SHOW_NEAR = true;
let SHOW_FLOATERS = true;
let SHOW_BREATH = true;
let SHOW_VIGNETTE = true;
let SHOW_GRAIN = true;

function setup() {
 createCanvas(900, 520);
 pixelDensity(1);

 world = new World();
 cam = new CameraRig();

 textFont("Georgia");

 // Build static grain ONCE (stable)
 grainLayer = createGraphics(width, height);
```

```
 makeStaticGrain(grainLayer, 18); // intensity: try 10-30
}

function draw() {
 background(6, 8, 12);

 world.update();
 cam.update(world);

 // World-space
 push();
 cam.apply();

 if (SHOW_FAR) world.drawFar();
 if (SHOW_MID) world.drawMid();
 if (SHOW_NEAR) world.drawNear();
 if (SHOW_FLOATERS) world.drawFloaters();

 pop();

 // Screen-space overlays
 if (SHOW_BREATH) world.drawBreathOverlay();
 if (SHOW_VIGNETTE) world.drawVignette();
 if (SHOW_GRAIN) world.drawGrain();

 if (showHUD) drawHUD();
}

// -------------------- Input --------------------
function keyPressed() {
 if (key === "h" || key === "H") showHUD = !showHUD;

 if (key === "r" || key === "R") {
   world = new World();
   cam = new CameraRig();
   grainLayer = createGraphics(width, height);
   makeStaticGrain(grainLayer, 18);
 }
```

```
 // Debug toggles
 if (key === "1") SHOW_FAR = !SHOW_FAR;
 if (key === "2") SHOW_MID = !SHOW_MID;
 if (key === "3") SHOW_NEAR = !SHOW_NEAR;
 if (key === "4") SHOW_FLOATERS = !SHOW_FLOATERS;
 if (key === "5") SHOW_BREATH = !SHOW_BREATH;
 if (key === "6") SHOW_VIGNETTE = !SHOW_VIGNETTE;
 if (key === "7") SHOW_GRAIN = !SHOW_GRAIN;
}

// -------------------- HUD --------------------
function drawHUD() {
 const pad = 16;
 noStroke();
 fill(0, 90);
 rect(pad, pad, 520, 96, 14);

 fill(230, 220);
 textSize(14);
 textAlign(LEFT, TOP);
 text(
   "Meditative camera drift\nWASD / Arrows: nudge focus  •  Hold SPACE:
pause drift\nH: hide text  •  R: reset\n1 Far  2 Mid  3 Near  4 Floaters
5 Breath  6 Vignette  7 Grain",
   pad + 12,
   pad + 10
 );
}

// -------------------- Static Grain Builder --------------------
function makeStaticGrain(pg, intensity = 18) {
 pg.pixelDensity(1);
 pg.loadPixels();

 for (let y = 0; y < pg.height; y++) {
   for (let x = 0; x < pg.width; x++) {
     const idx = 4 * (y * pg.width + x);
```

```
      const n = noise(x * 0.08, y * 0.08);
      const g = (n - 0.5) * 2 * intensity;

      pg.pixels[idx + 0] = 128 + g;
      pg.pixels[idx + 1] = 128 + g;
      pg.pixels[idx + 2] = 128 + g;
      pg.pixels[idx + 3] = 35; // subtle alpha baked in
    }
  }

  pg.updatePixels();
}

// ==================== Camera ====================
class CameraRig {
  constructor() {
    this.pos = createVector(0, 0);
    this.vel = createVector(0, 0);

    this.focus = createVector(0, 0);
    this.drift = createVector(0, 0);

    this.t = random(1000);
  }

  update(worldRef) {
    const nudge = createVector(0, 0);
    const push = 0.9;

    if (keyIsDown(LEFT_ARROW) || keyIsDown(65)) nudge.x -= push;  // A
    if (keyIsDown(RIGHT_ARROW) || keyIsDown(68)) nudge.x += push; // D
    if (keyIsDown(UP_ARROW) || keyIsDown(87)) nudge.y -= push;    // W
    if (keyIsDown(DOWN_ARROW) || keyIsDown(83)) nudge.y += push;  // S

    const paused = keyIsDown(32); // SPACE

    this.t += paused ? 0 : 0.0035;
```

```
    const wanderX = (noise(this.t) - 0.5) * 2;
    const wanderY = (noise(this.t + 50.0) - 0.5) * 2;

    if (!paused) {
      this.drift.x += 0.55 + wanderX * 0.22;
      this.drift.y = sin(this.t * 1.2) * 55 + wanderY * 35;
    }

    const softClamp = worldRef.softBounds;
    this.drift.x = constrain(this.drift.x, softClamp.minX, softClamp.maxX);
    this.drift.y = constrain(this.drift.y, softClamp.minY, softClamp.maxY);

    this.focus.add(nudge.mult(2.2));
    this.focus.lerp(this.drift, 0.035);

    const desired = p5.Vector.sub(
      this.focus,
      createVector(width * 0.5, height * 0.52)
    );
    const accel = p5.Vector.sub(desired, this.pos).mult(0.028);
    this.vel.add(accel);
    this.vel.mult(0.86);
    this.pos.add(this.vel);

    const floatX = (noise(this.t + 200) - 0.5) * 1.8;
    const floatY = (noise(this.t + 300) - 0.5) * 1.8;
    this.pos.x += floatX;
    this.pos.y += floatY;
  }

  apply() {
    translate(-this.pos.x, -this.pos.y);
  }
}

// ==================== World ====================
class World {
  constructor() {
```

```
  this.seed = floor(random(1e9));
  noiseSeed(this.seed);
  randomSeed(this.seed);

  this.softBounds = {
    minX: -900,
    maxX: 6800,
    minY: -650,
    maxY: 650
  };

  this.breathT = random(1000);

  this.floaters = [];
  for (let i = 0; i < 220; i++) this.floaters.push(this.makeFloater());

  this.stars = [];
  for (let i = 0; i < 420; i++) {
    this.stars.push({
      x: random(this.softBounds.minX, this.softBounds.maxX),
      y: random(this.softBounds.minY, this.softBounds.maxY),
      s: random(0.5, 2.2),
      a: random(70, 170)
    });
  }
}

makeFloater() {
  return {
    x: random(this.softBounds.minX, this.softBounds.maxX),
    y: random(-260, 260),
    r: random(1.2, 3.8),
    phase: random(TWO_PI),
    drift: random(0.2, 0.55),
    glow: random(40, 120)
  };
}
```

```
update() {
  this.breathT += 0.01;

  for (let p of this.floaters) {
    const w = noise(p.x * 0.0012, p.y * 0.0012, frameCount * 0.002) -
0.5;
    p.x += w * 0.5;
    p.y += sin(frameCount * 0.004 + p.phase) * 0.22 - p.drift;

    if (p.y < -520) {
      p.y = random(120, 420);
      p.x = random(this.softBounds.minX, this.softBounds.maxX);
    }
  }
}

// ---------- Layers ----------
drawFar() {
  push();
  noStroke();

  for (let st of this.stars) {
    fill(210, 220, 245, st.a);
    circle(st.x, st.y, st.s);
  }

  // Aurora ribbon (possible culprit for "white stream" — toggle with 1)
  for (let x = this.softBounds.minX; x < this.softBounds.maxX; x += 22) {
    const n = noise(x * 0.0007, this.seed * 0.00001);
    const y = -250 + n * 120 + sin(x * 0.002 + this.breathT * 0.35) * 18;
    const a = 22 + n * 28;
    fill(110, 160, 190, a);
    ellipse(x, y, 120, 70);
  }

  pop();
}
```

```
 drawMid() {
   push();

   const horizonY = 80;
   const groundBase = 260;

   noStroke();
   for (let y = -400; y < groundBase; y += 2) {
     const t = map(y, -400, groundBase, 0, 1);
     fill(10 + t * 18, 14 + t * 18, 22 + t * 24, 255);
     rect(this.softBounds.minX, y, this.softBounds.maxX -
this.softBounds.minX, 2);
   }

   fill(12, 16, 22, 220);
   beginShape();
   vertex(this.softBounds.minX, groundBase);
   for (let x = this.softBounds.minX; x <= this.softBounds.maxX; x += 18)
{
     const n = noise(x * 0.0009, 10.0);
     const yy = horizonY + n * 140;
     vertex(x, yy);
   }
   vertex(this.softBounds.maxX, groundBase);
   endShape(CLOSE);

   fill(8, 10, 14, 255);
   beginShape();
   vertex(this.softBounds.minX, height + 900);
   for (let x = this.softBounds.minX; x <= this.softBounds.maxX; x += 16)
{
     const n1 = noise(x * 0.0013, 100.0);
     const n2 = noise(x * 0.0035, 200.0);
     const yy = 240 + n1 * 210 + n2 * 45;
     vertex(x, yy);
   }
   vertex(this.softBounds.maxX, height + 900);
   endShape(CLOSE);
```

```
  const waterY = 300;
  for (let x = this.softBounds.minX; x < this.softBounds.maxX; x += 10) {
    const ripple =
      sin(x * 0.02 + this.breathT * 1.2) * 2 +
      (noise(x * 0.01, this.breathT * 0.2) - 0.5) * 3;
    const a = 14 + noise(x * 0.003, 400) * 22;
    stroke(180, 210, 230, a);
    line(x, waterY + ripple, x + 8, waterY + ripple + 0.4);
  }

  noStroke();
  for (let i = 0; i < 26; i++) {
    const px = i * 280 + 90 + sin(i * 2.1) * 35;
    const py = this.sampleGroundY(px) - 8;
    const tall = 18 + noise(i * 10.1) * 55;

    fill(18, 22, 30, 220);
    rect(px - 2, py - tall, 4, tall, 2);

    fill(200, 220, 240, 40);
    circle(px, py - tall - 6, 18);
  }

  pop();
}

drawNear() {
  push();

  for (let i = 0; i < 42; i++) {
    const x = i * 160 + 40 + sin(i * 0.9) * 20;
    const gy = this.sampleGroundY(x);
    const sway = sin(frameCount * 0.01 + i) * 2.5;

    stroke(12, 14, 18, 210);
    strokeWeight(3);
    line(x, gy - 8, x + sway, gy - 70);
```

```
    noStroke();
    fill(10, 12, 16, 240);
    ellipse(x + sway, gy - 82, 36, 24);
    ellipse(x + sway + 16, gy - 76, 22, 16);
    ellipse(x + sway - 14, gy - 74, 20, 14);
  }

  pop();
}

drawFloaters() {
  push();
  noStroke();
  for (let p of this.floaters) {
    const tw = sin(frameCount * 0.02 + p.phase) * 0.4 + 0.6;
    fill(190, 210, 230, p.glow * tw);
    circle(p.x, p.y, p.r * (0.8 + tw));
  }
  pop();
}

// ---------- Screen-space overlays ----------
drawBreathOverlay() {
  const breath = sin(this.breathT * 0.35) * 0.5 + 0.5;
  const alpha = 22 + breath * 26;

  noStroke();
  fill(90, 140, 170, alpha);
  rect(0, 0, width, height);
}

drawVignette() {
  push();
  noFill();
  for (let i = 0; i < 18; i++) {
    const t = i / 17;
    const a = 18 * t;
```

```
    stroke(0, 0, 0, a);
    const m = t * 70;
    rect(m, m, width - 2 * m, height - 2 * m, 28);
  }
  pop();
}

// Option B grain draw (less banding)
drawGrain() {
  push();
  blendMode(BLEND);
  tint(255, 18); // try 10-25
  image(grainLayer, 0, 0);
  noTint();
  pop();
}

// ---------- Helpers ----------
sampleGroundY(x) {
  const n1 = noise(x * 0.0013, 100.0);
  const n2 = noise(x * 0.0035, 200.0);
  return 240 + n1 * 210 + n2 * 45;
}
}
```

```
drawFar() {
 push();
 noStroke();

 // Stars (unchanged)
 for (let st of this.stars) {
   fill(210, 220, 245, st.a);
   circle(st.x, st.y, st.s);
 }

 // --- Safer aurora: capped alpha + less overlap + segmented ribbons ---
 // fewer samples across X reduces stacking
 for (let x = this.softBounds.minX; x < this.softBounds.maxX; x += 40) {
   // "segment mask" breaks the ribbon so it can't become a continuous
stream
   const gate = noise(x * 0.0016, this.seed * 0.00002);
   if (gate < 0.42) continue; // skip some chunks

   const n = noise(x * 0.0007, this.seed * 0.00001);
```

```
  // gentler vertical motion (reduces banding)
  const y =
    -260 +
    n * 90 +
    sin(x * 0.0016 + this.breathT * 0.25) * 10;

  // hard cap the alpha so it never blows out
  let a = 6 + n * 14;     // was 22 + n*28
  a = constrain(a, 0, 18);

  fill(110, 160, 190, a);

  // smaller shapes reduces overlap brightness
  const w = 70 + n * 35;  // was 120
  const h = 36 + n * 18;  // was 70
  ellipse(x, y, w, h);
 }

 pop();
}
```

```
// ---- Replace CameraRig.update(worldRef) with this version ----
update(worldRef) {
 // Player nudge: faster + more responsive
 const nudge = createVector(0, 0);
 const push = 1.8; // was 0.9

 if (keyIsDown(LEFT_ARROW) || keyIsDown(65)) nudge.x -= push;  // A
 if (keyIsDown(RIGHT_ARROW) || keyIsDown(68)) nudge.x += push; // D
 if (keyIsDown(UP_ARROW) || keyIsDown(87)) nudge.y -= push;    // W
 if (keyIsDown(DOWN_ARROW) || keyIsDown(83)) nudge.y += push;  // S

 const paused = keyIsDown(32); // SPACE

 // Drift: slower overall
 this.t += paused ? 0 : 0.0020; // was 0.0035
 const wanderX = (noise(this.t) - 0.5) * 2;
 const wanderY = (noise(this.t + 50.0) - 0.5) * 2;

 if (!paused) {
   this.drift.x += 0.22 + wanderX * 0.10; // was 0.55 + *0.22
   this.drift.y = sin(this.t * 1.0) * 28 + wanderY * 16; // was *55 + *35
 }

 const softClamp = worldRef.softBounds;
 this.drift.x = constrain(this.drift.x, softClamp.minX, softClamp.maxX);
 this.drift.y = constrain(this.drift.y, softClamp.minY, softClamp.maxY);
```

```
 // Nudge affects focus more
 this.focus.add(nudge.mult(3.2));  // was 2.2
 this.focus.lerp(this.drift, 0.020); // was 0.035 (less pull toward drift)

 // Camera easing: slightly calmer follow
 const desired = p5.Vector.sub(
   this.focus,
   createVector(width * 0.5, height * 0.52)
 );
 const accel = p5.Vector.sub(desired, this.pos).mult(0.020); // was 0.028
 this.vel.add(accel);
 this.vel.mult(0.88); // was 0.86 (a touch smoother)
 this.pos.add(this.vel);

 // Tiny handheld float reduced
 const floatX = (noise(this.t + 200) - 0.5) * 1.0; // was 1.8
 const floatY = (noise(this.t + 300) - 0.5) * 1.0; // was 1.8
 this.pos.x += floatX;
 this.pos.y += floatY;
}
```

```
/*
Meditative Camera Scroll — p5.js (single-file)
Drop into https://editor.p5js.org

Updates:
- FAR aurora reworked to prevent the bright “white stream”
- Drift slowed down
- Player nudge made faster/more responsive
- Debug layer toggles kept (1–7)
- Grain uses Option B (BLEND + low tint)

Controls:
- Move focus: Arrow keys / WASD (nudge)
```

```
- Pause drift: hold SPACE
- Reset: R
- Toggle UI: H
- Debug toggles:
 1 Far  2 Mid  3 Near  4 Floaters  5 Breath  6 Vignette  7 Grain
*/

let cam, world;
let showHUD = true;

let grainLayer;

// Debug layer toggles
let SHOW_FAR = true;
let SHOW_MID = true;
let SHOW_NEAR = true;
let SHOW_FLOATERS = true;
let SHOW_BREATH = true;
let SHOW_VIGNETTE = true;
let SHOW_GRAIN = true;

function setup() {
 createCanvas(900, 520);
 pixelDensity(1);

 world = new World();
 cam = new CameraRig();

 textFont("Georgia");

 grainLayer = createGraphics(width, height);
 makeStaticGrain(grainLayer, 18);
}

function draw() {
 background(6, 8, 12);

 world.update();
```

```
 cam.update(world);

 push();
 cam.apply();

 if (SHOW_FAR) world.drawFar();
 if (SHOW_MID) world.drawMid();
 if (SHOW_NEAR) world.drawNear();
 if (SHOW_FLOATERS) world.drawFloaters();

 pop();

 if (SHOW_BREATH) world.drawBreathOverlay();
 if (SHOW_VIGNETTE) world.drawVignette();
 if (SHOW_GRAIN) world.drawGrain();

 if (showHUD) drawHUD();
}

// -------------------- Input --------------------
function keyPressed() {
 if (key === "h" || key === "H") showHUD = !showHUD;

 if (key === "r" || key === "R") {
   world = new World();
   cam = new CameraRig();
   grainLayer = createGraphics(width, height);
   makeStaticGrain(grainLayer, 18);
 }

 // Debug toggles
 if (key === "1") SHOW_FAR = !SHOW_FAR;
 if (key === "2") SHOW_MID = !SHOW_MID;
 if (key === "3") SHOW_NEAR = !SHOW_NEAR;
 if (key === "4") SHOW_FLOATERS = !SHOW_FLOATERS;
 if (key === "5") SHOW_BREATH = !SHOW_BREATH;
 if (key === "6") SHOW_VIGNETTE = !SHOW_VIGNETTE;
 if (key === "7") SHOW_GRAIN = !SHOW_GRAIN;
```

```
}

// -------------------- HUD --------------------
function drawHUD() {
 const pad = 16;
 noStroke();
 fill(0, 90);
 rect(pad, pad, 520, 96, 14);

 fill(230, 220);
 textSize(14);
 textAlign(LEFT, TOP);
 text(
   "Meditative camera drift\nWASD / Arrows: nudge focus  •  Hold SPACE:
pause drift\nH: hide text  •  R: reset\n1 Far  2 Mid  3 Near  4 Floaters
5 Breath  6 Vignette  7 Grain",
   pad + 12,
   pad + 10
 );
}

// -------------------- Static Grain Builder --------------------
function makeStaticGrain(pg, intensity = 18) {
 pg.pixelDensity(1);
 pg.loadPixels();

 for (let y = 0; y < pg.height; y++) {
   for (let x = 0; x < pg.width; x++) {
     const idx = 4 * (y * pg.width + x);

     const n = noise(x * 0.08, y * 0.08);
     const g = (n - 0.5) * 2 * intensity;

     pg.pixels[idx + 0] = 128 + g;
     pg.pixels[idx + 1] = 128 + g;
     pg.pixels[idx + 2] = 128 + g;
     pg.pixels[idx + 3] = 35;
   }
```

```
 }

 pg.updatePixels();
}

// ==================== Camera ====================
class CameraRig {
 constructor() {
   this.pos = createVector(0, 0);
   this.vel = createVector(0, 0);

   this.focus = createVector(0, 0);
   this.drift = createVector(0, 0);

   this.t = random(1000);
 }

 // Slower drift + faster nudge
 update(worldRef) {
   // Player nudge: faster + more responsive
   const nudge = createVector(0, 0);
   const push = 1.8;

   if (keyIsDown(LEFT_ARROW) || keyIsDown(65)) nudge.x -= push;  // A
   if (keyIsDown(RIGHT_ARROW) || keyIsDown(68)) nudge.x += push; // D
   if (keyIsDown(UP_ARROW) || keyIsDown(87)) nudge.y -= push;    // W
   if (keyIsDown(DOWN_ARROW) || keyIsDown(83)) nudge.y += push;  // S

   const paused = keyIsDown(32); // SPACE

   // Drift: slower overall
   this.t += paused ? 0 : 0.0020;
   const wanderX = (noise(this.t) - 0.5) * 2;
   const wanderY = (noise(this.t + 50.0) - 0.5) * 2;

   if (!paused) {
     this.drift.x += 0.22 + wanderX * 0.10;
     this.drift.y = sin(this.t * 1.0) * 28 + wanderY * 16;
```

```
    }

    const softClamp = worldRef.softBounds;
    this.drift.x = constrain(this.drift.x, softClamp.minX, softClamp.maxX);
    this.drift.y = constrain(this.drift.y, softClamp.minY, softClamp.maxY);

    // Nudge affects focus more
    this.focus.add(nudge.mult(3.2));
    this.focus.lerp(this.drift, 0.020);

    // Camera easing: calmer follow
    const desired = p5.Vector.sub(
      this.focus,
      createVector(width * 0.5, height * 0.52)
    );
    const accel = p5.Vector.sub(desired, this.pos).mult(0.020);
    this.vel.add(accel);
    this.vel.mult(0.88);
    this.pos.add(this.vel);

    // Tiny handheld float reduced
    const floatX = (noise(this.t + 200) - 0.5) * 1.0;
    const floatY = (noise(this.t + 300) - 0.5) * 1.0;
    this.pos.x += floatX;
    this.pos.y += floatY;
  }

  apply() {
    translate(-this.pos.x, -this.pos.y);
  }
}

// ==================== World ====================
class World {
  constructor() {
    this.seed = floor(random(1e9));
    noiseSeed(this.seed);
    randomSeed(this.seed);
```

```
  this.softBounds = {
    minX: -900,
    maxX: 6800,
    minY: -650,
    maxY: 650
  };

  this.breathT = random(1000);

  this.floaters = [];
  for (let i = 0; i < 220; i++) this.floaters.push(this.makeFloater());

  this.stars = [];
  for (let i = 0; i < 420; i++) {
    this.stars.push({
      x: random(this.softBounds.minX, this.softBounds.maxX),
      y: random(this.softBounds.minY, this.softBounds.maxY),
      s: random(0.5, 2.2),
      a: random(70, 170)
    });
  }
}

makeFloater() {
  return {
    x: random(this.softBounds.minX, this.softBounds.maxX),
    y: random(-260, 260),
    r: random(1.2, 3.8),
    phase: random(TWO_PI),
    drift: random(0.2, 0.55),
    glow: random(40, 120)
  };
}

update() {
  this.breathT += 0.01;
```

```
  for (let p of this.floaters) {
    const w = noise(p.x * 0.0012, p.y * 0.0012, frameCount * 0.002) -
0.5;
    p.x += w * 0.5;
    p.y += sin(frameCount * 0.004 + p.phase) * 0.22 - p.drift;

    if (p.y < -520) {
      p.y = random(120, 420);
      p.x = random(this.softBounds.minX, this.softBounds.maxX);
    }
  }
}

// ---------- Layers ----------
drawFar() {
  push();
  noStroke();

  // Stars
  for (let st of this.stars) {
    fill(210, 220, 245, st.a);
    circle(st.x, st.y, st.s);
  }

  // Reworked aurora: capped alpha + less overlap + segmented ribbons
  for (let x = this.softBounds.minX; x < this.softBounds.maxX; x += 40) {
    const gate = noise(x * 0.0016, this.seed * 0.00002);
    if (gate < 0.42) continue;

    const n = noise(x * 0.0007, this.seed * 0.00001);
    const y =
      -260 +
      n * 90 +
      sin(x * 0.0016 + this.breathT * 0.25) * 10;

    let a = 6 + n * 14;
    a = constrain(a, 0, 18);
```

```
      fill(110, 160, 190, a);

      const w = 70 + n * 35;
      const h = 36 + n * 18;
      ellipse(x, y, w, h);
    }

    pop();
  }

  drawMid() {
    push();

    const horizonY = 80;
    const groundBase = 260;

    noStroke();
    for (let y = -400; y < groundBase; y += 2) {
      const t = map(y, -400, groundBase, 0, 1);
      fill(10 + t * 18, 14 + t * 18, 22 + t * 24, 255);
      rect(this.softBounds.minX, y, this.softBounds.maxX -
this.softBounds.minX, 2);
    }

    fill(12, 16, 22, 220);
    beginShape();
    vertex(this.softBounds.minX, groundBase);
    for (let x = this.softBounds.minX; x <= this.softBounds.maxX; x += 18)
{
      const n = noise(x * 0.0009, 10.0);
      const yy = horizonY + n * 140;
      vertex(x, yy);
    }
    vertex(this.softBounds.maxX, groundBase);
    endShape(CLOSE);

    fill(8, 10, 14, 255);
    beginShape();
```

```
  vertex(this.softBounds.minX, height + 900);
  for (let x = this.softBounds.minX; x <= this.softBounds.maxX; x += 16)
{
    const n1 = noise(x * 0.0013, 100.0);
    const n2 = noise(x * 0.0035, 200.0);
    const yy = 240 + n1 * 210 + n2 * 45;
    vertex(x, yy);
  }
  vertex(this.softBounds.maxX, height + 900);
  endShape(CLOSE);

  const waterY = 300;
  for (let x = this.softBounds.minX; x < this.softBounds.maxX; x += 10) {
    const ripple =
      sin(x * 0.02 + this.breathT * 1.2) * 2 +
      (noise(x * 0.01, this.breathT * 0.2) - 0.5) * 3;
    const a = 14 + noise(x * 0.003, 400) * 22;
    stroke(180, 210, 230, a);
    line(x, waterY + ripple, x + 8, waterY + ripple + 0.4);
  }

  noStroke();
  for (let i = 0; i < 26; i++) {
    const px = i * 280 + 90 + sin(i * 2.1) * 35;
    const py = this.sampleGroundY(px) - 8;
    const tall = 18 + noise(i * 10.1) * 55;

    fill(18, 22, 30, 220);
    rect(px - 2, py - tall, 4, tall, 2);

    fill(200, 220, 240, 40);
    circle(px, py - tall - 6, 18);
  }

  pop();
}

drawNear() {
```

```
  push();

  for (let i = 0; i < 42; i++) {
    const x = i * 160 + 40 + sin(i * 0.9) * 20;
    const gy = this.sampleGroundY(x);
    const sway = sin(frameCount * 0.01 + i) * 2.5;

    stroke(12, 14, 18, 210);
    strokeWeight(3);
    line(x, gy - 8, x + sway, gy - 70);

    noStroke();
    fill(10, 12, 16, 240);
    ellipse(x + sway, gy - 82, 36, 24);
    ellipse(x + sway + 16, gy - 76, 22, 16);
    ellipse(x + sway - 14, gy - 74, 20, 14);
  }

  pop();
}

drawFloaters() {
  push();
  noStroke();
  for (let p of this.floaters) {
    const tw = sin(frameCount * 0.02 + p.phase) * 0.4 + 0.6;
    fill(190, 210, 230, p.glow * tw);
    circle(p.x, p.y, p.r * (0.8 + tw));
  }
  pop();
}

// ---------- Screen-space overlays ----------
drawBreathOverlay() {
  const breath = sin(this.breathT * 0.35) * 0.5 + 0.5;
  const alpha = 22 + breath * 26;

  noStroke();
```

```
  fill(90, 140, 170, alpha);
  rect(0, 0, width, height);
}

drawVignette() {
  push();
  noFill();
  for (let i = 0; i < 18; i++) {
    const t = i / 17;
    const a = 18 * t;
    stroke(0, 0, 0, a);
    const m = t * 70;
    rect(m, m, width - 2 * m, height - 2 * m, 28);
  }
  pop();
}

// Option B grain draw
drawGrain() {
  push();
  blendMode(BLEND);
  tint(255, 18);
  image(grainLayer, 0, 0);
  noTint();
  pop();
}

// ---------- Helpers ----------
sampleGroundY(x) {
  const n1 = noise(x * 0.0013, 100.0);
  const n2 = noise(x * 0.0035, 200.0);
  return 240 + n1 * 210 + n2 * 45;
}
}
```

```
/*
Meditative Camera Scroll — p5.js (single-file)

Update:
- Removed automatic drift entirely.
- Camera ONLY moves when the player nudges (WASD / Arrows).
- SPACE now does nothing (kept unused).
- Everything else preserved (far aurora fix, stable grain, debug toggles).

Controls:
- Move camera: Arrow keys / WASD
- Reset: R
- Toggle UI: H
- Debug toggles:
 1 Far  2 Mid  3 Near  4 Floaters  5 Breath  6 Vignette  7 Grain
*/

let cam, world;
let showHUD = true;

let grainLayer;

// Debug layer toggles
let SHOW_FAR = true;
let SHOW_MID = true;
let SHOW_NEAR = true;
let SHOW_FLOATERS = true;
let SHOW_BREATH = true;
let SHOW_VIGNETTE = true;
let SHOW_GRAIN = true;

function setup() {
 createCanvas(900, 520);
 pixelDensity(1);

 world = new World();
```

```
 cam = new CameraRig();

 textFont("Georgia");

 grainLayer = createGraphics(width, height);
 makeStaticGrain(grainLayer, 18);
}

function draw() {
 background(6, 8, 12);

 world.update();
 cam.update(world);

 push();
 cam.apply();

 if (SHOW_FAR) world.drawFar();
 if (SHOW_MID) world.drawMid();
 if (SHOW_NEAR) world.drawNear();
 if (SHOW_FLOATERS) world.drawFloaters();

 pop();

 if (SHOW_BREATH) world.drawBreathOverlay();
 if (SHOW_VIGNETTE) world.drawVignette();
 if (SHOW_GRAIN) world.drawGrain();

 if (showHUD) drawHUD();
}

// -------------------- Input --------------------
function keyPressed() {
 if (key === "h" || key === "H") showHUD = !showHUD;

 if (key === "r" || key === "R") {
   world = new World();
   cam = new CameraRig();
```

```
  grainLayer = createGraphics(width, height);
  makeStaticGrain(grainLayer, 18);
 }

 // Debug toggles
 if (key === "1") SHOW_FAR = !SHOW_FAR;
 if (key === "2") SHOW_MID = !SHOW_MID;
 if (key === "3") SHOW_NEAR = !SHOW_NEAR;
 if (key === "4") SHOW_FLOATERS = !SHOW_FLOATERS;
 if (key === "5") SHOW_BREATH = !SHOW_BREATH;
 if (key === "6") SHOW_VIGNETTE = !SHOW_VIGNETTE;
 if (key === "7") SHOW_GRAIN = !SHOW_GRAIN;
}

// -------------------- HUD --------------------
function drawHUD() {
 const pad = 16;
 noStroke();
 fill(0, 90);
 rect(pad, pad, 520, 90, 14);

 fill(230, 220);
 textSize(14);
 textAlign(LEFT, TOP);
 text(
  "Manual camera (no auto drift)\nWASD / Arrows: move camera\nH: hide
text  •  R: reset\n1 Far  2 Mid  3 Near  4 Floaters  5 Breath  6 Vignette
7 Grain",
  pad + 12,
  pad + 10
 );
}

// -------------------- Static Grain Builder --------------------
function makeStaticGrain(pg, intensity = 18) {
 pg.pixelDensity(1);
 pg.loadPixels();
```

```
 for (let y = 0; y < pg.height; y++) {
   for (let x = 0; x < pg.width; x++) {
     const idx = 4 * (y * pg.width + x);

     const n = noise(x * 0.08, y * 0.08);
     const g = (n - 0.5) * 2 * intensity;

     pg.pixels[idx + 0] = 128 + g;
     pg.pixels[idx + 1] = 128 + g;
     pg.pixels[idx + 2] = 128 + g;
     pg.pixels[idx + 3] = 35;
   }
 }

 pg.updatePixels();
}

// ==================== Camera ====================
class CameraRig {
 constructor() {
   this.pos = createVector(0, 0);
   this.vel = createVector(0, 0);

   // Start centered-ish in the world
   this.pos.x = 0;
   this.pos.y = 0;
 }

 // Manual-only camera: no drift, no auto focus
 update(worldRef) {
   // Movement input
   let ix = 0;
   let iy = 0;

   if (keyIsDown(LEFT_ARROW) || keyIsDown(65)) ix -= 1;  // A
   if (keyIsDown(RIGHT_ARROW) || keyIsDown(68)) ix += 1; // D
   if (keyIsDown(UP_ARROW) || keyIsDown(87)) iy -= 1;    // W
   if (keyIsDown(DOWN_ARROW) || keyIsDown(83)) iy += 1;  // S
```

```
  // Normalize diagonal
  const mag = Math.hypot(ix, iy);
  if (mag > 0) {
    ix /= mag;
    iy /= mag;
  }

  // Manual speed + smooth acceleration
  const speed = 2.6;      // camera move speed (raise for faster)
  const accel = 0.55;     // how quickly velocity changes
  const damping = 0.82;   // how quickly camera slows when not moving

  // Apply acceleration based on input
  this.vel.x += ix * accel;
  this.vel.y += iy * accel;

  // Clamp velocity to keep it calm
  const maxV = speed;
  this.vel.x = constrain(this.vel.x, -maxV, maxV);
  this.vel.y = constrain(this.vel.y, -maxV, maxV);

  // Damping (smooth stop)
  if (mag === 0) {
    this.vel.mult(damping);
  } else {
    // still damp slightly even while moving for a softer feel
    this.vel.mult(0.92);
  }

  // Update position
  this.pos.add(this.vel);

  // Keep camera within world bounds (so you can't lose the scene)
  const b = worldRef.softBounds;
  // Since we translate by -pos, pos represents top-left camera in
world-space.
  // We'll constrain so you can still see content vertically.
```

```
    this.pos.x = constrain(this.pos.x, b.minX, b.maxX);
    this.pos.y = constrain(this.pos.y, b.minY, b.maxY);
  }

  apply() {
    translate(-this.pos.x, -this.pos.y);
  }
}

// ==================== World ====================
class World {
  constructor() {
    this.seed = floor(random(1e9));
    noiseSeed(this.seed);
    randomSeed(this.seed);

    this.softBounds = {
      minX: -900,
      maxX: 6800,
      minY: -650,
      maxY: 650
    };

    this.breathT = random(1000);

    this.floaters = [];
    for (let i = 0; i < 220; i++) this.floaters.push(this.makeFloater());

    this.stars = [];
    for (let i = 0; i < 420; i++) {
      this.stars.push({
        x: random(this.softBounds.minX, this.softBounds.maxX),
        y: random(this.softBounds.minY, this.softBounds.maxY),
        s: random(0.5, 2.2),
        a: random(70, 170)
      });
    }
  }
```

```
makeFloater() {
  return {
    x: random(this.softBounds.minX, this.softBounds.maxX),
    y: random(-260, 260),
    r: random(1.2, 3.8),
    phase: random(TWO_PI),
    drift: random(0.2, 0.55),
    glow: random(40, 120)
  };
}

update() {
  this.breathT += 0.01;

  for (let p of this.floaters) {
    const w = noise(p.x * 0.0012, p.y * 0.0012, frameCount * 0.002) -
0.5;
    p.x += w * 0.5;
    p.y += sin(frameCount * 0.004 + p.phase) * 0.22 - p.drift;

    if (p.y < -520) {
      p.y = random(120, 420);
      p.x = random(this.softBounds.minX, this.softBounds.maxX);
    }
  }
}

// ---------- Layers ----------
drawFar() {
  push();
  noStroke();

  for (let st of this.stars) {
    fill(210, 220, 245, st.a);
    circle(st.x, st.y, st.s);
  }
```

```
  // Safer aurora: capped alpha + less overlap + segmented ribbons
  for (let x = this.softBounds.minX; x < this.softBounds.maxX; x += 40) {
    const gate = noise(x * 0.0016, this.seed * 0.00002);
    if (gate < 0.42) continue;

    const n = noise(x * 0.0007, this.seed * 0.00001);
    const y =
      -260 +
      n * 90 +
      sin(x * 0.0016 + this.breathT * 0.25) * 10;

    let a = 6 + n * 14;
    a = constrain(a, 0, 18);

    fill(110, 160, 190, a);

    const w = 70 + n * 35;
    const h = 36 + n * 18;
    ellipse(x, y, w, h);
  }

  pop();
}

drawMid() {
  push();

  const horizonY = 80;
  const groundBase = 260;

  noStroke();
  for (let y = -400; y < groundBase; y += 2) {
    const t = map(y, -400, groundBase, 0, 1);
    fill(10 + t * 18, 14 + t * 18, 22 + t * 24, 255);
    rect(this.softBounds.minX, y, this.softBounds.maxX -
this.softBounds.minX, 2);
  }
```

```
  fill(12, 16, 22, 220);
  beginShape();
  vertex(this.softBounds.minX, groundBase);
  for (let x = this.softBounds.minX; x <= this.softBounds.maxX; x += 18)
{
    const n = noise(x * 0.0009, 10.0);
    const yy = horizonY + n * 140;
    vertex(x, yy);
  }
  vertex(this.softBounds.maxX, groundBase);
  endShape(CLOSE);

  fill(8, 10, 14, 255);
  beginShape();
  vertex(this.softBounds.minX, height + 900);
  for (let x = this.softBounds.minX; x <= this.softBounds.maxX; x += 16)
{
    const n1 = noise(x * 0.0013, 100.0);
    const n2 = noise(x * 0.0035, 200.0);
    const yy = 240 + n1 * 210 + n2 * 45;
    vertex(x, yy);
  }
  vertex(this.softBounds.maxX, height + 900);
  endShape(CLOSE);

  const waterY = 300;
  for (let x = this.softBounds.minX; x < this.softBounds.maxX; x += 10) {
    const ripple =
      sin(x * 0.02 + this.breathT * 1.2) * 2 +
      (noise(x * 0.01, this.breathT * 0.2) - 0.5) * 3;
    const a = 14 + noise(x * 0.003, 400) * 22;
    stroke(180, 210, 230, a);
    line(x, waterY + ripple, x + 8, waterY + ripple + 0.4);
  }

  noStroke();
  for (let i = 0; i < 26; i++) {
    const px = i * 280 + 90 + sin(i * 2.1) * 35;
```

```
    const py = this.sampleGroundY(px) - 8;
    const tall = 18 + noise(i * 10.1) * 55;

    fill(18, 22, 30, 220);
    rect(px - 2, py - tall, 4, tall, 2);

    fill(200, 220, 240, 40);
    circle(px, py - tall - 6, 18);
  }

  pop();
}

drawNear() {
  push();

  for (let i = 0; i < 42; i++) {
    const x = i * 160 + 40 + sin(i * 0.9) * 20;
    const gy = this.sampleGroundY(x);
    const sway = sin(frameCount * 0.01 + i) * 2.5;

    stroke(12, 14, 18, 210);
    strokeWeight(3);
    line(x, gy - 8, x + sway, gy - 70);

    noStroke();
    fill(10, 12, 16, 240);
    ellipse(x + sway, gy - 82, 36, 24);
    ellipse(x + sway + 16, gy - 76, 22, 16);
    ellipse(x + sway - 14, gy - 74, 20, 14);
  }

  pop();
}

drawFloaters() {
  push();
  noStroke();
```

```
  for (let p of this.floaters) {
    const tw = sin(frameCount * 0.02 + p.phase) * 0.4 + 0.6;
    fill(190, 210, 230, p.glow * tw);
    circle(p.x, p.y, p.r * (0.8 + tw));
  }
  pop();
}

// ---------- Screen-space overlays ----------
drawBreathOverlay() {
  const breath = sin(this.breathT * 0.35) * 0.5 + 0.5;
  const alpha = 22 + breath * 26;

  noStroke();
  fill(90, 140, 170, alpha);
  rect(0, 0, width, height);
}

drawVignette() {
  push();
  noFill();
  for (let i = 0; i < 18; i++) {
    const t = i / 17;
    const a = 18 * t;
    stroke(0, 0, 0, a);
    const m = t * 70;
    rect(m, m, width - 2 * m, height - 2 * m, 28);
  }
  pop();
}

drawGrain() {
  push();
  blendMode(BLEND);
  tint(255, 18);
  image(grainLayer, 0, 0);
  noTint();
  pop();
```

```
 }

 // ---------- Helpers ----------
 sampleGroundY(x) {
   const n1 = noise(x * 0.0013, 100.0);
   const n2 = noise(x * 0.0035, 200.0);
   return 240 + n1 * 210 + n2 * 45;
 }
}
```

```
/*
Meditative Camera Scroll — p5.js (single-file)

Now adding detail:
- Night farm “middle of nowhere” vibe
- Cows (simple silhouettes) placed on the ground
- Bunches of grass (tufts) scattered along the ground
- Everything remains manual camera only (WASD / Arrows)
- FAR aurora stays “safe” (no bright stream)
- Stable grain + debug toggles remain

Controls:
- Move camera: Arrow keys / WASD
- Reset: R
- Toggle UI: H
- Debug toggles:
 1 Far  2 Mid  3 Near  4 Floaters  5 Breath  6 Vignette  7 Grain
*/

let cam, world;
let showHUD = true;
```

```
let grainLayer;

// Debug layer toggles
let SHOW_FAR = true;
let SHOW_MID = true;
let SHOW_NEAR = true;
let SHOW_FLOATERS = true;
let SHOW_BREATH = true;
let SHOW_VIGNETTE = true;
let SHOW_GRAIN = true;

function setup() {
 createCanvas(900, 520);
 pixelDensity(1);

 world = new World();
 cam = new CameraRig();

 textFont("Georgia");

 grainLayer = createGraphics(width, height);
 makeStaticGrain(grainLayer, 18);
}

function draw() {
 background(6, 8, 12);

 world.update();
 cam.update(world);

 push();
 cam.apply();

 if (SHOW_FAR) world.drawFar();
 if (SHOW_MID) world.drawMid();
 if (SHOW_NEAR) world.drawNear();
 if (SHOW_FLOATERS) world.drawFloaters();
```

```
 pop();

 if (SHOW_BREATH) world.drawBreathOverlay();
 if (SHOW_VIGNETTE) world.drawVignette();
 if (SHOW_GRAIN) world.drawGrain();

 if (showHUD) drawHUD();
}

// -------------------- Input --------------------
function keyPressed() {
 if (key === "h" || key === "H") showHUD = !showHUD;

 if (key === "r" || key === "R") {
   world = new World();
   cam = new CameraRig();
   grainLayer = createGraphics(width, height);
   makeStaticGrain(grainLayer, 18);
 }

 // Debug toggles
 if (key === "1") SHOW_FAR = !SHOW_FAR;
 if (key === "2") SHOW_MID = !SHOW_MID;
 if (key === "3") SHOW_NEAR = !SHOW_NEAR;
 if (key === "4") SHOW_FLOATERS = !SHOW_FLOATERS;
 if (key === "5") SHOW_BREATH = !SHOW_BREATH;
 if (key === "6") SHOW_VIGNETTE = !SHOW_VIGNETTE;
 if (key === "7") SHOW_GRAIN = !SHOW_GRAIN;
}

// -------------------- HUD --------------------
function drawHUD() {
 const pad = 16;
 noStroke();
 fill(0, 90);
 rect(pad, pad, 540, 94, 14);

 fill(230, 220);
```

```
 textSize(14);
 textAlign(LEFT, TOP);
 text(
   "Night farm drift (manual camera)\nWASD / Arrows: move camera\nH: hide
text  •  R: reset\n1 Far  2 Mid  3 Near  4 Floaters  5 Breath  6 Vignette
7 Grain",
   pad + 12,
   pad + 10
 );
}

// -------------------- Static Grain Builder --------------------
function makeStaticGrain(pg, intensity = 18) {
 pg.pixelDensity(1);
 pg.loadPixels();

 for (let y = 0; y < pg.height; y++) {
   for (let x = 0; x < pg.width; x++) {
     const idx = 4 * (y * pg.width + x);

     const n = noise(x * 0.08, y * 0.08);
     const g = (n - 0.5) * 2 * intensity;

     pg.pixels[idx + 0] = 128 + g;
     pg.pixels[idx + 1] = 128 + g;
     pg.pixels[idx + 2] = 128 + g;
     pg.pixels[idx + 3] = 35;
   }
 }

 pg.updatePixels();
}

// ==================== Camera ====================
class CameraRig {
 constructor() {
   this.pos = createVector(0, 0);
   this.vel = createVector(0, 0);
```

```
}

// Manual-only camera: no drift, no auto focus
update(worldRef) {
  let ix = 0;
  let iy = 0;

  if (keyIsDown(LEFT_ARROW) || keyIsDown(65)) ix -= 1;  // A
  if (keyIsDown(RIGHT_ARROW) || keyIsDown(68)) ix += 1; // D
  if (keyIsDown(UP_ARROW) || keyIsDown(87)) iy -= 1;    // W
  if (keyIsDown(DOWN_ARROW) || keyIsDown(83)) iy += 1;  // S

  const mag = Math.hypot(ix, iy);
  if (mag > 0) {
    ix /= mag;
    iy /= mag;
  }

  const accel = 0.58;
  const dampingWhenIdle = 0.82;
  const dampingWhenMoving = 0.92;
  const maxV = 3.0; // speed

  this.vel.x += ix * accel;
  this.vel.y += iy * accel;

  this.vel.x = constrain(this.vel.x, -maxV, maxV);
  this.vel.y = constrain(this.vel.y, -maxV, maxV);

  if (mag === 0) this.vel.mult(dampingWhenIdle);
  else this.vel.mult(dampingWhenMoving);

  this.pos.add(this.vel);

  const b = worldRef.softBounds;
  this.pos.x = constrain(this.pos.x, b.minX, b.maxX);
  this.pos.y = constrain(this.pos.y, b.minY, b.maxY);
}
```

```
 apply() {
   translate(-this.pos.x, -this.pos.y);
 }
}

// ==================== World ====================
class World {
 constructor() {
   this.seed = floor(random(1e9));
   noiseSeed(this.seed);
   randomSeed(this.seed);

   this.softBounds = {
     minX: -900,
     maxX: 6800,
     minY: -650,
     maxY: 650
   };

   this.breathT = random(1000);

   // particles
   this.floaters = [];
   for (let i = 0; i < 220; i++) this.floaters.push(this.makeFloater());

   // stars
   this.stars = [];
   for (let i = 0; i < 420; i++) {
     this.stars.push({
       x: random(this.softBounds.minX, this.softBounds.maxX),
       y: random(this.softBounds.minY, this.softBounds.maxY),
       s: random(0.5, 2.2),
       a: random(70, 170)
     });
   }

   // ---- New detail: cows + grass tufts ----
```

```
    this.cows = this.makeCows(14);
    this.grass = this.makeGrass(160);
  }

  makeFloater() {
    return {
      x: random(this.softBounds.minX, this.softBounds.maxX),
      y: random(-260, 260),
      r: random(1.2, 3.8),
      phase: random(TWO_PI),
      drift: random(0.2, 0.55),
      glow: random(40, 120)
    };
  }

  makeCows(count) {
    const arr = [];
    for (let i = 0; i < count; i++) {
      const x = random(this.softBounds.minX + 200, this.softBounds.maxX -
200);
      const y = this.sampleGroundY(x);
      const s = random(0.85, 1.5);
      arr.push({
        x,
        y,
        s,
        flip: random() < 0.5 ? -1 : 1,
        idle: random(TWO_PI),
        // little “clusters” so cows feel grouped sometimes
        buddyOffset: random() < 0.35 ? random(-90, 90) : 0,
      });
    }
    // optional: nudge some cows into gentle pairs
    for (let c of arr) {
      if (c.buddyOffset !== 0) {
        c.x += c.buddyOffset;
        c.y = this.sampleGroundY(c.x);
      }
```

```
  }
  return arr;
}

makeGrass(count) {
  const arr = [];
  for (let i = 0; i < count; i++) {
    const x = random(this.softBounds.minX, this.softBounds.maxX);
    const y = this.sampleGroundY(x);
    const s = random(0.6, 1.35);
    const clump = floor(random(2, 6)); // blades per tuft
    arr.push({
      x,
      y,
      s,
      clump,
      phase: random(TWO_PI),
      tint: random(0.75, 1.05)
    });
  }
  return arr;
}

update() {
  this.breathT += 0.01;

  // floaters drift
  for (let p of this.floaters) {
    const w = noise(p.x * 0.0012, p.y * 0.0012, frameCount * 0.002) -
0.5;
    p.x += w * 0.5;
    p.y += sin(frameCount * 0.004 + p.phase) * 0.22 - p.drift;

    if (p.y < -520) {
      p.y = random(120, 420);
      p.x = random(this.softBounds.minX, this.softBounds.maxX);
    }
  }
```

```
}

// ---------- Layers ----------
drawFar() {
  push();
  noStroke();

  for (let st of this.stars) {
    fill(210, 220, 245, st.a);
    circle(st.x, st.y, st.s);
  }

  // safer aurora (segmented, capped alpha)
  for (let x = this.softBounds.minX; x < this.softBounds.maxX; x += 40) {
    const gate = noise(x * 0.0016, this.seed * 0.00002);
    if (gate < 0.42) continue;

    const n = noise(x * 0.0007, this.seed * 0.00001);
    const y = -260 + n * 90 + sin(x * 0.0016 + this.breathT * 0.25) * 10;

    let a = 6 + n * 14;
    a = constrain(a, 0, 18);

    fill(110, 160, 190, a);

    const w = 70 + n * 35;
    const h = 36 + n * 18;
    ellipse(x, y, w, h);
  }

  pop();
}

drawMid() {
  push();

  const horizonY = 80;
  const groundBase = 260;
```

```
  // sky gradient slab
  noStroke();
  for (let y = -400; y < groundBase; y += 2) {
    const t = map(y, -400, groundBase, 0, 1);
    fill(10 + t * 18, 14 + t * 18, 22 + t * 24, 255);
    rect(this.softBounds.minX, y, this.softBounds.maxX -
this.softBounds.minX, 2);
  }

  // distant hills
  fill(12, 16, 22, 220);
  beginShape();
  vertex(this.softBounds.minX, groundBase);
  for (let x = this.softBounds.minX; x <= this.softBounds.maxX; x += 18)
{
    const n = noise(x * 0.0009, 10.0);
    const yy = horizonY + n * 140;
    vertex(x, yy);
  }
  vertex(this.softBounds.maxX, groundBase);
  endShape(CLOSE);

  // main ground
  fill(8, 10, 14, 255);
  beginShape();
  vertex(this.softBounds.minX, height + 900);
  for (let x = this.softBounds.minX; x <= this.softBounds.maxX; x += 16)
{
    const n1 = noise(x * 0.0013, 100.0);
    const n2 = noise(x * 0.0035, 200.0);
    const yy = 240 + n1 * 210 + n2 * 45;
    vertex(x, yy);
  }
  vertex(this.softBounds.maxX, height + 900);
  endShape(CLOSE);

  // water line
```

```
  const waterY = 300;
  for (let x = this.softBounds.minX; x < this.softBounds.maxX; x += 10) {
    const ripple =
      sin(x * 0.02 + this.breathT * 1.2) * 2 +
      (noise(x * 0.01, this.breathT * 0.2) - 0.5) * 3;
    const a = 12 + noise(x * 0.003, 400) * 18;
    stroke(180, 210, 230, a);
    line(x, waterY + ripple, x + 8, waterY + ripple + 0.4);
  }

  // tiny “memory posts”
  noStroke();
  for (let i = 0; i < 26; i++) {
    const px = i * 280 + 90 + sin(i * 2.1) * 35;
    const py = this.sampleGroundY(px) - 8;
    const tall = 18 + noise(i * 10.1) * 55;

    fill(18, 22, 30, 220);
    rect(px - 2, py - tall, 4, tall, 2);

    fill(200, 220, 240, 40);
    circle(px, py - tall - 6, 18);
  }

  // ---- New: cows (mid-ground silhouettes with tiny idle motion) ----
  this.drawCows();

  pop();
}

drawNear() {
  push();

  // ---- New: grass tufts in the near layer (foreground detail) ----
  this.drawGrass();

  // sparse trees/reeds
  for (let i = 0; i < 42; i++) {
```

```
      const x = i * 160 + 40 + sin(i * 0.9) * 20;
      const gy = this.sampleGroundY(x);
      const sway = sin(frameCount * 0.01 + i) * 2.5;

      stroke(12, 14, 18, 210);
      strokeWeight(3);
      line(x, gy - 8, x + sway, gy - 70);

      noStroke();
      fill(10, 12, 16, 240);
      ellipse(x + sway, gy - 82, 36, 24);
      ellipse(x + sway + 16, gy - 76, 22, 16);
      ellipse(x + sway - 14, gy - 74, 20, 14);
    }

    pop();
  }

  drawFloaters() {
    push();
    noStroke();
    for (let p of this.floaters) {
      const tw = sin(frameCount * 0.02 + p.phase) * 0.4 + 0.6;
      fill(190, 210, 230, p.glow * tw);
      circle(p.x, p.y, p.r * (0.8 + tw));
    }
    pop();
  }

  // ---------- New: cows ----------
  drawCows() {
    push();

    // cow color is slightly lighter than ground so you can actually see
them
    // (still "night silhouettes", but readable)
    for (let i = 0; i < this.cows.length; i++) {
      const c = this.cows[i];
```

```
    const bob = sin(frameCount * 0.015 + c.idle) * 1.2;

    push();
    translate(c.x, c.y + bob);
    scale(c.flip * c.s, c.s);

    // body
    noStroke();
    fill(20, 24, 32, 235);
    ellipse(0, -18, 56, 30);

    // head
    ellipse(34, -26, 20, 16);

    // snout
    fill(24, 28, 38, 235);
    ellipse(42, -24, 10, 8);

    // legs
    fill(16, 18, 24, 240);
    rect(-18, -6, 6, 18, 2);
    rect(-6, -6, 6, 18, 2);
    rect(8, -6, 6, 18, 2);
    rect(20, -6, 6, 18, 2);

    // tail
    stroke(18, 20, 26, 220);
    strokeWeight(2);
    noFill();
    const wag = sin(frameCount * 0.02 + c.idle) * 4;
    beginShape();
    vertex(-28, -22);
    quadraticVertex(-40, -16 + wag, -36, -6);
    endShape();

    // tiny “patch” hint (subtle)
    noStroke();
    fill(28, 34, 46, 120);
```

```
    ellipse(-6, -20, 16, 10);

    pop();
  }

  pop();
}

// ---------- New: grass ----------
drawGrass() {
  push();
  // draw grass behind trees but near the viewer
  // use thin strokes; tufts sway a little
  strokeWeight(2);

  for (let i = 0; i < this.grass.length; i++) {
    const g = this.grass[i];

    // Only bother drawing tufts that might be near the camera view
    // (light perf improvement)
    // We'll keep it loose; still draws plenty.
    // (Camera is handled by translate, so we can skip based on
screen-ish bounds)
    // Not strictly necessary; leaving conservative.
    const sway = sin(frameCount * 0.02 + g.phase) * 3;

    // grass color: slightly lighter than ground, tinted by tuft
    const baseA = 170;
    stroke(22 * g.tint, 28 * g.tint, 34 * g.tint, baseA);

    const baseY = g.y - 6;
    const baseX = g.x;

    for (let b = 0; b < g.clump; b++) {
      const ox = (b - (g.clump - 1) / 2) * (4 * g.s);
      const h = (10 + (b % 3) * 6) * g.s;
      const bend = sway + ox * 0.2;
```

```
      line(baseX + ox, baseY, baseX + ox + bend, baseY - h);
    }
  }

  pop();
}

// ---------- Screen-space overlays ----------
drawBreathOverlay() {
  const breath = sin(this.breathT * 0.35) * 0.5 + 0.5;
  const alpha = 22 + breath * 26;

  noStroke();
  fill(90, 140, 170, alpha);
  rect(0, 0, width, height);
}

drawVignette() {
  push();
  noFill();
  for (let i = 0; i < 18; i++) {
    const t = i / 17;
    const a = 18 * t;
    stroke(0, 0, 0, a);
    const m = t * 70;
    rect(m, m, width - 2 * m, height - 2 * m, 28);
  }
  pop();
}

drawGrain() {
  push();
  blendMode(BLEND);
  tint(255, 18);
  image(grainLayer, 0, 0);
  noTint();
  pop();
}
```

```
 // ---------- Helpers ----------
 sampleGroundY(x) {
   const n1 = noise(x * 0.0013, 100.0);
   const n2 = noise(x * 0.0035, 200.0);
   return 240 + n1 * 210 + n2 * 45;
 }
}
```

```
/*
Meditative Camera Scroll — p5.js (single-file)

New:
- A sketched moon that is SCREEN-LOCKED (stays top-left no matter camera
position)

Controls:
- Move camera: Arrow keys / WASD
- Reset: R
- Toggle UI: H
- Debug toggles:
 1 Far  2 Mid  3 Near  4 Floaters  5 Breath  6 Vignette  7 Grain
*/

let cam, world;
let showHUD = true;

let grainLayer;

// Debug layer toggles
let SHOW_FAR = true;
let SHOW_MID = true;
```

```
let SHOW_NEAR = true;
let SHOW_FLOATERS = true;
let SHOW_BREATH = true;
let SHOW_VIGNETTE = true;
let SHOW_GRAIN = true;

function setup() {
 createCanvas(900, 520);
 pixelDensity(1);

 world = new World();
 cam = new CameraRig();

 textFont("Georgia");

 grainLayer = createGraphics(width, height);
 makeStaticGrain(grainLayer, 18);
}

function draw() {
 background(6, 8, 12);

 world.update();
 cam.update(world);

 // ---------- World space ----------
 push();
 cam.apply();

 if (SHOW_FAR) world.drawFar();
 if (SHOW_MID) world.drawMid();
 if (SHOW_NEAR) world.drawNear();
 if (SHOW_FLOATERS) world.drawFloaters();

 pop();

 // ---------- Screen space ----------
```

```
 // Moon should be screen-locked, so draw it AFTER pop() (no camera
transform)
 world.drawMoonUI();

 if (SHOW_BREATH) world.drawBreathOverlay();
 if (SHOW_VIGNETTE) world.drawVignette();
 if (SHOW_GRAIN) world.drawGrain();

 if (showHUD) drawHUD();
}

// -------------------- Input --------------------
function keyPressed() {
 if (key === "h" || key === "H") showHUD = !showHUD;

 if (key === "r" || key === "R") {
   world = new World();
   cam = new CameraRig();
   grainLayer = createGraphics(width, height);
   makeStaticGrain(grainLayer, 18);
 }

 // Debug toggles
 if (key === "1") SHOW_FAR = !SHOW_FAR;
 if (key === "2") SHOW_MID = !SHOW_MID;
 if (key === "3") SHOW_NEAR = !SHOW_NEAR;
 if (key === "4") SHOW_FLOATERS = !SHOW_FLOATERS;
 if (key === "5") SHOW_BREATH = !SHOW_BREATH;
 if (key === "6") SHOW_VIGNETTE = !SHOW_VIGNETTE;
 if (key === "7") SHOW_GRAIN = !SHOW_GRAIN;
}

// -------------------- HUD --------------------
function drawHUD() {
 const pad = 16;
 noStroke();
 fill(0, 90);
 rect(pad, pad, 560, 94, 14);
```

```
 fill(230, 220);
 textSize(14);
 textAlign(LEFT, TOP);
 text(
   "Night farm (manual camera)\nWASD / Arrows: move camera\nH: hide text
•  R: reset\n1 Far  2 Mid  3 Near  4 Floaters  5 Breath  6 Vignette  7
Grain",
   pad + 12,
   pad + 10
 );
}

// -------------------- Static Grain Builder --------------------
function makeStaticGrain(pg, intensity = 18) {
 pg.pixelDensity(1);
 pg.loadPixels();

 for (let y = 0; y < pg.height; y++) {
   for (let x = 0; x < pg.width; x++) {
     const idx = 4 * (y * pg.width + x);

     const n = noise(x * 0.08, y * 0.08);
     const g = (n - 0.5) * 2 * intensity;

     pg.pixels[idx + 0] = 128 + g;
     pg.pixels[idx + 1] = 128 + g;
     pg.pixels[idx + 2] = 128 + g;
     pg.pixels[idx + 3] = 35;
   }
 }

 pg.updatePixels();
}

// ==================== Camera ====================
class CameraRig {
 constructor() {
```

```
  this.pos = createVector(0, 0);
  this.vel = createVector(0, 0);
}

update(worldRef) {
  let ix = 0;
  let iy = 0;

  if (keyIsDown(LEFT_ARROW) || keyIsDown(65)) ix -= 1;  // A
  if (keyIsDown(RIGHT_ARROW) || keyIsDown(68)) ix += 1; // D
  if (keyIsDown(UP_ARROW) || keyIsDown(87)) iy -= 1;    // W
  if (keyIsDown(DOWN_ARROW) || keyIsDown(83)) iy += 1;  // S

  const mag = Math.hypot(ix, iy);
  if (mag > 0) {
    ix /= mag;
    iy /= mag;
  }

  const accel = 0.58;
  const dampingWhenIdle = 0.82;
  const dampingWhenMoving = 0.92;
  const maxV = 3.0;

  this.vel.x += ix * accel;
  this.vel.y += iy * accel;

  this.vel.x = constrain(this.vel.x, -maxV, maxV);
  this.vel.y = constrain(this.vel.y, -maxV, maxV);

  if (mag === 0) this.vel.mult(dampingWhenIdle);
  else this.vel.mult(dampingWhenMoving);

  this.pos.add(this.vel);

  const b = worldRef.softBounds;
  this.pos.x = constrain(this.pos.x, b.minX, b.maxX);
  this.pos.y = constrain(this.pos.y, b.minY, b.maxY);
```

```
 }

 apply() {
   translate(-this.pos.x, -this.pos.y);
 }
}

// ==================== World ====================
class World {
 constructor() {
   this.seed = floor(random(1e9));
   noiseSeed(this.seed);
   randomSeed(this.seed);

   this.softBounds = {
     minX: -900,
     maxX: 6800,
     minY: -650,
     maxY: 650
   };

   this.breathT = random(1000);

   this.floaters = [];
   for (let i = 0; i < 220; i++) this.floaters.push(this.makeFloater());

   this.stars = [];
   for (let i = 0; i < 420; i++) {
     this.stars.push({
       x: random(this.softBounds.minX, this.softBounds.maxX),
       y: random(this.softBounds.minY, this.softBounds.maxY),
       s: random(0.5, 2.2),
       a: random(70, 170)
     });
   }

   this.cows = this.makeCows(14);
   this.grass = this.makeGrass(160);
```

```
  // Moon doodle seed (stable scribble)
  this.moonSeed = floor(random(1e9));
}

makeFloater() {
  return {
    x: random(this.softBounds.minX, this.softBounds.maxX),
    y: random(-260, 260),
    r: random(1.2, 3.8),
    phase: random(TWO_PI),
    drift: random(0.2, 0.55),
    glow: random(40, 120)
  };
}

makeCows(count) {
  const arr = [];
  for (let i = 0; i < count; i++) {
    const x = random(this.softBounds.minX + 200, this.softBounds.maxX -
200);
    const y = this.sampleGroundY(x);
    const s = random(0.85, 1.5);
    arr.push({
      x,
      y,
      s,
      flip: random() < 0.5 ? -1 : 1,
      idle: random(TWO_PI),
      buddyOffset: random() < 0.35 ? random(-90, 90) : 0
    });
  }
  for (let c of arr) {
    if (c.buddyOffset !== 0) {
      c.x += c.buddyOffset;
      c.y = this.sampleGroundY(c.x);
    }
  }
```

```
  return arr;
}

makeGrass(count) {
  const arr = [];
  for (let i = 0; i < count; i++) {
    const x = random(this.softBounds.minX, this.softBounds.maxX);
    const y = this.sampleGroundY(x);
    const s = random(0.6, 1.35);
    const clump = floor(random(2, 6));
    arr.push({
      x,
      y,
      s,
      clump,
      phase: random(TWO_PI),
      tint: random(0.75, 1.05)
    });
  }
  return arr;
}

update() {
  this.breathT += 0.01;

  for (let p of this.floaters) {
    const w = noise(p.x * 0.0012, p.y * 0.0012, frameCount * 0.002) -
0.5;
    p.x += w * 0.5;
    p.y += sin(frameCount * 0.004 + p.phase) * 0.22 - p.drift;

    if (p.y < -520) {
      p.y = random(120, 420);
      p.x = random(this.softBounds.minX, this.softBounds.maxX);
    }
  }
}
```

```
// ---------- Layers ----------
drawFar() {
  push();
  noStroke();

  for (let st of this.stars) {
    fill(210, 220, 245, st.a);
    circle(st.x, st.y, st.s);
  }

  for (let x = this.softBounds.minX; x < this.softBounds.maxX; x += 40) {
    const gate = noise(x * 0.0016, this.seed * 0.00002);
    if (gate < 0.42) continue;

    const n = noise(x * 0.0007, this.seed * 0.00001);
    const y = -260 + n * 90 + sin(x * 0.0016 + this.breathT * 0.25) * 10;

    let a = 6 + n * 14;
    a = constrain(a, 0, 18);

    fill(110, 160, 190, a);

    const w = 70 + n * 35;
    const h = 36 + n * 18;
    ellipse(x, y, w, h);
  }

  pop();
}

drawMid() {
  push();

  const horizonY = 80;
  const groundBase = 260;

  noStroke();
  for (let y = -400; y < groundBase; y += 2) {
```

```
      const t = map(y, -400, groundBase, 0, 1);
      fill(10 + t * 18, 14 + t * 18, 22 + t * 24, 255);
      rect(this.softBounds.minX, y, this.softBounds.maxX -
this.softBounds.minX, 2);
    }

    fill(12, 16, 22, 220);
    beginShape();
    vertex(this.softBounds.minX, groundBase);
    for (let x = this.softBounds.minX; x <= this.softBounds.maxX; x += 18)
{
      const n = noise(x * 0.0009, 10.0);
      const yy = horizonY + n * 140;
      vertex(x, yy);
    }
    vertex(this.softBounds.maxX, groundBase);
    endShape(CLOSE);

    fill(8, 10, 14, 255);
    beginShape();
    vertex(this.softBounds.minX, height + 900);
    for (let x = this.softBounds.minX; x <= this.softBounds.maxX; x += 16)
{
      const n1 = noise(x * 0.0013, 100.0);
      const n2 = noise(x * 0.0035, 200.0);
      const yy = 240 + n1 * 210 + n2 * 45;
      vertex(x, yy);
    }
    vertex(this.softBounds.maxX, height + 900);
    endShape(CLOSE);

    const waterY = 300;
    for (let x = this.softBounds.minX; x < this.softBounds.maxX; x += 10) {
      const ripple =
        sin(x * 0.02 + this.breathT * 1.2) * 2 +
        (noise(x * 0.01, this.breathT * 0.2) - 0.5) * 3;
      const a = 12 + noise(x * 0.003, 400) * 18;
      stroke(180, 210, 230, a);
```

```
    line(x, waterY + ripple, x + 8, waterY + ripple + 0.4);
  }

  noStroke();
  for (let i = 0; i < 26; i++) {
    const px = i * 280 + 90 + sin(i * 2.1) * 35;
    const py = this.sampleGroundY(px) - 8;
    const tall = 18 + noise(i * 10.1) * 55;

    fill(18, 22, 30, 220);
    rect(px - 2, py - tall, 4, tall, 2);

    fill(200, 220, 240, 40);
    circle(px, py - tall - 6, 18);
  }

  this.drawCows();

  pop();
}

drawNear() {
  push();

  this.drawGrass();

  for (let i = 0; i < 42; i++) {
    const x = i * 160 + 40 + sin(i * 0.9) * 20;
    const gy = this.sampleGroundY(x);
    const sway = sin(frameCount * 0.01 + i) * 2.5;

    stroke(12, 14, 18, 210);
    strokeWeight(3);
    line(x, gy - 8, x + sway, gy - 70);

    noStroke();
    fill(10, 12, 16, 240);
    ellipse(x + sway, gy - 82, 36, 24);
```

```
    ellipse(x + sway + 16, gy - 76, 22, 16);
    ellipse(x + sway - 14, gy - 74, 20, 14);
  }

  pop();
}

drawFloaters() {
  push();
  noStroke();
  for (let p of this.floaters) {
    const tw = sin(frameCount * 0.02 + p.phase) * 0.4 + 0.6;
    fill(190, 210, 230, p.glow * tw);
    circle(p.x, p.y, p.r * (0.8 + tw));
  }
  pop();
}

// ---------- New: screen-locked sketched moon ----------
drawMoonUI() {
  push();
  // top-left position in SCREEN space
  const mx = 86;
  const my = 78;
  const r = 46;

  // very subtle glow
  noStroke();
  fill(210, 220, 240, 16);
  circle(mx, my, r * 2.35);

  // Moon body
  fill(215, 225, 240, 235);
  circle(mx, my, r * 2);

  // Hand-sketched outline: draw a few wobbly rings
  stroke(40, 50, 70, 85);
  strokeWeight(2);
```

```
  noFill();

  // Make scribble stable by seeding noise locally
  const saved = this.moonSeed;
  for (let k = 0; k < 3; k++) {
    beginShape();
    for (let a = 0; a <= TWO_PI + 0.001; a += 0.22) {
      const nx = noise(saved * 0.00001 + k * 10.0, a * 0.6);
      const wob = (nx - 0.5) * 6;
      const rr = r + wob + k * 1.2;
      vertex(mx + cos(a) * rr, my + sin(a) * rr);
    }
    endShape();
  }

  // Craters (sketchy)
  stroke(70, 85, 110, 70);
  strokeWeight(1.5);
  fill(200, 210, 230, 120);
  this.crater(mx - 14, my - 6, 10, 7);
  this.crater(mx + 12, my + 10, 14, 10);
  this.crater(mx + 6, my - 18, 8, 6);

  // A little cross-hatch shadow on lower-left edge
  stroke(90, 105, 130, 55);
  strokeWeight(1);
  for (let i = 0; i < 18; i++) {
    const t = i / 17;
    const x1 = mx - r + t * (r * 1.05);
    const y1 = my + r * 0.35 + t * 10;
    line(x1, y1, x1 + 14, y1 + 8);
  }

  pop();
}

crater(cx, cy, w, h) {
  push();
```

```
  translate(cx, cy);
  // scribbly oval
  noFill();
  for (let k = 0; k < 2; k++) {
    beginShape();
    for (let a = 0; a <= TWO_PI + 0.001; a += 0.35) {
      const wob = (noise(this.moonSeed * 0.00002 + k * 20.0, a * 0.8) -
0.5) * 2.2;
      const rx = (w * 0.5) + wob;
      const ry = (h * 0.5) + wob * 0.7;
      vertex(cos(a) * rx, sin(a) * ry);
    }
    endShape();
  }
  pop();
}

// ---------- New: cows ----------
drawCows() {
  push();
  for (let i = 0; i < this.cows.length; i++) {
    const c = this.cows[i];
    const bob = sin(frameCount * 0.015 + c.idle) * 1.2;

    push();
    translate(c.x, c.y + bob);
    scale(c.flip * c.s, c.s);

    noStroke();
    fill(20, 24, 32, 235);
    ellipse(0, -18, 56, 30);

    ellipse(34, -26, 20, 16);

    fill(24, 28, 38, 235);
    ellipse(42, -24, 10, 8);

    fill(16, 18, 24, 240);
```

```
    rect(-18, -6, 6, 18, 2);
    rect(-6, -6, 6, 18, 2);
    rect(8, -6, 6, 18, 2);
    rect(20, -6, 6, 18, 2);

    stroke(18, 20, 26, 220);
    strokeWeight(2);
    noFill();
    const wag = sin(frameCount * 0.02 + c.idle) * 4;
    beginShape();
    vertex(-28, -22);
    quadraticVertex(-40, -16 + wag, -36, -6);
    endShape();

    noStroke();
    fill(28, 34, 46, 120);
    ellipse(-6, -20, 16, 10);

    pop();
  }
  pop();
}

// ---------- New: grass ----------
drawGrass() {
  push();
  strokeWeight(2);

  for (let i = 0; i < this.grass.length; i++) {
    const g = this.grass[i];
    const sway = sin(frameCount * 0.02 + g.phase) * 3;

    stroke(22 * g.tint, 28 * g.tint, 34 * g.tint, 170);

    const baseY = g.y - 6;
    const baseX = g.x;

    for (let b = 0; b < g.clump; b++) {
```

```
      const ox = (b - (g.clump - 1) / 2) * (4 * g.s);
      const hh = (10 + (b % 3) * 6) * g.s;
      const bend = sway + ox * 0.2;
      line(baseX + ox, baseY, baseX + ox + bend, baseY - hh);
    }
  }

  pop();
}

// ---------- Screen-space overlays ----------
drawBreathOverlay() {
  const breath = sin(this.breathT * 0.35) * 0.5 + 0.5;
  const alpha = 22 + breath * 26;

  noStroke();
  fill(90, 140, 170, alpha);
  rect(0, 0, width, height);
}

drawVignette() {
  push();
  noFill();
  for (let i = 0; i < 18; i++) {
    const t = i / 17;
    const a = 18 * t;
    stroke(0, 0, 0, a);
    const m = t * 70;
    rect(m, m, width - 2 * m, height - 2 * m, 28);
  }
  pop();
}

drawGrain() {
  push();
  blendMode(BLEND);
  tint(255, 18);
  image(grainLayer, 0, 0);
```

```
  noTint();
  pop();
 }

 // ---------- Helpers ----------
 sampleGroundY(x) {
  const n1 = noise(x * 0.0013, 100.0);
  const n2 = noise(x * 0.0035, 200.0);
  return 240 + n1 * 210 + n2 * 45;
 }
}
```

```
world.drawMoonUI();
```

```
this.moon = {
 x: this.softBounds.minX + 260,
 y: this.softBounds.minY + 160, // up in the sky
 r: 46
};
```

```
this.drawMoonWorld();
```

```
drawMoonWorld() {
 push();

 const mx = this.moon.x;
 const my = this.moon.y;
 const r = this.moon.r;

 // subtle glow
 noStroke();
 fill(210, 220, 240, 16);
 circle(mx, my, r * 2.35);

 // Moon body
 fill(215, 225, 240, 235);
 circle(mx, my, r * 2);
```

```
// sketched outline
stroke(40, 50, 70, 85);
strokeWeight(2);
noFill();

const saved = this.moonSeed;
for (let k = 0; k < 3; k++) {
  beginShape();
  for (let a = 0; a <= TWO_PI + 0.001; a += 0.22) {
    const nx = noise(saved * 0.00001 + k * 10.0, a * 0.6);
    const wob = (nx - 0.5) * 6;
    const rr = r + wob + k * 1.2;
    vertex(mx + cos(a) * rr, my + sin(a) * rr);
  }
  endShape();
}

// craters
stroke(70, 85, 110, 70);
strokeWeight(1.5);
fill(200, 210, 230, 120);
this.crater(mx - 14, my - 6, 10, 7);
this.crater(mx + 12, my + 10, 14, 10);
this.crater(mx + 6, my - 18, 8, 6);

// hatch shading
stroke(90, 105, 130, 55);
```

```
 strokeWeight(1);
 for (let i = 0; i < 18; i++) {
   const t = i / 17;
   const x1 = mx - r + t * (r * 1.05);
   const y1 = my + r * 0.35 + t * 10;
   line(x1, y1, x1 + 14, y1 + 8);
 }

 pop();
}
```

```
class World {
 constructor() {
   ...
```

```
  this.softBounds = {
    minX: -900,
    maxX: 6800,
    minY: -650,
    maxY: 650
  };

  this.breathT = random(1000);
  ...
```

```
  this.moon = {
    x: this.softBounds.minX + 260,
    y: this.softBounds.minY + 160,
    r: 46
  };
```

```
/*
Meditative Camera Scroll — p5.js (single-file)

Update (moon):
- Moon is now WORLD-LOCKED (fixed in the sky in world coordinates).
- It is drawn in drawFar() (world-space), so it does NOT stay pinned to
the screen.

Other features kept:
- Night farm vibe (cows + grass tufts)
- Manual camera ONLY (WASD / Arrows)
- FAR aurora reworked to prevent bright "white stream"
- Stable grain (Option B draw)
```

```
- Debug toggles 1-7

Controls:
- Move camera: Arrow keys / WASD
- Reset: R
- Toggle UI: H
- Debug toggles:
 1 Far  2 Mid  3 Near  4 Floaters  5 Breath  6 Vignette  7 Grain
*/

let cam, world;
let showHUD = true;

let grainLayer;

// Debug layer toggles
let SHOW_FAR = true;
let SHOW_MID = true;
let SHOW_NEAR = true;
let SHOW_FLOATERS = true;
let SHOW_BREATH = true;
let SHOW_VIGNETTE = true;
let SHOW_GRAIN = true;

function setup() {
 createCanvas(900, 520);
 pixelDensity(1);

 world = new World();
 cam = new CameraRig();

 textFont("Georgia");

 grainLayer = createGraphics(width, height);
 makeStaticGrain(grainLayer, 18);
}

function draw() {
```

```
 background(6, 8, 12);

 world.update();
 cam.update(world);

 // ---------- World space ----------
 push();
 cam.apply();

 if (SHOW_FAR) world.drawFar();
 if (SHOW_MID) world.drawMid();
 if (SHOW_NEAR) world.drawNear();
 if (SHOW_FLOATERS) world.drawFloaters();

 pop();

 // ---------- Screen space overlays ----------
 if (SHOW_BREATH) world.drawBreathOverlay();
 if (SHOW_VIGNETTE) world.drawVignette();
 if (SHOW_GRAIN) world.drawGrain();

 if (showHUD) drawHUD();
}

// -------------------- Input --------------------
function keyPressed() {
 if (key === "h" || key === "H") showHUD = !showHUD;

 if (key === "r" || key === "R") {
   world = new World();
   cam = new CameraRig();
   grainLayer = createGraphics(width, height);
   makeStaticGrain(grainLayer, 18);
 }

 // Debug toggles
 if (key === "1") SHOW_FAR = !SHOW_FAR;
 if (key === "2") SHOW_MID = !SHOW_MID;
```

```
 if (key === "3") SHOW_NEAR = !SHOW_NEAR;
 if (key === "4") SHOW_FLOATERS = !SHOW_FLOATERS;
 if (key === "5") SHOW_BREATH = !SHOW_BREATH;
 if (key === "6") SHOW_VIGNETTE = !SHOW_VIGNETTE;
 if (key === "7") SHOW_GRAIN = !SHOW_GRAIN;
}

// -------------------- HUD --------------------
function drawHUD() {
 const pad = 16;
 noStroke();
 fill(0, 90);
 rect(pad, pad, 560, 94, 14);

 fill(230, 220);
 textSize(14);
 textAlign(LEFT, TOP);
 text(
   "Night farm (manual camera)\nWASD / Arrows: move camera\nH: hide text
•  R: reset\n1 Far  2 Mid  3 Near  4 Floaters  5 Breath  6 Vignette  7
Grain",
   pad + 12,
   pad + 10
 );
}

// -------------------- Static Grain Builder --------------------
function makeStaticGrain(pg, intensity = 18) {
 pg.pixelDensity(1);
 pg.loadPixels();

 for (let y = 0; y < pg.height; y++) {
   for (let x = 0; x < pg.width; x++) {
     const idx = 4 * (y * pg.width + x);

     const n = noise(x * 0.08, y * 0.08);
     const g = (n - 0.5) * 2 * intensity;
```

```
      pg.pixels[idx + 0] = 128 + g;
      pg.pixels[idx + 1] = 128 + g;
      pg.pixels[idx + 2] = 128 + g;
      pg.pixels[idx + 3] = 35;
    }
  }

  pg.updatePixels();
}

// ==================== Camera ====================
class CameraRig {
  constructor() {
    this.pos = createVector(0, 0);
    this.vel = createVector(0, 0);
  }

  // Manual-only camera
  update(worldRef) {
    let ix = 0;
    let iy = 0;

    if (keyIsDown(LEFT_ARROW) || keyIsDown(65)) ix -= 1;  // A
    if (keyIsDown(RIGHT_ARROW) || keyIsDown(68)) ix += 1; // D
    if (keyIsDown(UP_ARROW) || keyIsDown(87)) iy -= 1;    // W
    if (keyIsDown(DOWN_ARROW) || keyIsDown(83)) iy += 1;  // S

    const mag = Math.hypot(ix, iy);
    if (mag > 0) {
      ix /= mag;
      iy /= mag;
    }

    const accel = 0.58;
    const dampingWhenIdle = 0.82;
    const dampingWhenMoving = 0.92;
    const maxV = 3.0;
```

```
    this.vel.x += ix * accel;
    this.vel.y += iy * accel;

    this.vel.x = constrain(this.vel.x, -maxV, maxV);
    this.vel.y = constrain(this.vel.y, -maxV, maxV);

    if (mag === 0) this.vel.mult(dampingWhenIdle);
    else this.vel.mult(dampingWhenMoving);

    this.pos.add(this.vel);

    const b = worldRef.softBounds;
    this.pos.x = constrain(this.pos.x, b.minX, b.maxX);
    this.pos.y = constrain(this.pos.y, b.minY, b.maxY);
  }

  apply() {
    translate(-this.pos.x, -this.pos.y);
  }
}

// ==================== World ====================
class World {
  constructor() {
    this.seed = floor(random(1e9));
    noiseSeed(this.seed);
    randomSeed(this.seed);

    this.softBounds = {
      minX: -900,
      maxX: 6800,
      minY: -650,
      maxY: 650
    };

    // --- World-locked moon (fixed in the sky) ---
    this.moon = {
      x: this.softBounds.minX + 260,
```

```
    y: this.softBounds.minY + 160,
    r: 46
  };
  this.moonSeed = floor(random(1e9)); // stable scribble seed

  this.breathT = random(1000);

  // particles
  this.floaters = [];
  for (let i = 0; i < 220; i++) this.floaters.push(this.makeFloater());

  // stars
  this.stars = [];
  for (let i = 0; i < 420; i++) {
    this.stars.push({
      x: random(this.softBounds.minX, this.softBounds.maxX),
      y: random(this.softBounds.minY, this.softBounds.maxY),
      s: random(0.5, 2.2),
      a: random(70, 170)
    });
  }

  // detail
  this.cows = this.makeCows(14);
  this.grass = this.makeGrass(160);
}

makeFloater() {
  return {
    x: random(this.softBounds.minX, this.softBounds.maxX),
    y: random(-260, 260),
    r: random(1.2, 3.8),
    phase: random(TWO_PI),
    drift: random(0.2, 0.55),
    glow: random(40, 120)
  };
}
```

```
makeCows(count) {
  const arr = [];
  for (let i = 0; i < count; i++) {
    const x = random(this.softBounds.minX + 200, this.softBounds.maxX -
200);
    const y = this.sampleGroundY(x);
    const s = random(0.85, 1.5);
    arr.push({
      x,
      y,
      s,
      flip: random() < 0.5 ? -1 : 1,
      idle: random(TWO_PI),
      buddyOffset: random() < 0.35 ? random(-90, 90) : 0
    });
  }
  for (let c of arr) {
    if (c.buddyOffset !== 0) {
      c.x += c.buddyOffset;
      c.y = this.sampleGroundY(c.x);
    }
  }
  return arr;
}

makeGrass(count) {
  const arr = [];
  for (let i = 0; i < count; i++) {
    const x = random(this.softBounds.minX, this.softBounds.maxX);
    const y = this.sampleGroundY(x);
    const s = random(0.6, 1.35);
    const clump = floor(random(2, 6));
    arr.push({
      x,
      y,
      s,
      clump,
      phase: random(TWO_PI),
```

```
      tint: random(0.75, 1.05)
    });
  }
  return arr;
}

update() {
  this.breathT += 0.01;

  for (let p of this.floaters) {
    const w = noise(p.x * 0.0012, p.y * 0.0012, frameCount * 0.002) -
0.5;
    p.x += w * 0.5;
    p.y += sin(frameCount * 0.004 + p.phase) * 0.22 - p.drift;

    if (p.y < -520) {
      p.y = random(120, 420);
      p.x = random(this.softBounds.minX, this.softBounds.maxX);
    }
  }
}

// ---------- Layers ----------
drawFar() {
  push();
  noStroke();

  // WORLD-LOCKED moon lives in the far sky
  this.drawMoonWorld();

  // stars
  for (let st of this.stars) {
    fill(210, 220, 245, st.a);
    circle(st.x, st.y, st.s);
  }

  // safer aurora (segmented, capped alpha)
  for (let x = this.softBounds.minX; x < this.softBounds.maxX; x += 40) {
```

```
      const gate = noise(x * 0.0016, this.seed * 0.00002);
      if (gate < 0.42) continue;

      const n = noise(x * 0.0007, this.seed * 0.00001);
      const y = -260 + n * 90 + sin(x * 0.0016 + this.breathT * 0.25) * 10;

      let a = 6 + n * 14;
      a = constrain(a, 0, 18);

      fill(110, 160, 190, a);

      const w = 70 + n * 35;
      const h = 36 + n * 18;
      ellipse(x, y, w, h);
    }

    pop();
  }

  drawMid() {
    push();

    const horizonY = 80;
    const groundBase = 260;

    noStroke();
    for (let y = -400; y < groundBase; y += 2) {
      const t = map(y, -400, groundBase, 0, 1);
      fill(10 + t * 18, 14 + t * 18, 22 + t * 24, 255);
      rect(this.softBounds.minX, y, this.softBounds.maxX -
this.softBounds.minX, 2);
    }

    fill(12, 16, 22, 220);
    beginShape();
    vertex(this.softBounds.minX, groundBase);
    for (let x = this.softBounds.minX; x <= this.softBounds.maxX; x += 18)
{
```

```
      const n = noise(x * 0.0009, 10.0);
      const yy = horizonY + n * 140;
      vertex(x, yy);
    }
    vertex(this.softBounds.maxX, groundBase);
    endShape(CLOSE);

    fill(8, 10, 14, 255);
    beginShape();
    vertex(this.softBounds.minX, height + 900);
    for (let x = this.softBounds.minX; x <= this.softBounds.maxX; x += 16)
{
      const n1 = noise(x * 0.0013, 100.0);
      const n2 = noise(x * 0.0035, 200.0);
      const yy = 240 + n1 * 210 + n2 * 45;
      vertex(x, yy);
    }
    vertex(this.softBounds.maxX, height + 900);
    endShape(CLOSE);

    const waterY = 300;
    for (let x = this.softBounds.minX; x < this.softBounds.maxX; x += 10) {
      const ripple =
        sin(x * 0.02 + this.breathT * 1.2) * 2 +
        (noise(x * 0.01, this.breathT * 0.2) - 0.5) * 3;
      const a = 12 + noise(x * 0.003, 400) * 18;
      stroke(180, 210, 230, a);
      line(x, waterY + ripple, x + 8, waterY + ripple + 0.4);
    }

    noStroke();
    for (let i = 0; i < 26; i++) {
      const px = i * 280 + 90 + sin(i * 2.1) * 35;
      const py = this.sampleGroundY(px) - 8;
      const tall = 18 + noise(i * 10.1) * 55;

      fill(18, 22, 30, 220);
      rect(px - 2, py - tall, 4, tall, 2);
```

```
    fill(200, 220, 240, 40);
    circle(px, py - tall - 6, 18);
  }

  this.drawCows();

  pop();
}

drawNear() {
  push();

  this.drawGrass();

  for (let i = 0; i < 42; i++) {
    const x = i * 160 + 40 + sin(i * 0.9) * 20;
    const gy = this.sampleGroundY(x);
    const sway = sin(frameCount * 0.01 + i) * 2.5;

    stroke(12, 14, 18, 210);
    strokeWeight(3);
    line(x, gy - 8, x + sway, gy - 70);

    noStroke();
    fill(10, 12, 16, 240);
    ellipse(x + sway, gy - 82, 36, 24);
    ellipse(x + sway + 16, gy - 76, 22, 16);
    ellipse(x + sway - 14, gy - 74, 20, 14);
  }

  pop();
}

drawFloaters() {
  push();
  noStroke();
  for (let p of this.floaters) {
```

```
    const tw = sin(frameCount * 0.02 + p.phase) * 0.4 + 0.6;
    fill(190, 210, 230, p.glow * tw);
    circle(p.x, p.y, p.r * (0.8 + tw));
  }
  pop();
}

// ---------- Moon (WORLD-LOCKED) ----------
drawMoonWorld() {
  push();

  const mx = this.moon.x;
  const my = this.moon.y;
  const r = this.moon.r;

  // subtle glow
  noStroke();
  fill(210, 220, 240, 16);
  circle(mx, my, r * 2.35);

  // body
  fill(215, 225, 240, 235);
  circle(mx, my, r * 2);

  // sketched outline
  stroke(40, 50, 70, 85);
  strokeWeight(2);
  noFill();

  const saved = this.moonSeed;
  for (let k = 0; k < 3; k++) {
    beginShape();
    for (let a = 0; a <= TWO_PI + 0.001; a += 0.22) {
      const nx = noise(saved * 0.00001 + k * 10.0, a * 0.6);
      const wob = (nx - 0.5) * 6;
      const rr = r + wob + k * 1.2;
      vertex(mx + cos(a) * rr, my + sin(a) * rr);
    }
```

```
    endShape();
  }

  // craters
  stroke(70, 85, 110, 70);
  strokeWeight(1.5);
  fill(200, 210, 230, 120);
  this.crater(mx - 14, my - 6, 10, 7);
  this.crater(mx + 12, my + 10, 14, 10);
  this.crater(mx + 6, my - 18, 8, 6);

  // hatch shading
  stroke(90, 105, 130, 55);
  strokeWeight(1);
  for (let i = 0; i < 18; i++) {
    const t = i / 17;
    const x1 = mx - r + t * (r * 1.05);
    const y1 = my + r * 0.35 + t * 10;
    line(x1, y1, x1 + 14, y1 + 8);
  }

  pop();
}

crater(cx, cy, w, h) {
  push();
  translate(cx, cy);
  noFill();
  for (let k = 0; k < 2; k++) {
    beginShape();
    for (let a = 0; a <= TWO_PI + 0.001; a += 0.35) {
      const wob = (noise(this.moonSeed * 0.00002 + k * 20.0, a * 0.8) -
0.5) * 2.2;
      const rx = (w * 0.5) + wob;
      const ry = (h * 0.5) + wob * 0.7;
      vertex(cos(a) * rx, sin(a) * ry);
    }
    endShape();
```

```
  }
  pop();
}

// ---------- Cows ----------
drawCows() {
  push();
  for (let i = 0; i < this.cows.length; i++) {
    const c = this.cows[i];
    const bob = sin(frameCount * 0.015 + c.idle) * 1.2;

    push();
    translate(c.x, c.y + bob);
    scale(c.flip * c.s, c.s);

    noStroke();
    fill(20, 24, 32, 235);
    ellipse(0, -18, 56, 30);

    ellipse(34, -26, 20, 16);

    fill(24, 28, 38, 235);
    ellipse(42, -24, 10, 8);

    fill(16, 18, 24, 240);
    rect(-18, -6, 6, 18, 2);
    rect(-6, -6, 6, 18, 2);
    rect(8, -6, 6, 18, 2);
    rect(20, -6, 6, 18, 2);

    stroke(18, 20, 26, 220);
    strokeWeight(2);
    noFill();
    const wag = sin(frameCount * 0.02 + c.idle) * 4;
    beginShape();
    vertex(-28, -22);
    quadraticVertex(-40, -16 + wag, -36, -6);
    endShape();
```

```
    noStroke();
    fill(28, 34, 46, 120);
    ellipse(-6, -20, 16, 10);

    pop();
  }
  pop();
}

// ---------- Grass ----------
drawGrass() {
  push();
  strokeWeight(2);

  for (let i = 0; i < this.grass.length; i++) {
    const g = this.grass[i];
    const sway = sin(frameCount * 0.02 + g.phase) * 3;

    stroke(22 * g.tint, 28 * g.tint, 34 * g.tint, 170);

    const baseY = g.y - 6;
    const baseX = g.x;

    for (let b = 0; b < g.clump; b++) {
      const ox = (b - (g.clump - 1) / 2) * (4 * g.s);
      const hh = (10 + (b % 3) * 6) * g.s;
      const bend = sway + ox * 0.2;
      line(baseX + ox, baseY, baseX + ox + bend, baseY - hh);
    }
  }

  pop();
}

// ---------- Screen-space overlays ----------
drawBreathOverlay() {
  const breath = sin(this.breathT * 0.35) * 0.5 + 0.5;
```

```
  const alpha = 22 + breath * 26;

  noStroke();
  fill(90, 140, 170, alpha);
  rect(0, 0, width, height);
}

drawVignette() {
  push();
  noFill();
  for (let i = 0; i < 18; i++) {
    const t = i / 17;
    const a = 18 * t;
    stroke(0, 0, 0, a);
    const m = t * 70;
    rect(m, m, width - 2 * m, height - 2 * m, 28);
  }
  pop();
}

drawGrain() {
  push();
  blendMode(BLEND);
  tint(255, 18);
  image(grainLayer, 0, 0);
  noTint();
  pop();
}

// ---------- Helpers ----------
sampleGroundY(x) {
  const n1 = noise(x * 0.0013, 100.0);
  const n2 = noise(x * 0.0035, 200.0);
  return 240 + n1 * 210 + n2 * 45;
}
}
```

Thought for 13s

```
world.drawMoonParallax(cam.pos.x);
```

```
pop();

// Moon: moves with camera X, stays in sky (screen Y)
world.drawMoonParallax(cam.pos.x);

if (SHOW_BREATH) world.drawBreathOverlay();
...
```

```
this.drawMoonWorld();
```

```
drawMoonParallax(camX) {
 push();

 // ثابت في السماء (screen space)
 const my = 78;    // top-left sky position
 const r = this.moon.r;

 // Parallax: lower = farther (moves less than the camera)
 const parallax = 0.18;

 // Base position in screen coords (top-left)
 const baseX = 86;

 // As camX increases (camera moves right), the moon shifts left a bit
 let mx = baseX - camX * parallax;

 // Keep it from disappearing forever: wrap across the screen nicely
 const wrapW = width + r * 4;
 mx = ((mx % wrapW) + wrapW) % wrapW - r * 2;

 // glow
 noStroke();
```

```
fill(210, 220, 240, 16);
circle(mx, my, r * 2.35);

// body
fill(215, 225, 240, 235);
circle(mx, my, r * 2);

// sketched outline
stroke(40, 50, 70, 85);
strokeWeight(2);
noFill();

const saved = this.moonSeed;
for (let k = 0; k < 3; k++) {
  beginShape();
  for (let a = 0; a <= TWO_PI + 0.001; a += 0.22) {
    const nx = noise(saved * 0.00001 + k * 10.0, a * 0.6);
    const wob = (nx - 0.5) * 6;
    const rr = r + wob + k * 1.2;
    vertex(mx + cos(a) * rr, my + sin(a) * rr);
  }
  endShape();
}

// craters
stroke(70, 85, 110, 70);
strokeWeight(1.5);
fill(200, 210, 230, 120);
```

```
 this.crater(mx - 14, my - 6, 10, 7);
 this.crater(mx + 12, my + 10, 14, 10);
 this.crater(mx + 6, my - 18, 8, 6);

 // hatch shading
 stroke(90, 105, 130, 55);
 strokeWeight(1);
 for (let i = 0; i < 18; i++) {
   const t = i / 17;
   const x1 = mx - r + t * (r * 1.05);
   const y1 = my + r * 0.35 + t * 10;
   line(x1, y1, x1 + 14, y1 + 8);
 }

 pop();
}
```

```
/*
Meditative Camera Scroll — p5.js (single-file)
```

```
Moon update (cleaned up):
- Removed the world-locked moon (no fixed world position, no
drawMoonWorld()).
- Added a PARALLAX moon that:
 - moves left/right with camera X (parallax)
 - stays in the sky (fixed screen Y)
 - wraps so it doesn't disappear forever

Other features kept:
- Night farm vibe (cows + grass tufts)
- Manual camera ONLY (WASD / Arrows)
- FAR aurora reworked to prevent bright "white stream"
- Stable grain (Option B draw)
- Debug toggles 1-7

Controls:
- Move camera: Arrow keys / WASD
- Reset: R
- Toggle UI: H
- Debug toggles:
 1 Far  2 Mid  3 Near  4 Floaters  5 Breath  6 Vignette  7 Grain
*/

let cam, world;
let showHUD = true;

let grainLayer;

// Debug layer toggles
let SHOW_FAR = true;
let SHOW_MID = true;
let SHOW_NEAR = true;
let SHOW_FLOATERS = true;
let SHOW_BREATH = true;
let SHOW_VIGNETTE = true;
let SHOW_GRAIN = true;

function setup() {
```

```
 createCanvas(900, 520);
 pixelDensity(1);

 world = new World();
 cam = new CameraRig();

 textFont("Georgia");

 grainLayer = createGraphics(width, height);
 makeStaticGrain(grainLayer, 18);
}

function draw() {
 background(6, 8, 12);

 world.update();
 cam.update(world);

 // ---------- World space ----------
 push();
 cam.apply();

 if (SHOW_FAR) world.drawFar();
 if (SHOW_MID) world.drawMid();
 if (SHOW_NEAR) world.drawNear();
 if (SHOW_FLOATERS) world.drawFloaters();

 pop();

 // ---------- Screen space (moon parallax + overlays) ----------
 world.drawMoonParallax(cam.pos.x);

 if (SHOW_BREATH) world.drawBreathOverlay();
 if (SHOW_VIGNETTE) world.drawVignette();
 if (SHOW_GRAIN) world.drawGrain();

 if (showHUD) drawHUD();
}
```

```
// -------------------- Input --------------------
function keyPressed() {
 if (key === "h" || key === "H") showHUD = !showHUD;

 if (key === "r" || key === "R") {
   world = new World();
   cam = new CameraRig();
   grainLayer = createGraphics(width, height);
   makeStaticGrain(grainLayer, 18);
 }

 // Debug toggles
 if (key === "1") SHOW_FAR = !SHOW_FAR;
 if (key === "2") SHOW_MID = !SHOW_MID;
 if (key === "3") SHOW_NEAR = !SHOW_NEAR;
 if (key === "4") SHOW_FLOATERS = !SHOW_FLOATERS;
 if (key === "5") SHOW_BREATH = !SHOW_BREATH;
 if (key === "6") SHOW_VIGNETTE = !SHOW_VIGNETTE;
 if (key === "7") SHOW_GRAIN = !SHOW_GRAIN;
}

// -------------------- HUD --------------------
function drawHUD() {
 const pad = 16;
 noStroke();
 fill(0, 90);
 rect(pad, pad, 560, 94, 14);

 fill(230, 220);
 textSize(14);
 textAlign(LEFT, TOP);
 text(
   "Night farm (manual camera)\nWASD / Arrows: move camera\nH: hide text
•  R: reset\n1 Far  2 Mid  3 Near  4 Floaters  5 Breath  6 Vignette  7
Grain",
   pad + 12,
   pad + 10
```

```
 );
}

// -------------------- Static Grain Builder --------------------
function makeStaticGrain(pg, intensity = 18) {
 pg.pixelDensity(1);
 pg.loadPixels();

 for (let y = 0; y < pg.height; y++) {
   for (let x = 0; x < pg.width; x++) {
     const idx = 4 * (y * pg.width + x);

     const n = noise(x * 0.08, y * 0.08);
     const g = (n - 0.5) * 2 * intensity;

     pg.pixels[idx + 0] = 128 + g;
     pg.pixels[idx + 1] = 128 + g;
     pg.pixels[idx + 2] = 128 + g;
     pg.pixels[idx + 3] = 35;
   }
 }

 pg.updatePixels();
}

// ==================== Camera ====================
class CameraRig {
 constructor() {
   this.pos = createVector(0, 0);
   this.vel = createVector(0, 0);
 }

 // Manual-only camera
 update(worldRef) {
   let ix = 0;
   let iy = 0;

   if (keyIsDown(LEFT_ARROW) || keyIsDown(65)) ix -= 1;  // A
```

```
    if (keyIsDown(RIGHT_ARROW) || keyIsDown(68)) ix += 1; // D
    if (keyIsDown(UP_ARROW) || keyIsDown(87)) iy -= 1;    // W
    if (keyIsDown(DOWN_ARROW) || keyIsDown(83)) iy += 1;  // S

    const mag = Math.hypot(ix, iy);
    if (mag > 0) {
      ix /= mag;
      iy /= mag;
    }

    const accel = 0.58;
    const dampingWhenIdle = 0.82;
    const dampingWhenMoving = 0.92;
    const maxV = 3.0;

    this.vel.x += ix * accel;
    this.vel.y += iy * accel;

    this.vel.x = constrain(this.vel.x, -maxV, maxV);
    this.vel.y = constrain(this.vel.y, -maxV, maxV);

    if (mag === 0) this.vel.mult(dampingWhenIdle);
    else this.vel.mult(dampingWhenMoving);

    this.pos.add(this.vel);

    const b = worldRef.softBounds;
    this.pos.x = constrain(this.pos.x, b.minX, b.maxX);
    this.pos.y = constrain(this.pos.y, b.minY, b.maxY);
  }

  apply() {
    translate(-this.pos.x, -this.pos.y);
  }
}

// ==================== World ====================
class World {
```

```
constructor() {
  this.seed = floor(random(1e9));
  noiseSeed(this.seed);
  randomSeed(this.seed);

  this.softBounds = {
    minX: -900,
    maxX: 6800,
    minY: -650,
    maxY: 650
  };

  this.breathT = random(1000);

  // particles
  this.floaters = [];
  for (let i = 0; i < 220; i++) this.floaters.push(this.makeFloater());

  // stars
  this.stars = [];
  for (let i = 0; i < 420; i++) {
    this.stars.push({
      x: random(this.softBounds.minX, this.softBounds.maxX),
      y: random(this.softBounds.minY, this.softBounds.maxY),
      s: random(0.5, 2.2),
      a: random(70, 170)
    });
  }

  // detail
  this.cows = this.makeCows(14);
  this.grass = this.makeGrass(160);

  // Moon (screen-space parallax)
  this.moonR = 46;
  this.moonSeed = floor(random(1e9)); // stable scribble seed
}
```

```
makeFloater() {
  return {
    x: random(this.softBounds.minX, this.softBounds.maxX),
    y: random(-260, 260),
    r: random(1.2, 3.8),
    phase: random(TWO_PI),
    drift: random(0.2, 0.55),
    glow: random(40, 120)
  };
}

makeCows(count) {
  const arr = [];
  for (let i = 0; i < count; i++) {
    const x = random(this.softBounds.minX + 200, this.softBounds.maxX -
200);
    const y = this.sampleGroundY(x);
    const s = random(0.85, 1.5);
    arr.push({
      x,
      y,
      s,
      flip: random() < 0.5 ? -1 : 1,
      idle: random(TWO_PI),
      buddyOffset: random() < 0.35 ? random(-90, 90) : 0
    });
  }
  for (let c of arr) {
    if (c.buddyOffset !== 0) {
      c.x += c.buddyOffset;
      c.y = this.sampleGroundY(c.x);
    }
  }
  return arr;
}

makeGrass(count) {
  const arr = [];
```

```
  for (let i = 0; i < count; i++) {
    const x = random(this.softBounds.minX, this.softBounds.maxX);
    const y = this.sampleGroundY(x);
    const s = random(0.6, 1.35);
    const clump = floor(random(2, 6));
    arr.push({
      x,
      y,
      s,
      clump,
      phase: random(TWO_PI),
      tint: random(0.75, 1.05)
    });
  }
  return arr;
}

update() {
  this.breathT += 0.01;

  for (let p of this.floaters) {
    const w = noise(p.x * 0.0012, p.y * 0.0012, frameCount * 0.002) -
0.5;
    p.x += w * 0.5;
    p.y += sin(frameCount * 0.004 + p.phase) * 0.22 - p.drift;

    if (p.y < -520) {
      p.y = random(120, 420);
      p.x = random(this.softBounds.minX, this.softBounds.maxX);
    }
  }
}

// ---------- Layers ----------
drawFar() {
  push();
  noStroke();
```

```
  // stars
  for (let st of this.stars) {
    fill(210, 220, 245, st.a);
    circle(st.x, st.y, st.s);
  }

  // safer aurora (segmented, capped alpha)
  for (let x = this.softBounds.minX; x < this.softBounds.maxX; x += 40) {
    const gate = noise(x * 0.0016, this.seed * 0.00002);
    if (gate < 0.42) continue;

    const n = noise(x * 0.0007, this.seed * 0.00001);
    const y = -260 + n * 90 + sin(x * 0.0016 + this.breathT * 0.25) * 10;

    let a = 6 + n * 14;
    a = constrain(a, 0, 18);

    fill(110, 160, 190, a);

    const w = 70 + n * 35;
    const h = 36 + n * 18;
    ellipse(x, y, w, h);
  }

  pop();
}

drawMid() {
  push();

  const horizonY = 80;
  const groundBase = 260;

  // sky gradient slab
  noStroke();
  for (let y = -400; y < groundBase; y += 2) {
    const t = map(y, -400, groundBase, 0, 1);
    fill(10 + t * 18, 14 + t * 18, 22 + t * 24, 255);
```

```
      rect(this.softBounds.minX, y, this.softBounds.maxX -
this.softBounds.minX, 2);
    }

    // distant hills
    fill(12, 16, 22, 220);
    beginShape();
    vertex(this.softBounds.minX, groundBase);
    for (let x = this.softBounds.minX; x <= this.softBounds.maxX; x += 18)
{
      const n = noise(x * 0.0009, 10.0);
      const yy = horizonY + n * 140;
      vertex(x, yy);
    }
    vertex(this.softBounds.maxX, groundBase);
    endShape(CLOSE);

    // main ground
    fill(8, 10, 14, 255);
    beginShape();
    vertex(this.softBounds.minX, height + 900);
    for (let x = this.softBounds.minX; x <= this.softBounds.maxX; x += 16)
{
      const n1 = noise(x * 0.0013, 100.0);
      const n2 = noise(x * 0.0035, 200.0);
      const yy = 240 + n1 * 210 + n2 * 45;
      vertex(x, yy);
    }
    vertex(this.softBounds.maxX, height + 900);
    endShape(CLOSE);

    // water line
    const waterY = 300;
    for (let x = this.softBounds.minX; x < this.softBounds.maxX; x += 10) {
      const ripple =
        sin(x * 0.02 + this.breathT * 1.2) * 2 +
        (noise(x * 0.01, this.breathT * 0.2) - 0.5) * 3;
      const a = 12 + noise(x * 0.003, 400) * 18;
```

```
    stroke(180, 210, 230, a);
    line(x, waterY + ripple, x + 8, waterY + ripple + 0.4);
  }

  // tiny posts
  noStroke();
  for (let i = 0; i < 26; i++) {
    const px = i * 280 + 90 + sin(i * 2.1) * 35;
    const py = this.sampleGroundY(px) - 8;
    const tall = 18 + noise(i * 10.1) * 55;

    fill(18, 22, 30, 220);
    rect(px - 2, py - tall, 4, tall, 2);

    fill(200, 220, 240, 40);
    circle(px, py - tall - 6, 18);
  }

  // cows
  this.drawCows();

  pop();
}

drawNear() {
  push();

  // grass tufts
  this.drawGrass();

  // sparse trees/reeds
  for (let i = 0; i < 42; i++) {
    const x = i * 160 + 40 + sin(i * 0.9) * 20;
    const gy = this.sampleGroundY(x);
    const sway = sin(frameCount * 0.01 + i) * 2.5;

    stroke(12, 14, 18, 210);
    strokeWeight(3);
```

```
    line(x, gy - 8, x + sway, gy - 70);

    noStroke();
    fill(10, 12, 16, 240);
    ellipse(x + sway, gy - 82, 36, 24);
    ellipse(x + sway + 16, gy - 76, 22, 16);
    ellipse(x + sway - 14, gy - 74, 20, 14);
  }

  pop();
}

drawFloaters() {
  push();
  noStroke();
  for (let p of this.floaters) {
    const tw = sin(frameCount * 0.02 + p.phase) * 0.4 + 0.6;
    fill(190, 210, 230, p.glow * tw);
    circle(p.x, p.y, p.r * (0.8 + tw));
  }
  pop();
}

// ---------- Moon (screen-space, moves with camera X, stays in sky)
----------
drawMoonParallax(camX) {
  push();

  const r = this.moonR;
  const my = 78;     // fixed screen Y (sky)
  const baseX = 86;  // top-left-ish

  const parallax = 0.18; // tweak: 0.10 (farther) .. 0.30 (closer)

  // camera right => moon slides left a bit
  let mx = baseX - camX * parallax;

  // wrap so it cycles instead of disappearing forever
```

```
const wrapW = width + r * 4;
mx = ((mx % wrapW) + wrapW) % wrapW - r * 2;

// glow
noStroke();
fill(210, 220, 240, 16);
circle(mx, my, r * 2.35);

// body
fill(215, 225, 240, 235);
circle(mx, my, r * 2);

// sketched outline
stroke(40, 50, 70, 85);
strokeWeight(2);
noFill();

const saved = this.moonSeed;
for (let k = 0; k < 3; k++) {
  beginShape();
  for (let a = 0; a <= TWO_PI + 0.001; a += 0.22) {
    const nx = noise(saved * 0.00001 + k * 10.0, a * 0.6);
    const wob = (nx - 0.5) * 6;
    const rr = r + wob + k * 1.2;
    vertex(mx + cos(a) * rr, my + sin(a) * rr);
  }
  endShape();
}

// craters
stroke(70, 85, 110, 70);
strokeWeight(1.5);
fill(200, 210, 230, 120);
this.crater(mx - 14, my - 6, 10, 7);
this.crater(mx + 12, my + 10, 14, 10);
this.crater(mx + 6, my - 18, 8, 6);

// hatch shading
```

```
  stroke(90, 105, 130, 55);
  strokeWeight(1);
  for (let i = 0; i < 18; i++) {
    const t = i / 17;
    const x1 = mx - r + t * (r * 1.05);
    const y1 = my + r * 0.35 + t * 10;
    line(x1, y1, x1 + 14, y1 + 8);
  }

  pop();
}

crater(cx, cy, w, h) {
  push();
  translate(cx, cy);
  noFill();
  for (let k = 0; k < 2; k++) {
    beginShape();
    for (let a = 0; a <= TWO_PI + 0.001; a += 0.35) {
      const wob =
        (noise(this.moonSeed * 0.00002 + k * 20.0, a * 0.8) - 0.5) * 2.2;
      const rx = w * 0.5 + wob;
      const ry = h * 0.5 + wob * 0.7;
      vertex(cos(a) * rx, sin(a) * ry);
    }
    endShape();
  }
  pop();
}

// ---------- Cows ----------
drawCows() {
  push();
  for (let i = 0; i < this.cows.length; i++) {
    const c = this.cows[i];
    const bob = sin(frameCount * 0.015 + c.idle) * 1.2;

    push();
```

```
    translate(c.x, c.y + bob);
    scale(c.flip * c.s, c.s);

    noStroke();
    fill(20, 24, 32, 235);
    ellipse(0, -18, 56, 30);
    ellipse(34, -26, 20, 16);

    fill(24, 28, 38, 235);
    ellipse(42, -24, 10, 8);

    fill(16, 18, 24, 240);
    rect(-18, -6, 6, 18, 2);
    rect(-6, -6, 6, 18, 2);
    rect(8, -6, 6, 18, 2);
    rect(20, -6, 6, 18, 2);

    stroke(18, 20, 26, 220);
    strokeWeight(2);
    noFill();
    const wag = sin(frameCount * 0.02 + c.idle) * 4;
    beginShape();
    vertex(-28, -22);
    quadraticVertex(-40, -16 + wag, -36, -6);
    endShape();

    noStroke();
    fill(28, 34, 46, 120);
    ellipse(-6, -20, 16, 10);

    pop();
  }
  pop();
}

// ---------- Grass ----------
drawGrass() {
  push();
```

```
  strokeWeight(2);

  for (let i = 0; i < this.grass.length; i++) {
    const g = this.grass[i];
    const sway = sin(frameCount * 0.02 + g.phase) * 3;

    stroke(22 * g.tint, 28 * g.tint, 34 * g.tint, 170);

    const baseY = g.y - 6;
    const baseX = g.x;

    for (let b = 0; b < g.clump; b++) {
      const ox = (b - (g.clump - 1) / 2) * (4 * g.s);
      const hh = (10 + (b % 3) * 6) * g.s;
      const bend = sway + ox * 0.2;
      line(baseX + ox, baseY, baseX + ox + bend, baseY - hh);
    }
  }

  pop();
}

// ---------- Screen-space overlays ----------
drawBreathOverlay() {
  const breath = sin(this.breathT * 0.35) * 0.5 + 0.5;
  const alpha = 22 + breath * 26;

  noStroke();
  fill(90, 140, 170, alpha);
  rect(0, 0, width, height);
}

drawVignette() {
  push();
  noFill();
  for (let i = 0; i < 18; i++) {
    const t = i / 17;
    const a = 18 * t;
```

```
      stroke(0, 0, 0, a);
      const m = t * 70;
      rect(m, m, width - 2 * m, height - 2 * m, 28);
    }
    pop();
  }

  drawGrain() {
    push();
    blendMode(BLEND);
    tint(255, 18);
    image(grainLayer, 0, 0);
    noTint();
    pop();
  }

  // ---------- Helpers ----------
  sampleGroundY(x) {
    const n1 = noise(x * 0.0013, 100.0);
    const n2 = noise(x * 0.0035, 200.0);
    return 240 + n1 * 210 + n2 * 45;
  }
}
```

Thought for 26s

```
/*
Meditative Camera Scroll — p5.js (single-file)

BONUS added:
- Little coins that poke out of cows and trees
- Click a coin to collect it (+1 score)
- Coins are world-anchored (move with the camera), but attached visually
to cows/trees
```

```
Controls:
- Move camera: Arrow keys / WASD
- Click coins: +1 score
- Reset: R (resets world + score)
- Toggle UI: H
- Debug toggles:
 1 Far  2 Mid  3 Near  4 Floaters  5 Breath  6 Vignette  7 Grain
*/

let cam, world;
let showHUD = true;
let score = 0;

let grainLayer;

// Debug layer toggles
let SHOW_FAR = true;
let SHOW_MID = true;
let SHOW_NEAR = true;
let SHOW_FLOATERS = true;
let SHOW_BREATH = true;
let SHOW_VIGNETTE = true;
let SHOW_GRAIN = true;

function setup() {
 createCanvas(900, 520);
 pixelDensity(1);

 world = new World();
 cam = new CameraRig();

 textFont("Georgia");

 grainLayer = createGraphics(width, height);
 makeStaticGrain(grainLayer, 18);
}
```

```
function draw() {
 background(6, 8, 12);

 world.update();
 cam.update(world);

 // ---------- World space ----------
 push();
 cam.apply();

 if (SHOW_FAR) world.drawFar();
 if (SHOW_MID) world.drawMid();     // includes cows + cow coins
 if (SHOW_NEAR) world.drawNear();   // includes trees + tree coins + grass
 if (SHOW_FLOATERS) world.drawFloaters();

 pop();

 // ---------- Screen space ----------
 world.drawMoonParallax(cam.pos.x);

 if (SHOW_BREATH) world.drawBreathOverlay();
 if (SHOW_VIGNETTE) world.drawVignette();
 if (SHOW_GRAIN) world.drawGrain();

 if (showHUD) drawHUD();
}

function mousePressed() {
 // Convert screen click to world coords (camera is a translation)
 const wx = mouseX + cam.pos.x;
 const wy = mouseY + cam.pos.y;

 if (world.tryCollectCoinAt(wx, wy)) {
   score += 1;
 }
}

// -------------------- Input --------------------
```

```
function keyPressed() {
 if (key === "h" || key === "H") showHUD = !showHUD;

 if (key === "r" || key === "R") {
   score = 0;
   world = new World();
   cam = new CameraRig();
   grainLayer = createGraphics(width, height);
   makeStaticGrain(grainLayer, 18);
 }

 // Debug toggles
 if (key === "1") SHOW_FAR = !SHOW_FAR;
 if (key === "2") SHOW_MID = !SHOW_MID;
 if (key === "3") SHOW_NEAR = !SHOW_NEAR;
 if (key === "4") SHOW_FLOATERS = !SHOW_FLOATERS;
 if (key === "5") SHOW_BREATH = !SHOW_BREATH;
 if (key === "6") SHOW_VIGNETTE = !SHOW_VIGNETTE;
 if (key === "7") SHOW_GRAIN = !SHOW_GRAIN;
}

// -------------------- HUD --------------------
function drawHUD() {
 const pad = 16;
 noStroke();
 fill(0, 90);
 rect(pad, pad, 610, 112, 14);

 fill(230, 220);
 textSize(14);
 textAlign(LEFT, TOP);
 text(
   "Night farm (manual camera)\nWASD / Arrows: move camera  •  Click
coins: +1\nH: hide text  •  R: reset\n1 Far  2 Mid  3 Near  4 Floaters  5
Breath  6 Vignette  7 Grain",
   pad + 12,
   pad + 10
 );
```

```
 // Score pill
 const pillW = 120;
 const pillH = 30;
 const px = pad + 610 - pillW - 12;
 const py = pad + 12;
 fill(255, 20);
 rect(px, py, pillW, pillH, 999);
 fill(240, 230);
 textAlign(CENTER, CENTER);
 textSize(14);
 text(`Score: ${score}`, px + pillW / 2, py + pillH / 2);
}

// -------------------- Static Grain Builder --------------------
function makeStaticGrain(pg, intensity = 18) {
 pg.pixelDensity(1);
 pg.loadPixels();

 for (let y = 0; y < pg.height; y++) {
   for (let x = 0; x < pg.width; x++) {
     const idx = 4 * (y * pg.width + x);

     const n = noise(x * 0.08, y * 0.08);
     const g = (n - 0.5) * 2 * intensity;

     pg.pixels[idx + 0] = 128 + g;
     pg.pixels[idx + 1] = 128 + g;
     pg.pixels[idx + 2] = 128 + g;
     pg.pixels[idx + 3] = 35;
   }
 }

 pg.updatePixels();
}

// ==================== Camera ====================
class CameraRig {
```

```
constructor() {
  this.pos = createVector(0, 0);
  this.vel = createVector(0, 0);
}

// Manual-only camera
update(worldRef) {
  let ix = 0;
  let iy = 0;

  if (keyIsDown(LEFT_ARROW) || keyIsDown(65)) ix -= 1;  // A
  if (keyIsDown(RIGHT_ARROW) || keyIsDown(68)) ix += 1; // D
  if (keyIsDown(UP_ARROW) || keyIsDown(87)) iy -= 1;    // W
  if (keyIsDown(DOWN_ARROW) || keyIsDown(83)) iy += 1;  // S

  const mag = Math.hypot(ix, iy);
  if (mag > 0) {
    ix /= mag;
    iy /= mag;
  }

  const accel = 0.58;
  const dampingWhenIdle = 0.82;
  const dampingWhenMoving = 0.92;
  const maxV = 3.0;

  this.vel.x += ix * accel;
  this.vel.y += iy * accel;

  this.vel.x = constrain(this.vel.x, -maxV, maxV);
  this.vel.y = constrain(this.vel.y, -maxV, maxV);

  if (mag === 0) this.vel.mult(dampingWhenIdle);
  else this.vel.mult(dampingWhenMoving);

  this.pos.add(this.vel);

  const b = worldRef.softBounds;
```

```
    this.pos.x = constrain(this.pos.x, b.minX, b.maxX);
    this.pos.y = constrain(this.pos.y, b.minY, b.maxY);
  }

  apply() {
    translate(-this.pos.x, -this.pos.y);
  }
}

// ==================== World ====================
class World {
  constructor() {
    this.seed = floor(random(1e9));
    noiseSeed(this.seed);
    randomSeed(this.seed);

    this.softBounds = {
      minX: -900,
      maxX: 6800,
      minY: -650,
      maxY: 650
    };

    this.breathT = random(1000);

    // particles
    this.floaters = [];
    for (let i = 0; i < 220; i++) this.floaters.push(this.makeFloater());

    // stars
    this.stars = [];
    for (let i = 0; i < 420; i++) {
      this.stars.push({
        x: random(this.softBounds.minX, this.softBounds.maxX),
        y: random(this.softBounds.minY, this.softBounds.maxY),
        s: random(0.5, 2.2),
        a: random(70, 170)
      });
```

```
    }

    // detail
    this.cows = this.makeCows(14);
    this.grass = this.makeGrass(160);

    // trees/reeds (moved from procedural loop into an array so we can
attach coins)
    this.trees = this.makeTrees(42);

    // moon (screen-space parallax)
    this.moonR = 46;
    this.moonSeed = floor(random(1e9)); // stable scribble seed

    // coins
    this.coins = this.makeCoins();
  }

  makeFloater() {
    return {
      x: random(this.softBounds.minX, this.softBounds.maxX),
      y: random(-260, 260),
      r: random(1.2, 3.8),
      phase: random(TWO_PI),
      drift: random(0.2, 0.55),
      glow: random(40, 120)
    };
  }

  makeCows(count) {
    const arr = [];
    for (let i = 0; i < count; i++) {
      const x = random(this.softBounds.minX + 200, this.softBounds.maxX -
200);
      const y = this.sampleGroundY(x);
      const s = random(0.85, 1.5);
      arr.push({
        x,
```

```
      y,
      s,
      flip: random() < 0.5 ? -1 : 1,
      idle: random(TWO_PI),
      buddyOffset: random() < 0.35 ? random(-90, 90) : 0
    });
  }
  for (let c of arr) {
    if (c.buddyOffset !== 0) {
      c.x += c.buddyOffset;
      c.y = this.sampleGroundY(c.x);
    }
  }
  return arr;
}

makeGrass(count) {
  const arr = [];
  for (let i = 0; i < count; i++) {
    const x = random(this.softBounds.minX, this.softBounds.maxX);
    const y = this.sampleGroundY(x);
    const s = random(0.6, 1.35);
    const clump = floor(random(2, 6));
    arr.push({
      x,
      y,
      s,
      clump,
      phase: random(TWO_PI),
      tint: random(0.75, 1.05)
    });
  }
  return arr;
}

makeTrees(count) {
  const arr = [];
  for (let i = 0; i < count; i++) {
```

```
    // preserve your old spacing vibe but make it flexible across the big
world
    const x = i * 160 + 40 + sin(i * 0.9) * 20;
    const y = this.sampleGroundY(x);
    arr.push({
      x,
      y,
      phase: random(TWO_PI)
    });
  }
  // Also sprinkle a few extra trees further along the world
  for (let i = 0; i < 30; i++) {
    const x = random(this.softBounds.minX + 200, this.softBounds.maxX -
200);
    const y = this.sampleGroundY(x);
    if (random() < 0.45) {
      arr.push({ x, y, phase: random(TWO_PI) });
    }
  }
  return arr;
}

makeCoins() {
  const coins = [];

  // Coins poking out of cows (more common)
  for (let i = 0; i < this.cows.length; i++) {
    if (random() < 0.55) {
      coins.push({
        kind: "cow",
        idx: i,
        r: 10,
        collected: false
      });
    }
  }

  // Coins poking out of trees (less common)
```

```
  for (let i = 0; i < this.trees.length; i++) {
    if (random() < 0.35) {
      coins.push({
        kind: "tree",
        idx: i,
        r: 10,
        collected: false
      });
    }
  }

  return coins;
}

update() {
  this.breathT += 0.01;

  for (let p of this.floaters) {
    const w = noise(p.x * 0.0012, p.y * 0.0012, frameCount * 0.002) -
0.5;
    p.x += w * 0.5;
    p.y += sin(frameCount * 0.004 + p.phase) * 0.22 - p.drift;

    if (p.y < -520) {
      p.y = random(120, 420);
      p.x = random(this.softBounds.minX, this.softBounds.maxX);
    }
  }
}

// ---------- Coin logic ----------
getCoinPos(coin) {
  if (coin.kind === "cow") {
    const c = this.cows[coin.idx];
    if (!c) return null;

    const bob = sin(frameCount * 0.015 + c.idle) * 1.2;
```

```
    // local coin position "poking out" near the cow's shoulder/back
    const xLocal = 18;
    const yLocal = -38;

    const wx = c.x + xLocal * c.flip * c.s;
    const wy = (c.y + bob) + yLocal * c.s;
    return { x: wx, y: wy };
  }

  if (coin.kind === "tree") {
    const t = this.trees[coin.idx];
    if (!t) return null;

    const sway = sin(frameCount * 0.01 + t.phase) * 2.5;

    // poke out near the canopy
    const wx = t.x + sway * 0.6;
    const wy = t.y - 82;
    return { x: wx, y: wy };
  }

  return null;
}

tryCollectCoinAt(wx, wy) {
  // Find first coin within radius
  for (let coin of this.coins) {
    if (coin.collected) continue;
    const p = this.getCoinPos(coin);
    if (!p) continue;

    const d = dist(wx, wy, p.x, p.y);
    if (d <= coin.r + 6) {
      coin.collected = true;
      return true;
    }
  }
  return false;
```

```
}

drawCoinAt(x, y, r) {
  push();

  // outer glow
  noStroke();
  fill(255, 225, 120, 28);
  circle(x, y, r * 3.0);

  // coin body
  fill(245, 206, 90, 235);
  circle(x, y, r * 2);

  // rim
  noFill();
  stroke(120, 85, 30, 180);
  strokeWeight(2);
  circle(x, y, r * 2);

  // little "shine"
  stroke(255, 245, 200, 140);
  strokeWeight(2);
  line(x - r * 0.3, y - r * 0.6, x + r * 0.2, y - r * 0.9);

  // tiny notch (makes it feel "stuck" in the object)
  stroke(90, 60, 20, 160);
  strokeWeight(2);
  line(x - r * 0.9, y + r * 0.5, x - r * 0.2, y + r * 0.15);

  pop();
}

drawCoinsForKind(kind) {
  for (let coin of this.coins) {
    if (coin.collected) continue;
    if (coin.kind !== kind) continue;
```

```
    const p = this.getCoinPos(coin);
    if (!p) continue;
    this.drawCoinAt(p.x, p.y, coin.r);
  }
}

// ---------- Layers ----------
drawFar() {
  push();
  noStroke();

  // stars
  for (let st of this.stars) {
    fill(210, 220, 245, st.a);
    circle(st.x, st.y, st.s);
  }

  // safer aurora (segmented, capped alpha)
  for (let x = this.softBounds.minX; x < this.softBounds.maxX; x += 40) {
    const gate = noise(x * 0.0016, this.seed * 0.00002);
    if (gate < 0.42) continue;

    const n = noise(x * 0.0007, this.seed * 0.00001);
    const y = -260 + n * 90 + sin(x * 0.0016 + this.breathT * 0.25) * 10;

    let a = 6 + n * 14;
    a = constrain(a, 0, 18);

    fill(110, 160, 190, a);

    const w = 70 + n * 35;
    const h = 36 + n * 18;
    ellipse(x, y, w, h);
  }

  pop();
}
```

```
 drawMid() {
   push();

   const horizonY = 80;
   const groundBase = 260;

   // sky gradient slab
   noStroke();
   for (let y = -400; y < groundBase; y += 2) {
     const t = map(y, -400, groundBase, 0, 1);
     fill(10 + t * 18, 14 + t * 18, 22 + t * 24, 255);
     rect(this.softBounds.minX, y, this.softBounds.maxX -
this.softBounds.minX, 2);
   }

   // distant hills
   fill(12, 16, 22, 220);
   beginShape();
   vertex(this.softBounds.minX, groundBase);
   for (let x = this.softBounds.minX; x <= this.softBounds.maxX; x += 18)
{
     const n = noise(x * 0.0009, 10.0);
     const yy = horizonY + n * 140;
     vertex(x, yy);
   }
   vertex(this.softBounds.maxX, groundBase);
   endShape(CLOSE);

   // main ground
   fill(8, 10, 14, 255);
   beginShape();
   vertex(this.softBounds.minX, height + 900);
   for (let x = this.softBounds.minX; x <= this.softBounds.maxX; x += 16)
{
     const n1 = noise(x * 0.0013, 100.0);
     const n2 = noise(x * 0.0035, 200.0);
     const yy = 240 + n1 * 210 + n2 * 45;
     vertex(x, yy);
```

```
    }
    vertex(this.softBounds.maxX, height + 900);
    endShape(CLOSE);

    // water line
    const waterY = 300;
    for (let x = this.softBounds.minX; x < this.softBounds.maxX; x += 10) {
      const ripple =
        sin(x * 0.02 + this.breathT * 1.2) * 2 +
        (noise(x * 0.01, this.breathT * 0.2) - 0.5) * 3;
      const a = 12 + noise(x * 0.003, 400) * 18;
      stroke(180, 210, 230, a);
      line(x, waterY + ripple, x + 8, waterY + ripple + 0.4);
    }

    // tiny posts
    noStroke();
    for (let i = 0; i < 26; i++) {
      const px = i * 280 + 90 + sin(i * 2.1) * 35;
      const py = this.sampleGroundY(px) - 8;
      const tall = 18 + noise(i * 10.1) * 55;

      fill(18, 22, 30, 220);
      rect(px - 2, py - tall, 4, tall, 2);

      fill(200, 220, 240, 40);
      circle(px, py - tall - 6, 18);
    }

    // cows + cow coins (coins poke out of cows)
    this.drawCows();
    this.drawCoinsForKind("cow");

    pop();
  }

  drawNear() {
    push();
```

```
  // grass tufts
  this.drawGrass();

  // trees/reeds + tree coins
  this.drawTrees();
  this.drawCoinsForKind("tree");

  pop();
}

drawTrees() {
  for (let t of this.trees) {
    const sway = sin(frameCount * 0.01 + t.phase) * 2.5;

    stroke(12, 14, 18, 210);
    strokeWeight(3);
    line(t.x, t.y - 8, t.x + sway, t.y - 70);

    noStroke();
    fill(10, 12, 16, 240);
    ellipse(t.x + sway, t.y - 82, 36, 24);
    ellipse(t.x + sway + 16, t.y - 76, 22, 16);
    ellipse(t.x + sway - 14, t.y - 74, 20, 14);
  }
}

drawFloaters() {
  push();
  noStroke();
  for (let p of this.floaters) {
    const tw = sin(frameCount * 0.02 + p.phase) * 0.4 + 0.6;
    fill(190, 210, 230, p.glow * tw);
    circle(p.x, p.y, p.r * (0.8 + tw));
  }
  pop();
}
```

```
// ---------- Moon (screen-space, moves with camera X, stays in sky)
----------
drawMoonParallax(camX) {
  push();

  const r = this.moonR;
  const my = 78;     // fixed screen Y (sky)
  const baseX = 86;  // top-left-ish

  const parallax = 0.18; // tweak: 0.10 (farther) .. 0.30 (closer)

  let mx = baseX - camX * parallax;

  // wrap so it cycles instead of disappearing forever
  const wrapW = width + r * 4;
  mx = ((mx % wrapW) + wrapW) % wrapW - r * 2;

  // glow
  noStroke();
  fill(210, 220, 240, 16);
  circle(mx, my, r * 2.35);

  // body
  fill(215, 225, 240, 235);
  circle(mx, my, r * 2);

  // sketched outline
  stroke(40, 50, 70, 85);
  strokeWeight(2);
  noFill();

  const saved = this.moonSeed;
  for (let k = 0; k < 3; k++) {
    beginShape();
    for (let a = 0; a <= TWO_PI + 0.001; a += 0.22) {
      const nx = noise(saved * 0.00001 + k * 10.0, a * 0.6);
      const wob = (nx - 0.5) * 6;
      const rr = r + wob + k * 1.2;
```

```
      vertex(mx + cos(a) * rr, my + sin(a) * rr);
    }
    endShape();
  }

  // craters
  stroke(70, 85, 110, 70);
  strokeWeight(1.5);
  fill(200, 210, 230, 120);
  this.crater(mx - 14, my - 6, 10, 7);
  this.crater(mx + 12, my + 10, 14, 10);
  this.crater(mx + 6, my - 18, 8, 6);

  // hatch shading
  stroke(90, 105, 130, 55);
  strokeWeight(1);
  for (let i = 0; i < 18; i++) {
    const t = i / 17;
    const x1 = mx - r + t * (r * 1.05);
    const y1 = my + r * 0.35 + t * 10;
    line(x1, y1, x1 + 14, y1 + 8);
  }

  pop();
}

crater(cx, cy, w, h) {
  push();
  translate(cx, cy);
  noFill();
  for (let k = 0; k < 2; k++) {
    beginShape();
    for (let a = 0; a <= TWO_PI + 0.001; a += 0.35) {
      const wob =
        (noise(this.moonSeed * 0.00002 + k * 20.0, a * 0.8) - 0.5) * 2.2;
      const rx = w * 0.5 + wob;
      const ry = h * 0.5 + wob * 0.7;
      vertex(cos(a) * rx, sin(a) * ry);
```

```
    }
    endShape();
  }
  pop();
}

// ---------- Cows ----------
drawCows() {
  push();
  for (let i = 0; i < this.cows.length; i++) {
    const c = this.cows[i];
    const bob = sin(frameCount * 0.015 + c.idle) * 1.2;

    push();
    translate(c.x, c.y + bob);
    scale(c.flip * c.s, c.s);

    noStroke();
    fill(20, 24, 32, 235);
    ellipse(0, -18, 56, 30);
    ellipse(34, -26, 20, 16);

    fill(24, 28, 38, 235);
    ellipse(42, -24, 10, 8);

    fill(16, 18, 24, 240);
    rect(-18, -6, 6, 18, 2);
    rect(-6, -6, 6, 18, 2);
    rect(8, -6, 6, 18, 2);
    rect(20, -6, 6, 18, 2);

    stroke(18, 20, 26, 220);
    strokeWeight(2);
    noFill();
    const wag = sin(frameCount * 0.02 + c.idle) * 4;
    beginShape();
    vertex(-28, -22);
    quadraticVertex(-40, -16 + wag, -36, -6);
```

```
    endShape();

    noStroke();
    fill(28, 34, 46, 120);
    ellipse(-6, -20, 16, 10);

    pop();
  }
  pop();
}

// ---------- Grass ----------
drawGrass() {
  push();
  strokeWeight(2);

  for (let i = 0; i < this.grass.length; i++) {
    const g = this.grass[i];
    const sway = sin(frameCount * 0.02 + g.phase) * 3;

    stroke(22 * g.tint, 28 * g.tint, 34 * g.tint, 170);

    const baseY = g.y - 6;
    const baseX = g.x;

    for (let b = 0; b < g.clump; b++) {
      const ox = (b - (g.clump - 1) / 2) * (4 * g.s);
      const hh = (10 + (b % 3) * 6) * g.s;
      const bend = sway + ox * 0.2;
      line(baseX + ox, baseY, baseX + ox + bend, baseY - hh);
    }
  }

  pop();
}

// ---------- Screen-space overlays ----------
drawBreathOverlay() {
```

```
  const breath = sin(this.breathT * 0.35) * 0.5 + 0.5;
  const alpha = 22 + breath * 26;

  noStroke();
  fill(90, 140, 170, alpha);
  rect(0, 0, width, height);
}

drawVignette() {
  push();
  noFill();
  for (let i = 0; i < 18; i++) {
    const t = i / 17;
    const a = 18 * t;
    stroke(0, 0, 0, a);
    const m = t * 70;
    rect(m, m, width - 2 * m, height - 2 * m, 28);
  }
  pop();
}

drawGrain() {
  push();
  blendMode(BLEND);
  tint(255, 18);
  image(grainLayer, 0, 0);
  noTint();
  pop();
}

// ---------- Helpers ----------
sampleGroundY(x) {
  const n1 = noise(x * 0.0013, 100.0);
  const n2 = noise(x * 0.0035, 200.0);
  return 240 + n1 * 210 + n2 * 45;
}
}
```

```
/*
Meditative Camera Scroll — p5.js (single-file)

Update:
- Removed ALL layer toggles (no more 1–7 debug enable/disable).
- Kept only: H (hide HUD) and R (reset).
- Everything always renders: far/mid/near/floaters/breath/vignette/grain.

BONUS kept:
- Coins poke out of cows and trees
- Click coins to collect (+1 score)

Controls:
- Move camera: Arrow keys / WASD
- Click coins: +1 score
- Reset: R (resets world + score)
- Toggle UI: H
*/

let cam, world;
let showHUD = true;
let score = 0;

let grainLayer;

function setup() {
 createCanvas(900, 520);
 pixelDensity(1);

 world = new World();
 cam = new CameraRig();

 textFont("Georgia");
```

```
 grainLayer = createGraphics(width, height);
 makeStaticGrain(grainLayer, 18);
}

function draw() {
 background(6, 8, 12);

 world.update();
 cam.update(world);

 // ---------- World space ----------
 push();
 cam.apply();

 world.drawFar();
 world.drawMid();     // cows + cow coins
 world.drawNear();    // trees + tree coins + grass
 world.drawFloaters();

 pop();

 // ---------- Screen space ----------
 world.drawMoonParallax(cam.pos.x);

 world.drawBreathOverlay();
 world.drawVignette();
 world.drawGrain();

 if (showHUD) drawHUD();
}

function mousePressed() {
 // Convert screen click to world coords (camera is a translation)
 const wx = mouseX + cam.pos.x;
 const wy = mouseY + cam.pos.y;

 if (world.tryCollectCoinAt(wx, wy)) {
```

```
    score += 1;
  }
}

// -------------------- Input --------------------
function keyPressed() {
  if (key === "h" || key === "H") showHUD = !showHUD;

  if (key === "r" || key === "R") {
    score = 0;
    world = new World();
    cam = new CameraRig();
    grainLayer = createGraphics(width, height);
    makeStaticGrain(grainLayer, 18);
  }
}

// -------------------- HUD --------------------
function drawHUD() {
  const pad = 16;
  noStroke();
  fill(0, 90);
  rect(pad, pad, 610, 112, 14);

  fill(230, 220);
  textSize(14);
  textAlign(LEFT, TOP);
  text(
    "Night farm (manual camera)\nWASD / Arrows: move camera  •  Click
coins: +1\nH: hide text  •  R: reset",
    pad + 12,
    pad + 10
  );

  // Score pill
  const pillW = 120;
  const pillH = 30;
  const px = pad + 610 - pillW - 12;
```

```
 const py = pad + 12;
 fill(255, 20);
 rect(px, py, pillW, pillH, 999);
 fill(240, 230);
 textAlign(CENTER, CENTER);
 textSize(14);
 text(`Score: ${score}`, px + pillW / 2, py + pillH / 2);
}

// -------------------- Static Grain Builder --------------------
function makeStaticGrain(pg, intensity = 18) {
 pg.pixelDensity(1);
 pg.loadPixels();

 for (let y = 0; y < pg.height; y++) {
   for (let x = 0; x < pg.width; x++) {
     const idx = 4 * (y * pg.width + x);

     const n = noise(x * 0.08, y * 0.08);
     const g = (n - 0.5) * 2 * intensity;

     pg.pixels[idx + 0] = 128 + g;
     pg.pixels[idx + 1] = 128 + g;
     pg.pixels[idx + 2] = 128 + g;
     pg.pixels[idx + 3] = 35;
   }
 }

 pg.updatePixels();
}

// ==================== Camera ====================
class CameraRig {
 constructor() {
   this.pos = createVector(0, 0);
   this.vel = createVector(0, 0);
 }
```

```
// Manual-only camera
update(worldRef) {
  let ix = 0;
  let iy = 0;

  if (keyIsDown(LEFT_ARROW) || keyIsDown(65)) ix -= 1;  // A
  if (keyIsDown(RIGHT_ARROW) || keyIsDown(68)) ix += 1; // D
  if (keyIsDown(UP_ARROW) || keyIsDown(87)) iy -= 1;    // W
  if (keyIsDown(DOWN_ARROW) || keyIsDown(83)) iy += 1;  // S

  const mag = Math.hypot(ix, iy);
  if (mag > 0) {
    ix /= mag;
    iy /= mag;
  }

  const accel = 0.58;
  const dampingWhenIdle = 0.82;
  const dampingWhenMoving = 0.92;
  const maxV = 3.0;

  this.vel.x += ix * accel;
  this.vel.y += iy * accel;

  this.vel.x = constrain(this.vel.x, -maxV, maxV);
  this.vel.y = constrain(this.vel.y, -maxV, maxV);

  if (mag === 0) this.vel.mult(dampingWhenIdle);
  else this.vel.mult(dampingWhenMoving);

  this.pos.add(this.vel);

  const b = worldRef.softBounds;
  this.pos.x = constrain(this.pos.x, b.minX, b.maxX);
  this.pos.y = constrain(this.pos.y, b.minY, b.maxY);
}

apply() {
```

```
    translate(-this.pos.x, -this.pos.y);
  }
}

// ==================== World ====================
class World {
  constructor() {
    this.seed = floor(random(1e9));
    noiseSeed(this.seed);
    randomSeed(this.seed);

    this.softBounds = {
      minX: -900,
      maxX: 6800,
      minY: -650,
      maxY: 650
    };

    this.breathT = random(1000);

    // particles
    this.floaters = [];
    for (let i = 0; i < 220; i++) this.floaters.push(this.makeFloater());

    // stars
    this.stars = [];
    for (let i = 0; i < 420; i++) {
      this.stars.push({
        x: random(this.softBounds.minX, this.softBounds.maxX),
        y: random(this.softBounds.minY, this.softBounds.maxY),
        s: random(0.5, 2.2),
        a: random(70, 170)
      });
    }

    // detail
    this.cows = this.makeCows(14);
    this.grass = this.makeGrass(160);
```

```
  // trees/reeds (array so we can attach coins)
  this.trees = this.makeTrees(42);

  // moon (screen-space parallax)
  this.moonR = 46;
  this.moonSeed = floor(random(1e9)); // stable scribble seed

  // coins
  this.coins = this.makeCoins();
}

makeFloater() {
  return {
    x: random(this.softBounds.minX, this.softBounds.maxX),
    y: random(-260, 260),
    r: random(1.2, 3.8),
    phase: random(TWO_PI),
    drift: random(0.2, 0.55),
    glow: random(40, 120)
  };
}

makeCows(count) {
  const arr = [];
  for (let i = 0; i < count; i++) {
    const x = random(this.softBounds.minX + 200, this.softBounds.maxX -
200);
    const y = this.sampleGroundY(x);
    const s = random(0.85, 1.5);
    arr.push({
      x,
      y,
      s,
      flip: random() < 0.5 ? -1 : 1,
      idle: random(TWO_PI),
      buddyOffset: random() < 0.35 ? random(-90, 90) : 0
    });
```

```
  }
  for (let c of arr) {
    if (c.buddyOffset !== 0) {
      c.x += c.buddyOffset;
      c.y = this.sampleGroundY(c.x);
    }
  }
  return arr;
}

makeGrass(count) {
  const arr = [];
  for (let i = 0; i < count; i++) {
    const x = random(this.softBounds.minX, this.softBounds.maxX);
    const y = this.sampleGroundY(x);
    const s = random(0.6, 1.35);
    const clump = floor(random(2, 6));
    arr.push({
      x,
      y,
      s,
      clump,
      phase: random(TWO_PI),
      tint: random(0.75, 1.05)
    });
  }
  return arr;
}

makeTrees(count) {
  const arr = [];
  for (let i = 0; i < count; i++) {
    const x = i * 160 + 40 + sin(i * 0.9) * 20;
    const y = this.sampleGroundY(x);
    arr.push({
      x,
      y,
      phase: random(TWO_PI)
```

```
    });
  }
  for (let i = 0; i < 30; i++) {
    const x = random(this.softBounds.minX + 200, this.softBounds.maxX -
200);
    const y = this.sampleGroundY(x);
    if (random() < 0.45) arr.push({ x, y, phase: random(TWO_PI) });
  }
  return arr;
}

makeCoins() {
  const coins = [];

  // Coins poking out of cows
  for (let i = 0; i < this.cows.length; i++) {
    if (random() < 0.55) {
      coins.push({ kind: "cow", idx: i, r: 10, collected: false });
    }
  }

  // Coins poking out of trees
  for (let i = 0; i < this.trees.length; i++) {
    if (random() < 0.35) {
      coins.push({ kind: "tree", idx: i, r: 10, collected: false });
    }
  }

  return coins;
}

update() {
  this.breathT += 0.01;

  for (let p of this.floaters) {
    const w = noise(p.x * 0.0012, p.y * 0.0012, frameCount * 0.002) -
0.5;
    p.x += w * 0.5;
```

```
    p.y += sin(frameCount * 0.004 + p.phase) * 0.22 - p.drift;

    if (p.y < -520) {
      p.y = random(120, 420);
      p.x = random(this.softBounds.minX, this.softBounds.maxX);
    }
  }
}

// ---------- Coin logic ----------
getCoinPos(coin) {
  if (coin.kind === "cow") {
    const c = this.cows[coin.idx];
    if (!c) return null;

    const bob = sin(frameCount * 0.015 + c.idle) * 1.2;

    const xLocal = 18;   // shoulder/back
    const yLocal = -38;  // poking up

    const wx = c.x + xLocal * c.flip * c.s;
    const wy = (c.y + bob) + yLocal * c.s;
    return { x: wx, y: wy };
  }

  if (coin.kind === "tree") {
    const t = this.trees[coin.idx];
    if (!t) return null;

    const sway = sin(frameCount * 0.01 + t.phase) * 2.5;

    const wx = t.x + sway * 0.6;
    const wy = t.y - 82;
    return { x: wx, y: wy };
  }

  return null;
}
```

```
tryCollectCoinAt(wx, wy) {
  for (let coin of this.coins) {
    if (coin.collected) continue;
    const p = this.getCoinPos(coin);
    if (!p) continue;

    const d = dist(wx, wy, p.x, p.y);
    if (d <= coin.r + 6) {
      coin.collected = true;
      return true;
    }
  }
  return false;
}

drawCoinAt(x, y, r) {
  push();

  noStroke();
  fill(255, 225, 120, 28);
  circle(x, y, r * 3.0);

  fill(245, 206, 90, 235);
  circle(x, y, r * 2);

  noFill();
  stroke(120, 85, 30, 180);
  strokeWeight(2);
  circle(x, y, r * 2);

  stroke(255, 245, 200, 140);
  strokeWeight(2);
  line(x - r * 0.3, y - r * 0.6, x + r * 0.2, y - r * 0.9);

  stroke(90, 60, 20, 160);
  strokeWeight(2);
  line(x - r * 0.9, y + r * 0.5, x - r * 0.2, y + r * 0.15);
```

```
  pop();
}

drawCoinsForKind(kind) {
  for (let coin of this.coins) {
    if (coin.collected) continue;
    if (coin.kind !== kind) continue;

    const p = this.getCoinPos(coin);
    if (!p) continue;
    this.drawCoinAt(p.x, p.y, coin.r);
  }
}

// ---------- Layers ----------
drawFar() {
  push();
  noStroke();

  for (let st of this.stars) {
    fill(210, 220, 245, st.a);
    circle(st.x, st.y, st.s);
  }

  // safer aurora
  for (let x = this.softBounds.minX; x < this.softBounds.maxX; x += 40) {
    const gate = noise(x * 0.0016, this.seed * 0.00002);
    if (gate < 0.42) continue;

    const n = noise(x * 0.0007, this.seed * 0.00001);
    const y = -260 + n * 90 + sin(x * 0.0016 + this.breathT * 0.25) * 10;

    let a = 6 + n * 14;
    a = constrain(a, 0, 18);

    fill(110, 160, 190, a);
```

```
      const w = 70 + n * 35;
      const h = 36 + n * 18;
      ellipse(x, y, w, h);
    }

    pop();
  }

  drawMid() {
    push();

    const horizonY = 80;
    const groundBase = 260;

    noStroke();
    for (let y = -400; y < groundBase; y += 2) {
      const t = map(y, -400, groundBase, 0, 1);
      fill(10 + t * 18, 14 + t * 18, 22 + t * 24, 255);
      rect(this.softBounds.minX, y, this.softBounds.maxX -
this.softBounds.minX, 2);
    }

    fill(12, 16, 22, 220);
    beginShape();
    vertex(this.softBounds.minX, groundBase);
    for (let x = this.softBounds.minX; x <= this.softBounds.maxX; x += 18)
{
      const n = noise(x * 0.0009, 10.0);
      const yy = horizonY + n * 140;
      vertex(x, yy);
    }
    vertex(this.softBounds.maxX, groundBase);
    endShape(CLOSE);

    fill(8, 10, 14, 255);
    beginShape();
    vertex(this.softBounds.minX, height + 900);
```

```
  for (let x = this.softBounds.minX; x <= this.softBounds.maxX; x += 16)
{
    const n1 = noise(x * 0.0013, 100.0);
    const n2 = noise(x * 0.0035, 200.0);
    const yy = 240 + n1 * 210 + n2 * 45;
    vertex(x, yy);
  }
  vertex(this.softBounds.maxX, height + 900);
  endShape(CLOSE);

  const waterY = 300;
  for (let x = this.softBounds.minX; x < this.softBounds.maxX; x += 10) {
    const ripple =
      sin(x * 0.02 + this.breathT * 1.2) * 2 +
      (noise(x * 0.01, this.breathT * 0.2) - 0.5) * 3;
    const a = 12 + noise(x * 0.003, 400) * 18;
    stroke(180, 210, 230, a);
    line(x, waterY + ripple, x + 8, waterY + ripple + 0.4);
  }

  noStroke();
  for (let i = 0; i < 26; i++) {
    const px = i * 280 + 90 + sin(i * 2.1) * 35;
    const py = this.sampleGroundY(px) - 8;
    const tall = 18 + noise(i * 10.1) * 55;

    fill(18, 22, 30, 220);
    rect(px - 2, py - tall, 4, tall, 2);

    fill(200, 220, 240, 40);
    circle(px, py - tall - 6, 18);
  }

  this.drawCows();
  this.drawCoinsForKind("cow");

  pop();
}
```

```
drawNear() {
  push();

  this.drawGrass();
  this.drawTrees();
  this.drawCoinsForKind("tree");

  pop();
}

drawTrees() {
  for (let t of this.trees) {
    const sway = sin(frameCount * 0.01 + t.phase) * 2.5;

    stroke(12, 14, 18, 210);
    strokeWeight(3);
    line(t.x, t.y - 8, t.x + sway, t.y - 70);

    noStroke();
    fill(10, 12, 16, 240);
    ellipse(t.x + sway, t.y - 82, 36, 24);
    ellipse(t.x + sway + 16, t.y - 76, 22, 16);
    ellipse(t.x + sway - 14, t.y - 74, 20, 14);
  }
}

// ---------- Moon ----------
drawMoonParallax(camX) {
  push();

  const r = this.moonR;
  const my = 78;
  const baseX = 86;
  const parallax = 0.18;

  let mx = baseX - camX * parallax;
```

```
const wrapW = width + r * 4;
mx = ((mx % wrapW) + wrapW) % wrapW - r * 2;

noStroke();
fill(210, 220, 240, 16);
circle(mx, my, r * 2.35);

fill(215, 225, 240, 235);
circle(mx, my, r * 2);

stroke(40, 50, 70, 85);
strokeWeight(2);
noFill();

const saved = this.moonSeed;
for (let k = 0; k < 3; k++) {
  beginShape();
  for (let a = 0; a <= TWO_PI + 0.001; a += 0.22) {
    const nx = noise(saved * 0.00001 + k * 10.0, a * 0.6);
    const wob = (nx - 0.5) * 6;
    const rr = r + wob + k * 1.2;
    vertex(mx + cos(a) * rr, my + sin(a) * rr);
  }
  endShape();
}

stroke(70, 85, 110, 70);
strokeWeight(1.5);
fill(200, 210, 230, 120);
this.crater(mx - 14, my - 6, 10, 7);
this.crater(mx + 12, my + 10, 14, 10);
this.crater(mx + 6, my - 18, 8, 6);

stroke(90, 105, 130, 55);
strokeWeight(1);
for (let i = 0; i < 18; i++) {
  const t = i / 17;
  const x1 = mx - r + t * (r * 1.05);
```

```
    const y1 = my + r * 0.35 + t * 10;
    line(x1, y1, x1 + 14, y1 + 8);
  }

  pop();
}

crater(cx, cy, w, h) {
  push();
  translate(cx, cy);
  noFill();
  for (let k = 0; k < 2; k++) {
    beginShape();
    for (let a = 0; a <= TWO_PI + 0.001; a += 0.35) {
      const wob =
        (noise(this.moonSeed * 0.00002 + k * 20.0, a * 0.8) - 0.5) * 2.2;
      const rx = w * 0.5 + wob;
      const ry = h * 0.5 + wob * 0.7;
      vertex(cos(a) * rx, sin(a) * ry);
    }
    endShape();
  }
  pop();
}

// ---------- Cows ----------
drawCows() {
  push();
  for (let i = 0; i < this.cows.length; i++) {
    const c = this.cows[i];
    const bob = sin(frameCount * 0.015 + c.idle) * 1.2;

    push();
    translate(c.x, c.y + bob);
    scale(c.flip * c.s, c.s);

    noStroke();
    fill(20, 24, 32, 235);
```

```
    ellipse(0, -18, 56, 30);
    ellipse(34, -26, 20, 16);

    fill(24, 28, 38, 235);
    ellipse(42, -24, 10, 8);

    fill(16, 18, 24, 240);
    rect(-18, -6, 6, 18, 2);
    rect(-6, -6, 6, 18, 2);
    rect(8, -6, 6, 18, 2);
    rect(20, -6, 6, 18, 2);

    stroke(18, 20, 26, 220);
    strokeWeight(2);
    noFill();
    const wag = sin(frameCount * 0.02 + c.idle) * 4;
    beginShape();
    vertex(-28, -22);
    quadraticVertex(-40, -16 + wag, -36, -6);
    endShape();

    noStroke();
    fill(28, 34, 46, 120);
    ellipse(-6, -20, 16, 10);

    pop();
  }
  pop();
}

// ---------- Grass ----------
drawGrass() {
  push();
  strokeWeight(2);

  for (let i = 0; i < this.grass.length; i++) {
    const g = this.grass[i];
    const sway = sin(frameCount * 0.02 + g.phase) * 3;
```

```
    stroke(22 * g.tint, 28 * g.tint, 34 * g.tint, 170);

    const baseY = g.y - 6;
    const baseX = g.x;

    for (let b = 0; b < g.clump; b++) {
      const ox = (b - (g.clump - 1) / 2) * (4 * g.s);
      const hh = (10 + (b % 3) * 6) * g.s;
      const bend = sway + ox * 0.2;
      line(baseX + ox, baseY, baseX + ox + bend, baseY - hh);
    }
  }

  pop();
}

drawFloaters() {
  push();
  noStroke();
  for (let p of this.floaters) {
    const tw = sin(frameCount * 0.02 + p.phase) * 0.4 + 0.6;
    fill(190, 210, 230, p.glow * tw);
    circle(p.x, p.y, p.r * (0.8 + tw));
  }
  pop();
}

// ---------- Screen overlays ----------
drawBreathOverlay() {
  const breath = sin(this.breathT * 0.35) * 0.5 + 0.5;
  const alpha = 22 + breath * 26;

  noStroke();
  fill(90, 140, 170, alpha);
  rect(0, 0, width, height);
}
```

```
drawVignette() {
  push();
  noFill();
  for (let i = 0; i < 18; i++) {
    const t = i / 17;
    const a = 18 * t;
    stroke(0, 0, 0, a);
    const m = t * 70;
    rect(m, m, width - 2 * m, height - 2 * m, 28);
  }
  pop();
}

drawGrain() {
  push();
  blendMode(BLEND);
  tint(255, 18);
  image(grainLayer, 0, 0);
  noTint();
  pop();
}

// ---------- Helpers ----------
sampleGroundY(x) {
  const n1 = noise(x * 0.0013, 100.0);
  const n2 = noise(x * 0.0035, 200.0);
  return 240 + n1 * 210 + n2 * 45;
}
}
```

```
/*
Meditative Camera Scroll — p5.js (single-file)

Update:
```

```
- Added FAR farmhouses + silos on the horizon (mid/far distance).
 They are subtle silhouettes with tiny window glows.
- Kept: coins, score, moon parallax, cows, grass, trees, overlays.
- Removed: any layer toggle ability (only H and R).

Controls:
- Move camera: Arrow keys / WASD
- Click coins: +1 score
- Reset: R
- Toggle UI: H
*/

let cam, world;
let showHUD = true;
let score = 0;

let grainLayer;

function setup() {
 createCanvas(900, 520);
 pixelDensity(1);

 world = new World();
 cam = new CameraRig();

 textFont("Georgia");

 grainLayer = createGraphics(width, height);
 makeStaticGrain(grainLayer, 18);
}

function draw() {
 background(6, 8, 12);

 world.update();
 cam.update(world);

 // ---------- World space ----------
```

```
 push();
 cam.apply();

 world.drawFar();
 world.drawMid();      // includes horizon buildings + cows + cow coins
 world.drawNear();     // trees + tree coins + grass
 world.drawFloaters();

 pop();

 // ---------- Screen space ----------
 world.drawMoonParallax(cam.pos.x);

 world.drawBreathOverlay();
 world.drawVignette();
 world.drawGrain();

 if (showHUD) drawHUD();
}

function mousePressed() {
 const wx = mouseX + cam.pos.x;
 const wy = mouseY + cam.pos.y;

 if (world.tryCollectCoinAt(wx, wy)) {
   score += 1;
 }
}

// -------------------- Input --------------------
function keyPressed() {
 if (key === "h" || key === "H") showHUD = !showHUD;

 if (key === "r" || key === "R") {
   score = 0;
   world = new World();
   cam = new CameraRig();
   grainLayer = createGraphics(width, height);
```

```
    makeStaticGrain(grainLayer, 18);
  }
}

// -------------------- HUD --------------------
function drawHUD() {
  const pad = 16;
  noStroke();
  fill(0, 90);
  rect(pad, pad, 610, 112, 14);

  fill(230, 220);
  textSize(14);
  textAlign(LEFT, TOP);
  text(
    "Night farm (manual camera)\nWASD / Arrows: move camera  •  Click
coins: +1\nH: hide text  •  R: reset",
    pad + 12,
    pad + 10
  );

  const pillW = 120;
  const pillH = 30;
  const px = pad + 610 - pillW - 12;
  const py = pad + 12;
  fill(255, 20);
  rect(px, py, pillW, pillH, 999);
  fill(240, 230);
  textAlign(CENTER, CENTER);
  textSize(14);
  text(`Score: ${score}`, px + pillW / 2, py + pillH / 2);
}

// -------------------- Static Grain Builder --------------------
function makeStaticGrain(pg, intensity = 18) {
  pg.pixelDensity(1);
  pg.loadPixels();
```

```
 for (let y = 0; y < pg.height; y++) {
   for (let x = 0; x < pg.width; x++) {
     const idx = 4 * (y * pg.width + x);

     const n = noise(x * 0.08, y * 0.08);
     const g = (n - 0.5) * 2 * intensity;

     pg.pixels[idx + 0] = 128 + g;
     pg.pixels[idx + 1] = 128 + g;
     pg.pixels[idx + 2] = 128 + g;
     pg.pixels[idx + 3] = 35;
   }
 }

 pg.updatePixels();
}

// ==================== Camera ====================
class CameraRig {
 constructor() {
   this.pos = createVector(0, 0);
   this.vel = createVector(0, 0);
 }

 update(worldRef) {
   let ix = 0;
   let iy = 0;

   if (keyIsDown(LEFT_ARROW) || keyIsDown(65)) ix -= 1;  // A
   if (keyIsDown(RIGHT_ARROW) || keyIsDown(68)) ix += 1; // D
   if (keyIsDown(UP_ARROW) || keyIsDown(87)) iy -= 1;    // W
   if (keyIsDown(DOWN_ARROW) || keyIsDown(83)) iy += 1;  // S

   const mag = Math.hypot(ix, iy);
   if (mag > 0) {
     ix /= mag;
     iy /= mag;
   }
```

```
    const accel = 0.58;
    const dampingWhenIdle = 0.82;
    const dampingWhenMoving = 0.92;
    const maxV = 3.0;

    this.vel.x += ix * accel;
    this.vel.y += iy * accel;

    this.vel.x = constrain(this.vel.x, -maxV, maxV);
    this.vel.y = constrain(this.vel.y, -maxV, maxV);

    if (mag === 0) this.vel.mult(dampingWhenIdle);
    else this.vel.mult(dampingWhenMoving);

    this.pos.add(this.vel);

    const b = worldRef.softBounds;
    this.pos.x = constrain(this.pos.x, b.minX, b.maxX);
    this.pos.y = constrain(this.pos.y, b.minY, b.maxY);
  }

  apply() {
    translate(-this.pos.x, -this.pos.y);
  }
}

// ==================== World ====================
class World {
  constructor() {
    this.seed = floor(random(1e9));
    noiseSeed(this.seed);
    randomSeed(this.seed);

    this.softBounds = {
      minX: -900,
      maxX: 6800,
      minY: -650,
```

```
    maxY: 650
  };

  this.breathT = random(1000);

  this.floaters = [];
  for (let i = 0; i < 220; i++) this.floaters.push(this.makeFloater());

  this.stars = [];
  for (let i = 0; i < 420; i++) {
    this.stars.push({
      x: random(this.softBounds.minX, this.softBounds.maxX),
      y: random(this.softBounds.minY, this.softBounds.maxY),
      s: random(0.5, 2.2),
      a: random(70, 170)
    });
  }

  this.cows = this.makeCows(14);
  this.grass = this.makeGrass(160);

  this.trees = this.makeTrees(42);

  // NEW: distant buildings (farmhouses + silos)
  this.buildings = this.makeBuildings(18);

  this.moonR = 46;
  this.moonSeed = floor(random(1e9));

  this.coins = this.makeCoins();
}

makeFloater() {
  return {
    x: random(this.softBounds.minX, this.softBounds.maxX),
    y: random(-260, 260),
    r: random(1.2, 3.8),
    phase: random(TWO_PI),
```

```
      drift: random(0.2, 0.55),
      glow: random(40, 120)
    };
  }

  makeCows(count) {
    const arr = [];
    for (let i = 0; i < count; i++) {
      const x = random(this.softBounds.minX + 200, this.softBounds.maxX -
200);
      const y = this.sampleGroundY(x);
      const s = random(0.85, 1.5);
      arr.push({
        x,
        y,
        s,
        flip: random() < 0.5 ? -1 : 1,
        idle: random(TWO_PI),
        buddyOffset: random() < 0.35 ? random(-90, 90) : 0
      });
    }
    for (let c of arr) {
      if (c.buddyOffset !== 0) {
        c.x += c.buddyOffset;
        c.y = this.sampleGroundY(c.x);
      }
    }
    return arr;
  }

  makeGrass(count) {
    const arr = [];
    for (let i = 0; i < count; i++) {
      const x = random(this.softBounds.minX, this.softBounds.maxX);
      const y = this.sampleGroundY(x);
      const s = random(0.6, 1.35);
      const clump = floor(random(2, 6));
      arr.push({
```

```
      x,
      y,
      s,
      clump,
      phase: random(TWO_PI),
      tint: random(0.75, 1.05)
    });
  }
  return arr;
}

makeTrees(count) {
  const arr = [];
  for (let i = 0; i < count; i++) {
    const x = i * 160 + 40 + sin(i * 0.9) * 20;
    const y = this.sampleGroundY(x);
    arr.push({ x, y, phase: random(TWO_PI) });
  }
  for (let i = 0; i < 30; i++) {
    const x = random(this.softBounds.minX + 200, this.softBounds.maxX -
200);
    const y = this.sampleGroundY(x);
    if (random() < 0.45) arr.push({ x, y, phase: random(TWO_PI) });
  }
  return arr;
}

// NEW: farmhouses + silos "far away"
makeBuildings(count) {
  const arr = [];
  for (let i = 0; i < count; i++) {
    const x = random(this.softBounds.minX + 120, this.softBounds.maxX -
120);
    // Put them near the horizon band (not on the main ground)
    const baseY = 150 + noise(x * 0.0009, 33.3) * 60;

    const type = random() < 0.55 ? "house" : "silo";
    const s = random(0.65, 1.25);
```

```
    arr.push({
      x,
      y: baseY,
      type,
      s,
      // tiny "window warmth"
      glow: random(0.25, 1.0),
      // slight variety in roof/silo height
      v: random()
    });
  }
  return arr;
}

makeCoins() {
  const coins = [];

  for (let i = 0; i < this.cows.length; i++) {
    if (random() < 0.55) coins.push({ kind: "cow", idx: i, r: 10,
collected: false });
  }

  for (let i = 0; i < this.trees.length; i++) {
    if (random() < 0.35) coins.push({ kind: "tree", idx: i, r: 10,
collected: false });
  }

  return coins;
}

update() {
  this.breathT += 0.01;

  for (let p of this.floaters) {
    const w = noise(p.x * 0.0012, p.y * 0.0012, frameCount * 0.002) -
0.5;
    p.x += w * 0.5;
```

```
    p.y += sin(frameCount * 0.004 + p.phase) * 0.22 - p.drift;

    if (p.y < -520) {
      p.y = random(120, 420);
      p.x = random(this.softBounds.minX, this.softBounds.maxX);
    }
  }
}

// ---------- Coin logic ----------
getCoinPos(coin) {
  if (coin.kind === "cow") {
    const c = this.cows[coin.idx];
    if (!c) return null;

    const bob = sin(frameCount * 0.015 + c.idle) * 1.2;
    const xLocal = 18;
    const yLocal = -38;

    return {
      x: c.x + xLocal * c.flip * c.s,
      y: (c.y + bob) + yLocal * c.s
    };
  }

  if (coin.kind === "tree") {
    const t = this.trees[coin.idx];
    if (!t) return null;

    const sway = sin(frameCount * 0.01 + t.phase) * 2.5;
    return { x: t.x + sway * 0.6, y: t.y - 82 };
  }

  return null;
}

tryCollectCoinAt(wx, wy) {
  for (let coin of this.coins) {
```

```
    if (coin.collected) continue;
    const p = this.getCoinPos(coin);
    if (!p) continue;

    if (dist(wx, wy, p.x, p.y) <= coin.r + 6) {
      coin.collected = true;
      return true;
    }
  }
  return false;
}

drawCoinAt(x, y, r) {
  push();

  noStroke();
  fill(255, 225, 120, 28);
  circle(x, y, r * 3.0);

  fill(245, 206, 90, 235);
  circle(x, y, r * 2);

  noFill();
  stroke(120, 85, 30, 180);
  strokeWeight(2);
  circle(x, y, r * 2);

  stroke(255, 245, 200, 140);
  strokeWeight(2);
  line(x - r * 0.3, y - r * 0.6, x + r * 0.2, y - r * 0.9);

  stroke(90, 60, 20, 160);
  strokeWeight(2);
  line(x - r * 0.9, y + r * 0.5, x - r * 0.2, y + r * 0.15);

  pop();
}
```

```
drawCoinsForKind(kind) {
  for (let coin of this.coins) {
    if (coin.collected) continue;
    if (coin.kind !== kind) continue;
    const p = this.getCoinPos(coin);
    if (!p) continue;
    this.drawCoinAt(p.x, p.y, coin.r);
  }
}

// ---------- Layers ----------
drawFar() {
  push();
  noStroke();

  for (let st of this.stars) {
    fill(210, 220, 245, st.a);
    circle(st.x, st.y, st.s);
  }

  for (let x = this.softBounds.minX; x < this.softBounds.maxX; x += 40) {
    const gate = noise(x * 0.0016, this.seed * 0.00002);
    if (gate < 0.42) continue;

    const n = noise(x * 0.0007, this.seed * 0.00001);
    const y = -260 + n * 90 + sin(x * 0.0016 + this.breathT * 0.25) * 10;

    let a = 6 + n * 14;
    a = constrain(a, 0, 18);

    fill(110, 160, 190, a);

    const w = 70 + n * 35;
    const h = 36 + n * 18;
    ellipse(x, y, w, h);
  }

  pop();
```

```
 }

 drawMid() {
   push();

   const horizonY = 80;
   const groundBase = 260;

   // sky gradient
   noStroke();
   for (let y = -400; y < groundBase; y += 2) {
     const t = map(y, -400, groundBase, 0, 1);
     fill(10 + t * 18, 14 + t * 18, 22 + t * 24, 255);
     rect(this.softBounds.minX, y, this.softBounds.maxX -
this.softBounds.minX, 2);
   }

   // distant hills
   fill(12, 16, 22, 220);
   beginShape();
   vertex(this.softBounds.minX, groundBase);
   for (let x = this.softBounds.minX; x <= this.softBounds.maxX; x += 18)
{
     const n = noise(x * 0.0009, 10.0);
     const yy = horizonY + n * 140;
     vertex(x, yy);
   }
   vertex(this.softBounds.maxX, groundBase);
   endShape(CLOSE);

   // NEW: distant farm buildings on the horizon band
   this.drawBuildings();

   // main ground
   fill(8, 10, 14, 255);
   beginShape();
   vertex(this.softBounds.minX, height + 900);
```

```
  for (let x = this.softBounds.minX; x <= this.softBounds.maxX; x += 16)
{
    const n1 = noise(x * 0.0013, 100.0);
    const n2 = noise(x * 0.0035, 200.0);
    const yy = 240 + n1 * 210 + n2 * 45;
    vertex(x, yy);
  }
  vertex(this.softBounds.maxX, height + 900);
  endShape(CLOSE);

  // water line
  const waterY = 300;
  for (let x = this.softBounds.minX; x < this.softBounds.maxX; x += 10) {
    const ripple =
      sin(x * 0.02 + this.breathT * 1.2) * 2 +
      (noise(x * 0.01, this.breathT * 0.2) - 0.5) * 3;
    const a = 12 + noise(x * 0.003, 400) * 18;
    stroke(180, 210, 230, a);
    line(x, waterY + ripple, x + 8, waterY + ripple + 0.4);
  }

  // tiny posts
  noStroke();
  for (let i = 0; i < 26; i++) {
    const px = i * 280 + 90 + sin(i * 2.1) * 35;
    const py = this.sampleGroundY(px) - 8;
    const tall = 18 + noise(i * 10.1) * 55;

    fill(18, 22, 30, 220);
    rect(px - 2, py - tall, 4, tall, 2);

    fill(200, 220, 240, 40);
    circle(px, py - tall - 6, 18);
  }

  // cows + coins
  this.drawCows();
  this.drawCoinsForKind("cow");
```

```
  pop();
}

// NEW: building renderer (simple silhouettes + faint windows)
drawBuildings() {
  push();

  for (let b of this.buildings) {
    const s = b.s;

    if (b.type === "house") {
      // silhouette
      noStroke();
      fill(10, 12, 16, 240);
      const w = 52 * s;
      const h = 26 * s;
      const x = b.x;
      const y = b.y;

      rect(x - w / 2, y - h, w, h, 3);

      // roof
      fill(8, 10, 14, 245);
      beginShape();
      vertex(x - w / 2, y - h);
      vertex(x, y - h - (18 + b.v * 10) * s);
      vertex(x + w / 2, y - h);
      endShape(CLOSE);

      // chimney
      fill(8, 10, 14, 240);
      rect(x + w * 0.18, y - h - 16 * s, 6 * s, 14 * s, 2);

      // tiny window glow
      if (b.glow > 0.55) {
        fill(245, 220, 160, 40 + b.glow * 35);
        rect(x - 10 * s, y - h + 8 * s, 8 * s, 8 * s, 2);
```

```
        rect(x + 2 * s, y - h + 10 * s, 7 * s, 7 * s, 2);
      }
    } else {
      // silo
      noStroke();
      fill(9, 11, 15, 245);
      const w = 18 * s;
      const h = (44 + b.v * 22) * s;
      const x = b.x;
      const y = b.y;

      rect(x - w / 2, y - h, w, h, 7);

      // cap
      fill(7, 9, 13, 245);
      arc(x, y - h, w, w, PI, TWO_PI);

      // tiny highlight
      fill(200, 220, 240, 10 + b.glow * 10);
      rect(x - w * 0.15, y - h + 6 * s, w * 0.18, h - 12 * s, 6);
    }
  }

  pop();
}

drawNear() {
  push();
  this.drawGrass();
  this.drawTrees();
  this.drawCoinsForKind("tree");
  pop();
}

drawTrees() {
  for (let t of this.trees) {
    const sway = sin(frameCount * 0.01 + t.phase) * 2.5;
```

```
    stroke(12, 14, 18, 210);
    strokeWeight(3);
    line(t.x, t.y - 8, t.x + sway, t.y - 70);

    noStroke();
    fill(10, 12, 16, 240);
    ellipse(t.x + sway, t.y - 82, 36, 24);
    ellipse(t.x + sway + 16, t.y - 76, 22, 16);
    ellipse(t.x + sway - 14, t.y - 74, 20, 14);
  }
}

drawCows() {
  push();
  for (let i = 0; i < this.cows.length; i++) {
    const c = this.cows[i];
    const bob = sin(frameCount * 0.015 + c.idle) * 1.2;

    push();
    translate(c.x, c.y + bob);
    scale(c.flip * c.s, c.s);

    noStroke();
    fill(20, 24, 32, 235);
    ellipse(0, -18, 56, 30);
    ellipse(34, -26, 20, 16);

    fill(24, 28, 38, 235);
    ellipse(42, -24, 10, 8);

    fill(16, 18, 24, 240);
    rect(-18, -6, 6, 18, 2);
    rect(-6, -6, 6, 18, 2);
    rect(8, -6, 6, 18, 2);
    rect(20, -6, 6, 18, 2);

    stroke(18, 20, 26, 220);
    strokeWeight(2);
```

```
    noFill();
    const wag = sin(frameCount * 0.02 + c.idle) * 4;
    beginShape();
    vertex(-28, -22);
    quadraticVertex(-40, -16 + wag, -36, -6);
    endShape();

    noStroke();
    fill(28, 34, 46, 120);
    ellipse(-6, -20, 16, 10);

    pop();
  }
  pop();
}

drawGrass() {
  push();
  strokeWeight(2);

  for (let i = 0; i < this.grass.length; i++) {
    const g = this.grass[i];
    const sway = sin(frameCount * 0.02 + g.phase) * 3;

    stroke(22 * g.tint, 28 * g.tint, 34 * g.tint, 170);

    const baseY = g.y - 6;
    const baseX = g.x;

    for (let b = 0; b < g.clump; b++) {
      const ox = (b - (g.clump - 1) / 2) * (4 * g.s);
      const hh = (10 + (b % 3) * 6) * g.s;
      const bend = sway + ox * 0.2;
      line(baseX + ox, baseY, baseX + ox + bend, baseY - hh);
    }
  }

  pop();
```

```
}

drawFloaters() {
  push();
  noStroke();
  for (let p of this.floaters) {
    const tw = sin(frameCount * 0.02 + p.phase) * 0.4 + 0.6;
    fill(190, 210, 230, p.glow * tw);
    circle(p.x, p.y, p.r * (0.8 + tw));
  }
  pop();
}

// ---------- Moon ----------
drawMoonParallax(camX) {
  push();

  const r = this.moonR;
  const my = 78;
  const baseX = 86;
  const parallax = 0.18;

  let mx = baseX - camX * parallax;

  const wrapW = width + r * 4;
  mx = ((mx % wrapW) + wrapW) % wrapW - r * 2;

  noStroke();
  fill(210, 220, 240, 16);
  circle(mx, my, r * 2.35);

  fill(215, 225, 240, 235);
  circle(mx, my, r * 2);

  stroke(40, 50, 70, 85);
  strokeWeight(2);
  noFill();
```

```
  const saved = this.moonSeed;
  for (let k = 0; k < 3; k++) {
    beginShape();
    for (let a = 0; a <= TWO_PI + 0.001; a += 0.22) {
      const nx = noise(saved * 0.00001 + k * 10.0, a * 0.6);
      const wob = (nx - 0.5) * 6;
      const rr = r + wob + k * 1.2;
      vertex(mx + cos(a) * rr, my + sin(a) * rr);
    }
    endShape();
  }

  stroke(70, 85, 110, 70);
  strokeWeight(1.5);
  fill(200, 210, 230, 120);
  this.crater(mx - 14, my - 6, 10, 7);
  this.crater(mx + 12, my + 10, 14, 10);
  this.crater(mx + 6, my - 18, 8, 6);

  stroke(90, 105, 130, 55);
  strokeWeight(1);
  for (let i = 0; i < 18; i++) {
    const t = i / 17;
    const x1 = mx - r + t * (r * 1.05);
    const y1 = my + r * 0.35 + t * 10;
    line(x1, y1, x1 + 14, y1 + 8);
  }

  pop();
}

crater(cx, cy, w, h) {
  push();
  translate(cx, cy);
  noFill();
  for (let k = 0; k < 2; k++) {
    beginShape();
    for (let a = 0; a <= TWO_PI + 0.001; a += 0.35) {
```

```
      const wob = (noise(this.moonSeed * 0.00002 + k * 20.0, a * 0.8) -
0.5) * 2.2;
      const rx = w * 0.5 + wob;
      const ry = h * 0.5 + wob * 0.7;
      vertex(cos(a) * rx, sin(a) * ry);
    }
    endShape();
  }
  pop();
}

// ---------- Screen overlays ----------
drawBreathOverlay() {
  const breath = sin(this.breathT * 0.35) * 0.5 + 0.5;
  const alpha = 22 + breath * 26;

  noStroke();
  fill(90, 140, 170, alpha);
  rect(0, 0, width, height);
}

drawVignette() {
  push();
  noFill();
  for (let i = 0; i < 18; i++) {
    const t = i / 17;
    const a = 18 * t;
    stroke(0, 0, 0, a);
    const m = t * 70;
    rect(m, m, width - 2 * m, height - 2 * m, 28);
  }
  pop();
}

drawGrain() {
  push();
  blendMode(BLEND);
  tint(255, 18);
```

```
  image(grainLayer, 0, 0);
  noTint();
  pop();
}

// ---------- Helpers ----------
sampleGroundY(x) {
  const n1 = noise(x * 0.0013, 100.0);
  const n2 = noise(x * 0.0035, 200.0);
  return 240 + n1 * 210 + n2 * 45;
}
}
```